Net-Z™

User's Guide

PROKSIM SOFTWARE INC

Copyright © 1998-2001 Proksim software Inc. All rights reserved.

Trademarks

Net-Z™ is a trademark of Proksim Software Inc.

Unified Modelling Language™ is a trademark of the Object Management Group.

Acrobat® Reader is a registered trademark of Adobe Systems Incorporated.

ActiveSync®, DirectX®, eMbedded Visual C++®, Visual C++®, and Windows® are registered trademarks of Microsoft Corporation.

PlayStation® is a registered trademark of Sony Computer Entertainment Inc.

All other trademarks are the property of their respective owners.

Publication History

July 2000	First release 1.0 (Draft version)
November 2000	Updated for 1.5
August 2001	Updated for 2.0 RC1

Proksim Software Inc.
433 place Jacques-Cartier
Suite 200
Montréal, QC, H2Y 3B1
Canada

Phone: (514) 940-9090
Fax: (514) 940-9091

Email: support@proksim.com or info@proksim.com
World Wide Web: http://www.proksim.com

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 The Net-Z Software Developer Kit 1

1.2 System Requirements and Platforms Supported 2

1.3 How to Use this Manual 2

1.4 Intended Audience 3

1.5 Typographical Conventions 3

2 AN INTRODUCTION TO NET-Z 5

2.1 Key Net-Z Concepts 5

High-Level Abstraction 5

Descriptive Approach 6

Distributed Objects Concepts 6

2.2 Net-Z Features 7

3 NET-Z DATA DEFINITION LANGUAGE (DDL) 9

3.1 DDL File Syntax 9

3.1.1 Duplicated Object Classes 9

3.1.2 Datasets 11

3.1.3 Duplication Spaces 13

3.1.4 Remote Method Calls 16

3.1.5 Actions 17

3.2 DDF File Syntax 18

3.3 DDL Compiler Usage 19

3.4 Compiler Output and Class Generation 20

Delegation 26

3.5 Developer Responsibilities 27

4 IMPLEMENTING NET-Z FEATURES 29
4.1 Session Management 29
4.2 Duplicated Object Classes 31
4.3 Well-known Objects 35
4.4 Datasets 36
4.4.1 Data Propagation 37
4.4.2 Hierarchical Messaging 39
4.5 Duplication Spaces 40
4.5.1 PS Match Function 43
4.5.1 Cell Match Function 46
4.5.3 Calling Match Functions 48
4.5.4 Multiple Duplication Spaces 49
4.6 Remote Method Calls 51
4.7 Actions 53
4.8 Load Balancing 55
4.8.1 Object Load Balancing 56
4.8.2 Sub Classing Load Balancing 57
4.9 Fault Tolerance 59
4.10 Dead Reckoning 60
4.10.1 Constant Extrapolation Error 62
Current Extrapolation Error 62
Error Tolerance 63
4.10.2 Distanced-based Extrapolation Error 63
Current Extrapolation Error 64
Error Tolerance 64
Calculation of Distance 66
Example of the Implementation of Distance-based Dead Reckoning 68
4.10.3 Quality Factor 71
4.10.4 Fine-tuning Dead Reckoning 75
Dataset Updates 75
Postdate Updates 78
Continuity Breaks 79

Local Corrections 80
4.11 Call Contexts 81
4.12 User-defined Contexts 83
4.13 Iterators 83
4.14 Net-Z System Operations 84
4.14.1 Operation Sequences 85
4.14.2 Operation locks 90
4.15 Time Management 92
4.15.1 Local Clock 92
4.15.2 Session Clock 92
Fine-tuning Session Clock Synchronization 92
4.15.3 Chrono 94
4.15.4 Pacer 94
4.16 Plug-ins 95
4.17 Performance Fine-tuning 96
4.17.1 Performance Counters 96
4.17.2 Emulation Device 97
4.18 System Errors 98
4.19 System Traces 99

5 HOW TO BUILD A NET-Z GAME 101
5.1 Samples 101
How to Run a Sample 102
Joining a Session 102
5.1.1 BasicSample 103
5.1.2 EnumerateLANSessions 109
5.1.3 Inheritance 110
5.1.4 ActionDungeonZ 111
5.1.5 DungeonZ 111
5.1.6 Delegation 114
5.1.7 Basic3DSample 115
5.1.8 RoomZ 116
RoomZ DDL File 118

5.1.9 SphereZ 119
5.2 Game Scalability 123
5.3 Network Configuration Options 125

6 TROUBLESHOOTING 127
6.1 DDL Compiler Errors 127
6.2 Known Bugs 129

7 GLOSSARY 131

8 GRAMMAR 135
Net-Z Keywords 135
Lexical Conventions 135
DDF Grammar 136
DDL Grammar 136

9 TECHNICAL SUPPORT 141
9.1 Contact Details 141

10 INDEX 143

1 INTRODUCTION

Net-Z is a powerful, robust, and innovative solution to the networking needs of a variety of ephemeral games. Created by Proksim Software, Net-Z can simplify and accelerate the development of your game by allowing network developers to focus on game design issues, rather than network problems.

The Net-Z philosophy is very different from traditional message-based networking solutions. Its high-level abstraction, descriptive approach, and distributed objects model provide outstanding features such as transparent fault tolerance, automatic load balancing, late join, dead reckoning, time synchronization, monitoring, and platform independence. These technical features, and overall ease of use, combine to make Net-Z the ideal solution for adding networking to most ephemeral games.

1.1 The Net-Z Software Developer Kit

The Net-Z software developer kit (SDK) includes a Net-Z CD and this User's Guide. Contained on the CD is:

- The Net-Z software,
- Two white papers, entitled Technical Overview and Duplication Spaces, in PDF format,
- An online help manual,
- The Net-Z Atlantis demo that illustrates various features of Net-Z,
- A number of sample programs that illustrate how to program the various functions of Net-Z,
- Additional resources: Acrobat Reader, DirectX 7.0a, and Winsock2 for Windows 95, which are required to be installed on your station to enable either the documentation to be viewed or some of the samples to be run.

1.2 System Requirements and Platforms Supported

The installation of Net-Z requires 25MB of free hard disk space. The platforms supported by Net-Z are listed below and each platform is able to communicate with each other platform. Our highest levels of testing are performed on Windows 98/2000/ME.

- Windows millennium
- Windows 2000 (i386)
- Windows 98
- Windows NT 4.0 (i386 with ServicePack 4)
- Windows 95 (with Winsock 2 update installed)
- Windows CE Pocket PC (with an Intel Strong Arm or Hitachi SH3 processor)
- PlayStation2
- Linux

The minimum system requirements for the installation of the Atlantis demo program are:

- Pentium 200MHz
- Windows 95/ 98/ 2000
- 3D Accelerator with 8MB RAM
- 2MB of free hard disk space
- DirectX 7.0a

1.3 How to Use this Manual

This User's Guide details how to use and take full advantage of the Net-Z architecture. For a description of all the methods of the Net-Z API, the Net-Z grammar, and other reference material, refer to the online help manual.

Throughout this manual the implementation of various features of Net-Z, except for dead reckoning, is explained in the context of the *RoomZ* sample program, which is included on the Net-Z CD. A description of the sample is given in §5.1.8 as well as the complete DDL for easy reference. Dead reckoning is explained via the *SphereZ* sample program, which is also included on the Net-Z CD. A description of the sample is given in §5.1.9.

1.4 Intended Audience

This manual is written for programmers who wish to use Net-Z as the network layer for their game. A good background in C++ and object oriented programming is assumed.

1.5 Typographical Conventions

The following conventions are used throughout this manual:

`Constant width` is used for Net-Z class names and functions, and pieces of code that appear throughout the text.

Italic is used for names of parameters, filenames, and sample programs.

The name of each class and function begins with an uppercase letter and all compound names use uppercase to begin each embedded word. All class diagrams use the conventions of the Unified Modelling Language (UML).

In the code of the samples _t (*e.g.* _tprintf, _T, _getts, _ttoi, etc.) is used in the context of string handling to avoid difficulties due to the differences in character storage between UNICODE and ANSI. For more information on this refer to the section on string handling routines in any good C++ book.

References to parts of this manual are of the form §2.3.4 (Chapter 2, Section 3, Subsection 4).

2 AN INTRODUCTION TO NET-Z

Net-Z is a powerful, robust, and innovative tool kit designed to meet your networking needs. Net-Z can simplify the development of small and ephemeral online games by allowing network developers to focus on game design issues, rather than communication and network problems.

2.1 Key Net-Z Concepts

Net-Z offers a new approach to multi-player game design and development by offering an alternative to the traditional message passing approach used for creating network games. The Net-Z philosophy is based on three fundamental concepts. Firstly, it provides high-level abstraction in which game developers can easily express their programming talents. Secondly, it uses a descriptive approach that allows the game designer take an active part in designing the multi-player game and finally, it is based on a distributed object system speci?cally targeted at game development. These concepts are explained in detail below.

High-Level Abstraction

You won't find any reference to packets and message passing in Net-Z because these concepts are not part of the game design process. Instead, Net-Z deals with game objects, a natural way of thinking for most game designers and developers.

Many developers are not used to dealing with the variety of problems that distributed computing introduces. Therefore, Net-Z provides developers with the means to deal with the most common distributed issues of this complex field of computer science. This considerably reduces the time to market for multiplayer games.

Some developers believe that high-level abstraction leads to a lack of performance and flexibility. Net-Z was designed with these concerns in mind and by the use of C++ programming advanced features are implemented efficiently. High-level languages, such as C or C++, have historically proven to be very efficient and few applications now need to be written in assembly language. Due to the efficiency of high-level languages their use can lead to an increase in productivity, which is an important consideration in today's competitive markets.

Net-Z uses high-level abstraction to facilitate the provision of features, to the developer, in a more transparent manner. Load balancing and fault tolerance are examples of Net-Z features where high-level implementation frees the developer of these two complex issues. Unless a high level of abstraction is used to encapsulate concepts such as these they cannot be performed transparently.

Descriptive Approach

The strength of Net-Z comes from its ability to describe data, which needs to be sent over the network, using the Net-Z Data De?nition Language (DDL). The DDL describes the data structures and game objects that are required to be managed by Net-Z. Using such an approach has several signi?cant advantages.

Firstly, the challenges associated with game networking are effectively addressed by describing data in the early phases of game development. Instead of thinking about sockets and IP protocols, Net-Z requires game designers to understand which data really needs to be duplicated on all stations.

Secondly, once data has been described, multiplayer aspects of the game can be rapidly prototyped with a fully functional networking layer tailored to the game's needs. By modifying data descriptions in the DDL ?les, a game designer can tweak performance and observe the results immediately.

Furthermore, an added advantage of using Net-Z is platform independence of game data, since the DDL compiler may perform marshalling, unmarshalling, and complex data structure streaming automatically. Generated code can be optimized to minimize buffer copies or take advantage of platform speci?c optimizations.

Finally, using Net-Z leads to a better understanding, by game designers, of the multiplayer aspects of the game. It is very easy for them to communicate with game developers via this high level description.

Distributed Objects Concepts

Object oriented programming is a proven way to develop reusable software components. Distributed object computing is a natural extension of this paradigm and is widely used in today's distributed applications. Multiplayer game development is no exception and bene?ts from the advantages of such an approach. Net-Z leverages this paradigm by providing a custom implementation tailored to meet the game development field.

From the point of view of distributed object computing, a game is a collection of various game objects such as characters, avatars, non-player characters, walls, and weapons. These objects need to be distributed and duplicated across all the stations participating in a game. Furthermore, the attributes contained within these objects, such as position and colour, need to be coherent and propagated to other participants upon modi?cation.

In a game developed using Net-Z, every game object is automatically replicated to all other stations giving each participant a global view of the game world. Once an object is published, it is automatically replicated to the other stations and constantly updated according to the relevant update policy.

2.2 Net-Z Features

The Net-Z philosophy is very different from traditional message-based networking solutions. Its high-level abstraction, descriptive approach, and distributed objects model provide outstanding features such as:

- Transparent fault tolerance. A game will recover from a fault without disturbing the participants in the game. In a Net-Z game stations may fail but duplicated objects will continue to exist as if nothing had happened.
- Load balancing with minimal programming effort. Net-Z can evenly distribute the workload between the stations on the network. As the developer defines the load balancing policies any work quantity, such as CPU load, bandwidth, memory load, and duplicated objects, can be load balanced.
- The use of data extrapolation algorithms and dead reckoning reduces network traffic. This leads to optimal game performance for the available bandwidth.
- Net-Z provides monitoring tools to help fine-tune game performance. Various network conditions can be simulated to test game performance under different conditions and by monitoring various parameters a game can be fine-tuned. The default settings of Net-Z will be suitable for most games but they can be easily overridden to optimize your particular game.
- Scalability. The implementation of features such as duplication spaces and hierarchical updates allows games to be deployed on a small network of a few nodes up to very large scale networks, while still maintaining superior performance and reliability.
- Time synchronization across all the stations connected to the session to ensure that events on each station remain synchronized.

- Cross-platform. It is possible to integrate games created with Net-Z on several different platforms.
- Plug-in architecture. It is effortless to implement a custom transport or security protocol via plug-ins.

These technical features and overall ease of use combine to make Net-Z the ideal solution for adding networking to most distributed games.

3 NET-Z DATA DEFINITION LANGUAGE (DDL)

The Net-Z Data Definition Language (DDL) contains details of the data that is required to be sent over the network. In the DDL the developer describes the data structures and game objects that are required to be managed by Net-Z.

For a single game, the DDL compiler expects one Duplicated object class Definition File (DDF) and a variable number of DDL files. The syntax of these two types of files is explained in this chapter, as well as how to use the DDL compiler and the class generation performed by the DDL compiler. In addition, details of the information that you are required to provide are discussed.

3.1 DDL File Syntax

As in most object-oriented languages, the Net-Z DDL is based on classes. The three most important classes are the DuplicatedObject class, the DataSet class, and the DupSpace class, which define all the objects that belong to the game, their related datasets, and the duplication spaces that they are a part of. All duplicated objects, datasets, duplication spaces, and actions, which are remotely called methods, are declared in the DDL file.

Comments within the DDL code are indicated by two backslashes, //, at the beginning of the line of comment. For example,

```
// This is a comment
```

3.1.1 Duplicated Object Classes

In a multi-player game, the objects of the session need to be duplicated across the network in order for the players to see each other. Object duplication is the mechanism used by Net-Z to achieve this. Net-Z duplicated objects are declared using duplicated object classes (DOClasses). A duplicated object is an instance of a duplicated object class, and multiple instances of the same duplicated object class may be created.

The relationship between a duplicated object and a duplicated object class is very similar to the relationship between an object and a class in standard C++ programming. The distinction between C++ objects and Net-Z duplicated objects is apparent when an object is instantiated. In Net-Z, whenever an

instance of a duplicated object class is created on a given station, copies of the instance will automatically be created on all remote stations that require that object. The local duplicated object is called the duplication master and is the controlling instance of the object, while the others are its duplicas.

A duplicated object class is declared in a DDL file where the relevant dataset, publisher, subscriber, cell, RMC, and action declarations for each duplicated object are made. These important concepts are defined below and will be explained in more detail later.

Dataset	A dataset describes data that needs to be duplicated to other stations.
Publisher	A publisher is an object that can be discovered by other objects within a duplication space.
Subscriber	A subscriber is an object that can discover other objects within a duplication space.
Cell	A cell is a subset of a duplication space that is created to enable the process ing of the matching computations to be distributed among the connected stations. If cells are not declared by the developer, they are automatically implemented by Net-Z according to the demands of the game.
RMC	A remote method call (RMC) is a method called remotely on a duplicated object that may return a value.
Action	An action is a method called remotely on a duplicated object that cannot return a value.

To declare a duplicated object in the DDL file, the following syntax is used:

```
doclass ObjectName [: inheritance_class] {
     [declarations];
};
```

where `[inheritance_class]` refers to the name of another duplicated object class from which the class ObjectName inherits and `[declarations]` are all the relevant dataset, duplication space, RMC, and action declarations for the class.

Net-Z contains a built-in class `GlobalDO` from which user-defined classes may inherit. Each `GlobalDO` class is known globally across the session and is guaranteed to be duplicated to all stations participating in the session. If no duplication spaces are defined in a game, by default all objects inherit from the `GlobalDO` class and are duplicated to all stations. However, when duplication spaces are defined if you wish the objects of a particular class to be known across the session it is necessary to define that the class inherits from the `GlobalDO` class. Duplication spaces and use of the `GlobalDO` class are detailed in §4.5.

Part of the *RoomZ* DDL is shown below. Five duplicated object classes are declared in the DDL: Delegate, Sage, Moderator, Room, and SageBalancing. Each of these duplicated object classes may contain datasets, duplication space declarations, remote method calls, and actions. The datasets, duplication space declarations, remote method calls, and actions of the Sage and Moderator classes inherit from those de?ned in the Delegate class. The Room class inherits from the `GlobalDO` class and therefore, this class will be duplicated to all participants in the session. The SageBalancing class is a user-defined load balancing class that inherits from the LoadBalancingDomain. More information on load balancing can be found in §4.8.

```
doclass Delegate {
     [declarations];
};

doclass Sage : Delegate {
     [declarations];
};

doclass Moderator : Delegate {
     [declarations];
};

doclass Room : GlobalDO {
     [declarations];
};

doclass SageBalancing : LoadBalancingDomain {
     [declarations];
};
```

Each duplicated object class that you declare in the DDL file must also be declared in the DDF file, as detailed in §3.2. In addition, for each class that you define you must write the source and header files and then implement the class, the details of which are discussed in §3.5 and §4.2 respectively.

3.1.2 Datasets

The syntax used to define a dataset in the DDL is as follows:

```
dataset DatasetName {
     Dataset attributes;
}[Update Policy];
```

where *`Dataset attributes`* can be any number of simple data types and `[Update Policy]` can be any valid combination of the update qualifiers used by Net-Z. The supported data types for variables declared in the DDL are: *char, uchar, short, ushort, long, ulong, longlong, ulonglong, int, uint, float, double, bool, dohandle*, and an *array* of any of these types. The special data types *string*

and the containers *vector*, *list*, and *queue* are also supported. In the case of a *string*, its length is variable and it is duplicated up to the terminating null character. It should be noted that a dataset sent over an unreliable channel cannot contain more than 3584 bytes of data.

An update policy consists of one or more update qualifiers and specifies the manner in which a dataset is updated. The addition of an update policy provides information used by the DDL compiler to generate efficient update code. Also, by the use of the appropriate update qualifiers, bandwidth usage can be minimised. The update qualifiers used by Net-Z are detailed in Table 3.1. If no update policy is specified the default policy is `reliable`, `variable`, which means that the data is sent over a reliable channel and is assumed to be variable.

Continuing the *RoomZ* example started previously, in the DDL we define three datasets; StringInfo, Location, and SageKnowledge that contain the variables *Info*, *room*, and *Quote* respectively. As we always wish to know the current values of these datasets we send the data over a reliable channel. Since this is the default policy none is required to be specified.

```
dataset StringInfo {
     char Info[64];
};

dataset Location {
     dohandle room;
};

dataset SageKnowledge {
     string m_sQuote[512];
};
```

When a dataset is defined in the DDL it must be associated with a duplicated object. For example, to associate the dataset Location with the duplicated object class Delegate the following syntax is used:

```
doclass Delegate {
     Location m_dsLocation;
     // other declarations
};
```

As the duplicated object class Moderator inherits from Delegate, it will automatically include the dataset Location.

An example of the use of other update qualifiers is given in the *SphereZ* sample program. In this sample we have Sphere objects that are associated to the datasets Position and SphereData. The Position dataset is updated via an extrapolation filter and the SpereData dataset uses an upon request filter so that it is updated only when a request to do so is sent. The definitions of these two datasets in the DDL is as follows:

```
dataset Position {
     double x;
     double y;
     double z;
} extrapolation_filter, unreliable;

dataset SphereData {
     ushort m_usTexture;
} upon_request_filter;
```

The containers used in Net-Z function in a very similar manner to the containers of the C++ standard library. For information on the usage of containers refer to any good C++ programming book. The kinds of containers used in Net-Z are *vector*, *list*, and *queue* and the supported types for the containers are the same as those listed above for dataset variables, except for the type *string*. When declared in the DDL a container must specify that updates are sent over a reliable channel, which is the default, and they cannot specify the use of an extrapolation filter. For example, the dataset class MyContainer, which contains three containers and is associated to the duplicated object class Avatar, would be defined in the DDL as follows:

```
dataset MyContainer{
     vector<double> vec;
     list<dohandle> lst;
     queue<int> que;
} reliable;

doclass Avatar{
     MyContainer m_dsContainer;
};
```

When a dataset is declared in the DDL file it must be implemented by the user which is detailed in §4.4.

3.1.3 Duplication Spaces

Duplication spaces is one of the mechanisms used by Net-Z that enables a game to be scaled by giving the developer control over object duplication. When duplication spaces are implemented, the developer has the ability to decide where and how many duplicas of each object are published so that objects are only duplicated to the stations where they are needed. This facilitates the optimization of resource usage, such as the available memory and bandwidth, as each station only receives information about the objects it is interested in.

In a duplication space, specific duplicated objects are grouped together and are related by a user-defined function, which is termed the match policy. The duplicated objects within a duplication space are split into two categories: subscriber and publisher. An object in a duplication space must be

Update Qualifier	Description
`reliable`	Dataset updates are sent over a reliable channel. This qualifier is well suited for updates that are important, and for datasets that do not change very often. This is the default channel used if none is stated. *Incompatibilities*: `unreliable`
`unreliable`	Dataset updates are sent over an unreliable channel. Updates are transmitted faster than over a reliable channel, but delivery is not guaranteed. This qualifier is well suited for datasets that contain variables that change frequently where a missing update is not detrimental to the game. *Incompatibilities*: `reliable`
`variable`	The values of the dataset members will change. This is the default policy if none is supplied. The user must regularly call `DuplicatedObject::Update` for the dataset to be updated. *Incompatibilities*: `constant`
`constant`	The values of the dataset members are constant. Upon creation of the duplicated object the dataset values are sent to the object's duplicas, but the dataset is never required to be updated. *Incompatibilities*: `variable, buffered`
`buffered`	The values of the dataset members on the duplica are not directly updated. When an update message is received by the duplica, it stores the result in a buffer. The content of the buffer is copied to the dataset member variables when `DataSet::Refresh` is called. The buffer is automatically locked when data is being transferred to or from it. *Incompatibilities*: `constant`
`extrapolation_filter`	This qualifer indicates that a prediction algorithm is applied to the values of the dataset members. This update policy is well suited to datasets that change in a continuous way. When an extrapolation filter is used, the dataset is automatically buffered and therefore, the method `DataSet::Refresh` needs to be called to update the dataset members. If an extrapolation filter is specified but the error tolerance is not set then a default value of zero for the error tolerance is used. The valid data types are int, float, and double. *Incompatabilities*: `constant,upon_request_filter`
`upon_request_filter`	This qualifier indicates that dataset updates will be sent according to a user-defined scheme. Typically the user would call the method `DataSet::RequestUpdate` each time an update is required. It is used for variables that do not change often. *Incompatabilities*: `constant, extrapolation_filter`

Table 3.1: Net-Z update qualifiers.

assigned the role of a subscriber, a publisher, or both, and may belong to more than one duplication space. A subscriber is an object that can discover other objects within the duplication space, and a publisher is an object that can be discovered within a duplication space. More specifically, a subscriber will discover the duplica of a publisher on the station where the duplication master of the subscriber resides, and a publisher will create a duplica of itself on the station where the duplication master of the subscriber resides. The decision of whether a subscriber discovers an object and if the publisher publishes a duplica of itself depends on the `PSMatch` function, which is the specific match policy that relates publishers to subscribers. The `PSMatch` function is detailed in §4.5.1 and is a user-defined Boolean function that determines if a particular publisher and a particular subscriber correspond in that particular duplication space.

As the size of a session increases the number of computations required will increase. To reduce the resource usage on a particular station we can distribute the processing of the matching computations between many stations. To do this we divide the duplication space into cells, where each cell is responsible for a certain portion of the matching computations. A cell is a subset of a duplication space and typically groups publishers and subscribers of a similar "type". To determine if a particular publisher or subscriber belongs to a particular cell, the `CellMatch` function is used, which is described in §4.5.2.

When duplication spaces are integrated into a game it is necessary for the developer to define the duplication space and the publisher, subscriber, and cell objects in the DDL and to implement the relevant `PSMatch` and `CellMatch` functions. The syntax used to define a duplication space and its associated publisher, subscriber, and cell objects in the DDL is as follows:

```
dupspace DupspaceName;

doclass ObjectName {
     [publisher in DupspaceName];
     [subscriber in DupspaceName];
     [cell in DupspaceName];
};
```

Net-Z contains a built-in class `GlobalDO` from which user-defined classes may inherit. Each `GlobalDO` class is known globally across the session and is guaranteed to be duplicated to all stations participating in the session. If no duplication spaces are defined in a game, by default all objects inherit from the `GlobalDO` class and are duplicated to all stations. However, when duplication spaces are implemented if you wish the objects of a particular class to be known across the session it is necessary to define that the class inherits from the `GlobalDO` class. The syntax used in the DDL is as follows:

```
doclass ObjectName : GlobalDO {
     [declarations];
};
```

For example, in the *RoomZ* sample we define the duplication space RoomSpace and the relevant publisher, subscriber, and cell declarations as follows. We also define that the duplicated object class Room inherits from the `GlobalDO` class and therefore, it will be duplicated to all stations in the session.

```
dupspace RoomSpace;

doclass Delegate {
     publisher in RoomSpace;
     subscriber in RoomSpace;
     // other declarations
};

doclass Room : GlobalDO {
     cell in RoomSpace;
     // other declarations
};
```

When a duplication space is declared in the DDL file, it must be implemented by the user which is detailed in §4.5.

3.1.4 Remote Method Calls

The manner in which Net-Z remotely calls a method on a duplicated object is via Remote Method Calls (RMC) or via actions, which are described in §3.1.5. RMCs are methods that are called on one specific instance of an object, which may be a duplication master or a duplica, and they may return a value. RMCs give the developer significantly more control over the method than actions with respect to the manner in which they are called and the information the developer may obtain about the `State` and the `Outcome` of the call.

Remote method calls are declared in the DDL file and may contain parameters. The valid arguments are the same as those valid for datasets which are: *char, uchar, short, ushort, long, ulong, longlong, ulonglong, int, uint, float, double, bool, dohandle,* an *array* of any of these types, and *string.* A RMC cannot include a container as a parameter. The syntax used for a RMC is as follows, where `returntype` is the parameter type of the return value or `void` and `[argument list]` can be any number of arguments, similar to a C++ function declaration. The arguments of the input and output of the RMC may be specified via the `in`, `out`, or `in out` modifiers.

```
returntype RMCName( [Argument List] );
```

In the *RoomZ* example we set the current room that a Delegate is in via the RMC SetCurrentRoom. This method specifies a `DOHandle` as the in modifier.

```
doclass Delegate {
     void SetCurrentRoom(in dohandle hRoom);
     // other declarations
};
```

When a RMC is declared in the DDL file it must be implemented by the user which is detailed in §4.6.

3.1.5 Actions

The manner in which Net-Z remotely calls a method on a duplicated object is via Remote Method Calls (RMC), as described above, or via actions. Actions are methods that are called within a duplicated object class and are the easiest way to call a method on an object to ensure that the method is executed on all duplicas of the object. However, as opposed to a RMC an action cannot return a value.

Actions may be called on the duplication master or on a duplica of an object. When an action is called on a duplication master, the duplication master will then call the corresponding method on all of its duplicas. If an action is called on a duplica, that duplica will call the corresponding method on its duplication master. If necessary, the duplication master will then update the datasets of its duplicas.

Actions are declared in the DDL ?le and may contain parameters. The valid arguments are the same as those valid for datasets which are: *char, uchar, short, ushort, long, ulong, longlong, ulonglong, int, uint, float, double, bool, dohandle,* an *array* of any of these types, and *string.* An action cannot include a container as a parameter. The syntax used for an action is as follows, where `[argument list]` can be any number of arguments, similar to a C++ function declaration.

```
action ActionName( [Argument List] );
```

In the *RoomZ* example a participant sends a message to another participant via the action PrintMsg. This method is defined as an action as it can be called remotely and we do not wish to receive a return value. The particular Delegate to which the message is sent is specified via a `DOHandle` and the Boolean variable Whisper determines if the message is sent to all or only one Delegate in the same room as the sender. The declaration of the PrintMsg action in the DDL ?le is as follows:

```
doclass Delegate {
     // other declarations
     action PrintMsg(dohandle hSpeaker, string strMessage[128], bool
bWhisper)
};
```

When an action is declared in the DDL file it must be implemented by the user which is detailed in §4.7.

3.2 DDF File Syntax

The Duplicated object class Definition File (DDF) defines the duplicated object classes that belong to the game. These duplicated object classes then need to be declared in the DDL file. The DDF also defines global game information, such as the application title and well-known handles.

The application title defines the type of game. It is necessary to define an application title in the DDF as it is used to authenticate any station that tries to join the session. It is also included in the `SessionDescription`. The application title should not be confused with the session name, which is used to identify a particular instance of a type of game. The session name is defined when a session is created, via the method `Session::CreateSession` as detailed in §4.1. As opposed to the application title, it is not essential to define a session name. The syntax used to define the application title in the DDF is as follows:

```
gametitle "string";
```

Any duplicated object class declared in the DDL must be defined in the DDF using the following syntax:

```
define DOClassName;
```

The DDF is also used to declare well-known instances of a particular duplicated object class. When a duplicated object is declared as a well-known object, the DDL compiler declares and initializes a global variable of the type `WKHandle`. The implementation of well-known objects is detailed in §4.3 and they are declared in the DDF using the following syntax:

```
wellknown DOClassName aVariableName;
```

The following is the DDF for *RoomZ* that defines the application title and the duplicated object classes Room, Delegate, Moderator, Sage, and SageBalancing. When the session is created, the Lobby and SageBalancing classes are created as well-known instances of the Room and SageBalancing classes respectively so that when a participant joins the session he is guaranteed to discover both these objects.

```
gametitle "RoomZ sample";
define          Room;
define          Delegate;
define          Moderator;
define          Sage;
define          SageBalancing;
wellknown       Room g_hLobby;
wellknown       SageBalancing g_hSageBalancing;
```

The duplicated object classes Room, Delegate, Moderator, Sage, and SageBalancing must be declared in the DDL file. Since one instance of Room is well-known, in the file generated by the DDL compiler a global variable *g_hLobby* of the type `WKHandle` will be defined. Similarly, the global variable *g_hSageBalancing* of the type `WKHandle` will be defined. For each class defined in the DDF a variable of the type `DOHandle` will be generated by the compiler, which is used to refer to a duplicated object of a particular class. The `WKHandle` instance inherits from the `DOHandle` class.

3.3 DDL Compiler Usage

The syntax used to invoke the DDL compiler and to generate the code necessary to implement the objects and datasets of the session described in the DDL is as follows:

```
DDL.EXE [options] <filename> <-d filename>
```

where *<filename>* specifies the name of the DDL file and *<-d filename>* specifies the name of the DDF associated with the DDL file. The options that can be used with this command are detailed in Table 3.2 and the error and warning codes that may be returned by the DDL compiler are detailed in §6.1.

Switch	Description
-h	Displays the Help screen
-p <*HeaderFilename*>	This is the name of a file to be included at the beginning of all the generated .cpp files. This option is useful for projects with precompiled headers. For example, when building a Net-Z project that uses the precompiled header stdheaders.h, use the -p stdheaders.h switch when invoking the DDL compiler.
-v Verbose.	This switch is used to determine the size of the datasets. Note that a dataset sent over an unreliable channel cannot contain more than 3584 bytes of data.

Table 3.2: DDL compiler options.

For example, to compile the DDL file *RoomZ.ddl* the following syntax is used:

```
ddl.exe RoomZ.ddl -d RoomZ.ddf
```

3.4 Compiler Output and Class Generation

When a game built with Net-Z is compiled, the DDL and DDF files are used by the Net-Z DDL compiler to generate the DOCLASS, DATASET, and DUPSPACE classes and their relevant source and header files. From the DDF the compiler generates source and header files with the file names *DDFFileNameDDF.cpp* and *DDFFileNameDDF.h*. The compiler will also generate a source and header file, with the names *ClassNameDDL.cpp* and *ClassNameDDL.h*, for each duplicated object, dataset, and duplication space declaration made in the DDL. The generated classes and files, along with the user-implemented classes and code and the Net-Z library are then used by the C++ complier to generate the executable file. This is illustrated in Figure 3.1.

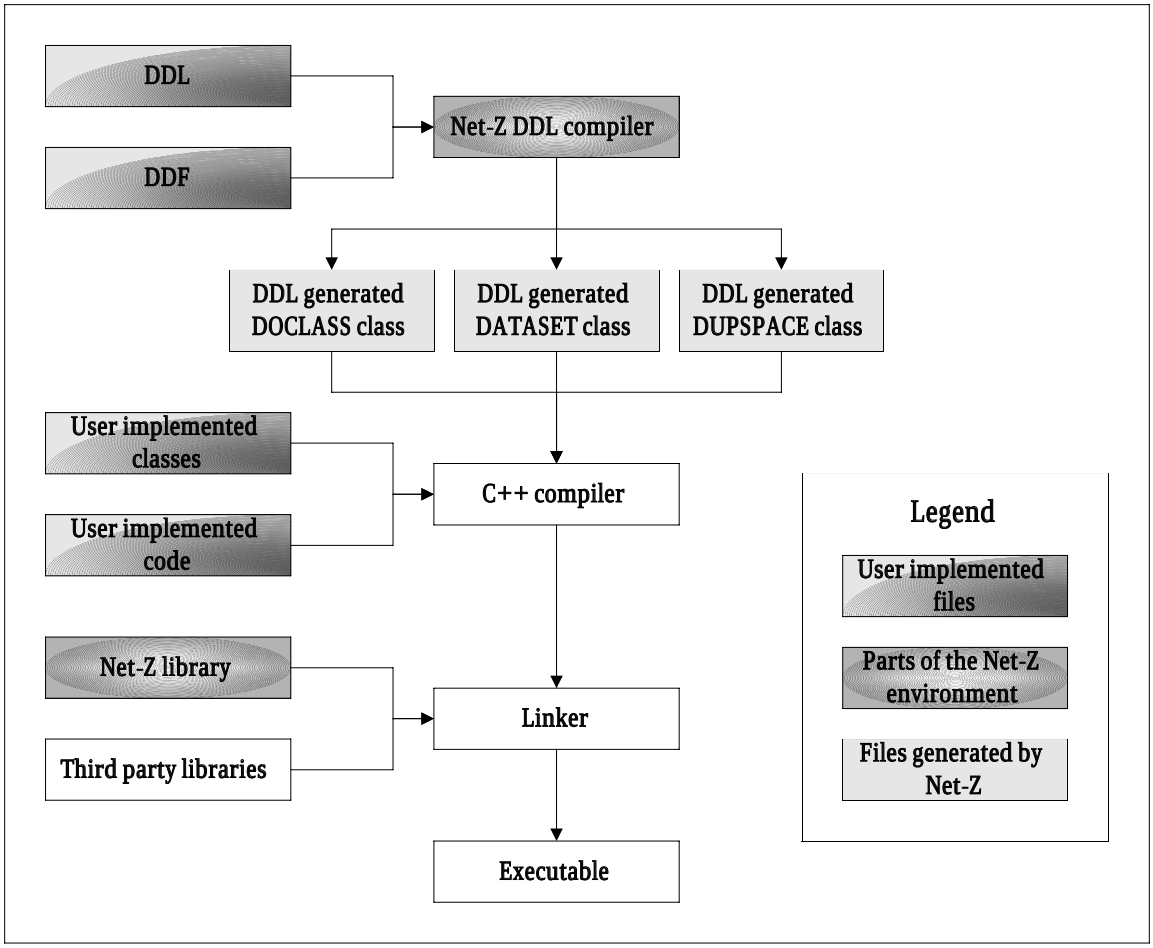


Figure 3.1: Executable file generation.

The Net-Z DDL compiler generates C++ classes corresponding to the description declared in the DDL file. The three main user implemented classes of Net-Z are the duplicated object, dataset, and duplication space classes. These user-implemented classes inherit from the classes generated by the DDL compiler, which in turn inherit from the Net-Z `DuplicatedObject`, `DataSet`, and `DuplicationSpace` classes. This class hierarchy is illustrated in Figure 3.2.

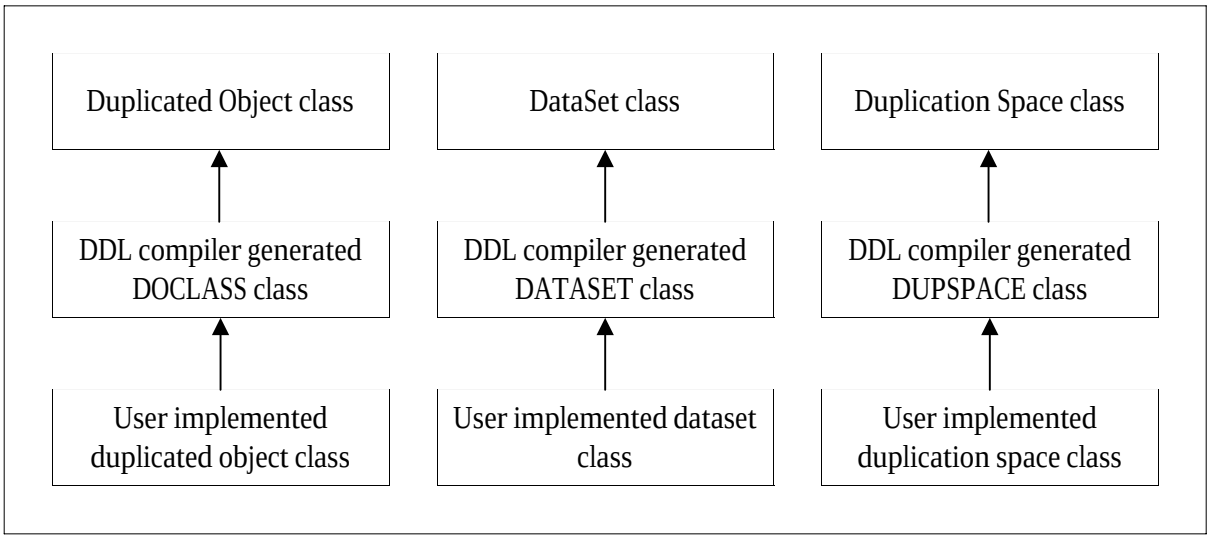


Figure 3.2: General class inheritance diagrams for duplicated objects, datasets, and duplication spaces.

In general, if a duplicated object class UserDOClass is specified in the DDL, the user must implement a class UserDOClass that inherits from the DDL generated class. This generated class is accessible via DOCLASS(UserDOClass), where DOCLASS is a macro that builds a name that refers to the generated class. Any class generated as a result of a DOClass declaration in the DDL file, will automatically ultimately inherit from the root class `DuplicatedObject`. For the `DuplicatedObject` class, as opposed to the `DataSet` class, inheritance between user-implemented classes can be specified in the DDL.

If a dataset UserDataSet is specified in the DDL, the user must implement a class UserDataSet that inherits from the DDL generated class. This generated class is accessible via DATASET(UserDataSet), where `DATASET` is a macro that builds a name that refers to the generated class. The DDL generated class automatically ultimately inherits from the root class `DataSet`. The variables declared as part of the class UserDataSet inherit from the variables declared in the generated class DATASET(UserDataSet). Similarly, if a duplication space UserDupSpace is specified in the DDL, the user must implement a class UserDupSpace that inherits from the DDL generated class. This generated class is accessible via DUPSPACE(UserDupSpace), where `DUPSPACE` is a macro that builds a name that refers to the generated class. The DDL generated class automatically ultimately inherits from the root class `DuplicationSpace`.

Neither duplicated objects, datasets, nor duplication spaces should be directly instantiated. Duplicated objects should be created via the method `DuplicatedObject::Create`. This method creates an instance of the class that can be published. Dataset and duplication space classes do not need to be directly instantiated because they are implemented automatically by Net-Z, by the DDL generated code, according to the declarations made in the DDL file. In the case of dataset classes, the DDL generated code implements the classes as member variables.

For duplicated objects, the class generated by the DDL compiler encapsulates all the network code. That is, how to instantiate a class and ensure it is discovered on all remote stations that require the object, how to ensure that when the variables of a duplicated object change its duplicas are updated appropriately, and how to remotely call the methods when a RMC or action is called. The user-implemented class declares the dataset variables, implements methods corresponding to the declared RMCs and actions, and it may also add any local variables or methods pertinent to the game.

Similarly, for datasets the class that the DDL compiler generates holds the dataset variables and is responsible for the dataset update, including marshalling, unmarshalling, and streaming of the data. If an extrapolation filter is used to update the dataset, a prediction model is also generated. The user-implemented class adds any local variables or methods that are applicable to the game. This local information makes no sense on a remote station and therefore, is not replicated. In the case of a duplication space, the class that the DDL compiler generates encapsulates all the network code relating to the space: how to instantiate the space and how to ensure that the cell, publisher, and subscriber objects are discovered on the correct stations. The user-implemented class implements methods relating to the cell match and publisher/subscriber match functions,

In previous sections we have defined the duplicated objects, datasets, duplication spaces, and actions relevant for the *RoomZ* sample. Therefore, the complete DDL file looks like this:

```
dupspace RoomSpace;

dataset StringInfo {
     char Info[64];
};

dataset Location {
     dohandle room;
};

doclass Delegate {
     Location m_dsLocation;
     StringInfo m_dsName;
     subscriber in RoomSpace;
     publisher in RoomSpace;
     action PrintMsg(dohandle hSpeaker, string strMessage[128], bool
bWhisper);
     void SetCurrentRoom(in dohandle hRoom);
};

dataset SageKnowledge {
     string m_sQuote[512];
};

doclass Sage : Delegate {
     SageKnowledge m_dsKnowledge;
};
doclass Moderator : Delegate {
};

doclass Room : GlobalDO {
     cell in Roomspace
     StringInfo m_dsTopic;
};

doclass SageBalancing : LoadBalancingDomain {
};
```

The class inheritance of duplicated objects, duplication spaces, and datasets, which results from the descriptions in the above DDL file, are given in Figure 3.3, Figure 3.4, and Figure 3.5 respectively. The user-defined duplicated object classes Sage and Moderator inherit from the generic Delegate class, which in turn inherits from the Net-Z `DuplicatedObject` class. Likewise, the user-defined Room class inherits from the Net-Z `GlobalDO` class, which in turn inherits from the `DuplicatedObject` class. Similarly, the user-defined duplication space RoomSpace and dataset classes StringInfo, Location, and SageKnowledge automatically ultimately inherit from the Net-Z `DuplicationSpace` and `DataSet` classes respectively.

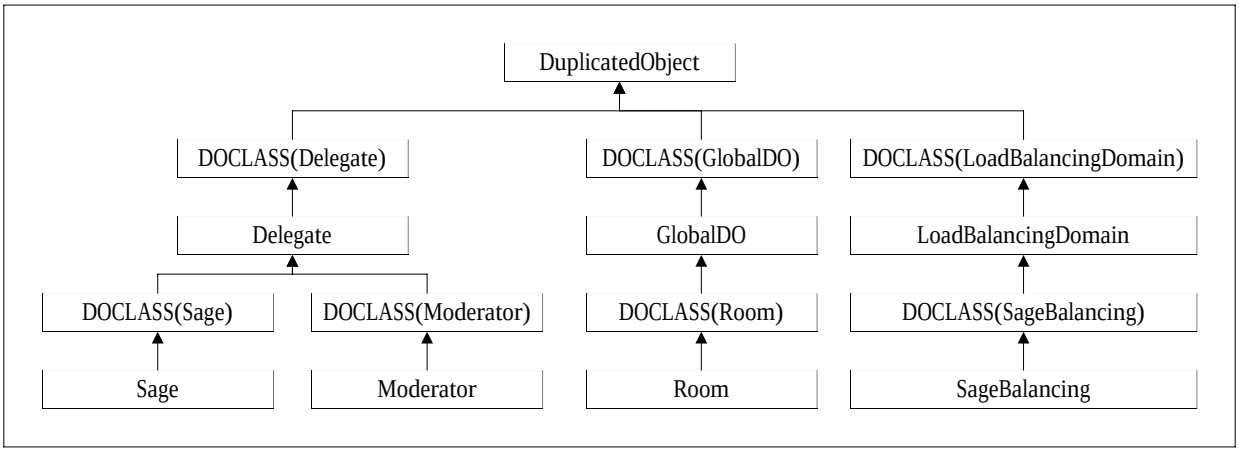


Figure 3.3: Class inheritance of duplicated objects.

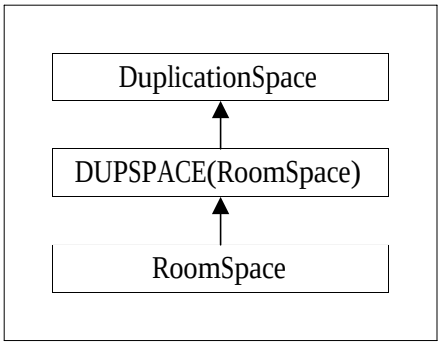


Figure 3.4: Class inheritance of duplication spaces.

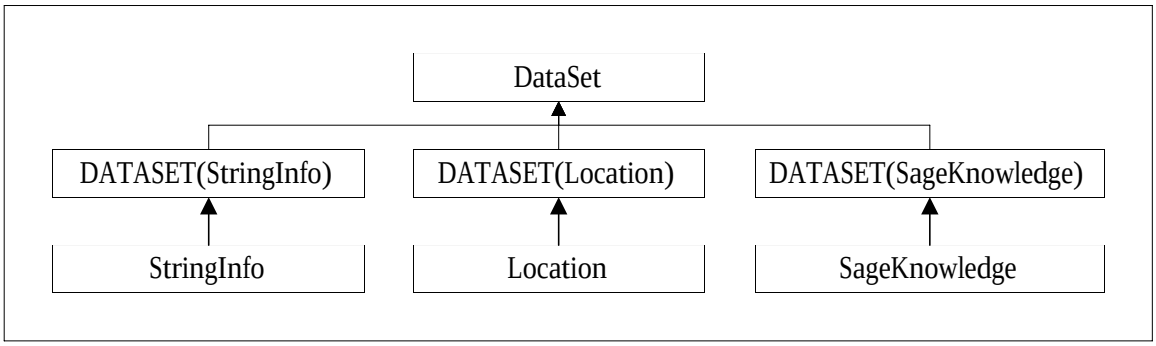


Figure 3.5: Class inheritance of datasets.

For each class that is defined in the DDL, the inheritance of the class is defined in the corresponding header file. In this example, the *Moderator.h* file defines that the Moderator class inherits from the DDL generated DOCLASS(Moderator), which is generated based on the definition in the DDL file. The generated file has the name *ModeratorDDL.h*. Similarly, the DDL generates the classes DOCLASS(Delegate), DOCLASS(Sage), DOCLASS(Room), and DOCLASS(SageBalancing) and the files *DelegateDDL.h, SageDDL.h, RoomDDL.h,* and *SageBalancing.h*. The Moderator class then inherits from the Delegate class and ultimately from the Net-Z `DuplicatedObject` class. The user must include the DDL generated header files before the corresponding user class is defined. The compiler also generates a source file corresponding to each defined class that needs to be linked to the other classes and compiled with the project. For instance, the source files generated by the DDL in this example are *DelegateDDL.cpp, ModeratorDDL.cpp, SageDDL.cpp, RoomDDL.cpp,* and *SageBalancingDDL.cpp*.

The duplication space RoomSpace is defined in the *RoomSpace.h* file. The duplication space class RoomSpace inherits from the DDL generated class DATASET(RoomSpace), which automatically ultimately inherits from the Net-Z `DuplicationSpace` class. The files generated by the DDL have the names *RoomspaceDDL.h* and *RoomSpaceDDL.cpp*. As mentioned above, the user must include the generated header files before the corresponding user classes are defined and the source files must be compiled with the project.

Similarly the dataset Location is defined in the *Location.h* file. The dataset class Location inherits from the DDL generated class DATASET(Location), which automatically ultimately inherits from the Net-Z `DataSet` class. The variables declared as part of the class Location inherit from the variables declared in the generated class DATASET(Location). The files generated by the DDL have the names *LocationDDL.h* and *LocationDDL.cpp*. Likewise, the DDL generates the files *StringInfoDDL.h, StringInfoDDL.cpp, SageKnowledgeDDL.h,* and *SageKnowledgeDDL.cpp* that correspond to the StringInfo and SageKnowledge datasets. As always, the user must include the generated header files before the corresponding user classes are defined and the source files must be compiled with the project.

When a duplicated object is instantiated, the member variable that corresponds to the DOClass declaration uses the developer's own class, rather than the one generated by the DDL compiler. For example, the DOCLASS(Location) generated by the DDL compiler includes a variable *m_dsLocation*, of the type Location, rather than DATASET(Location). Thus, code added in the Location class is available when the duplicated object is instantiated. This situation is illustrated in Figure 3.6.

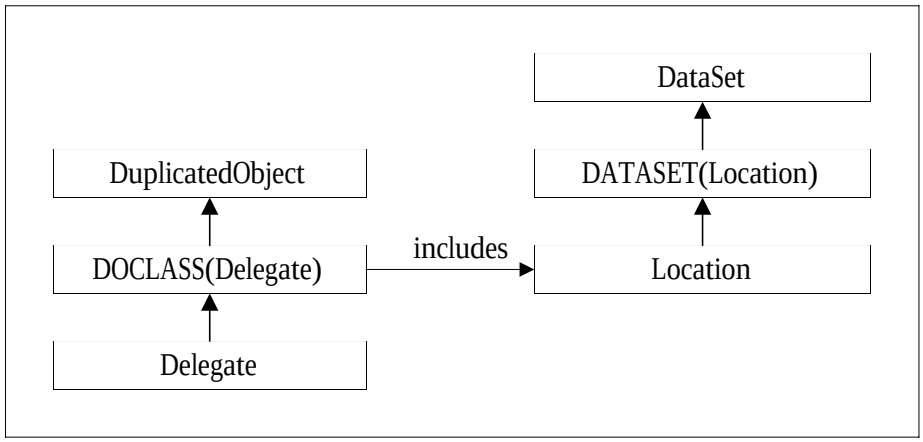


Figure 3.6: Relationship between duplicated object and dataset classes.

Delegation

Due to the class hierarchy generated by Net-Z, in some cases it may be difficult to integrate Net-Z into an existing class hierarchy and still maintain that existing hierarchy. There are two general methods to achieve this; via delegation or via multiple inheritance, the former of which is the preferred method. Delegation is when an object assigns a request to another object and the delegate carries out the request on behalf of the original object. Delegation allows two or more class hierarchies to operate in parallel, which enables an existing class hierarchy to be maintained and avoids the creation of two versions of the same class. It also makes it easy to compose behaviours at run-time and to change the way they are composed. On the other hand, multiple inheritance is when a class inherits from two or more base classes, which can lead to ambiguities when two base classes contain member functions with the same name.

A simple example of delegation is illustrated in the *Delegation* sample program where the physics and rendering of the objects are performed via an external class hierarchy. In this sample the class NetSphere delegates the physics and rendering to the Sphere class via the methods of the NetSphere class. The class hierarchy is described in Figure 3.7 and further details are given in §5.1.6.

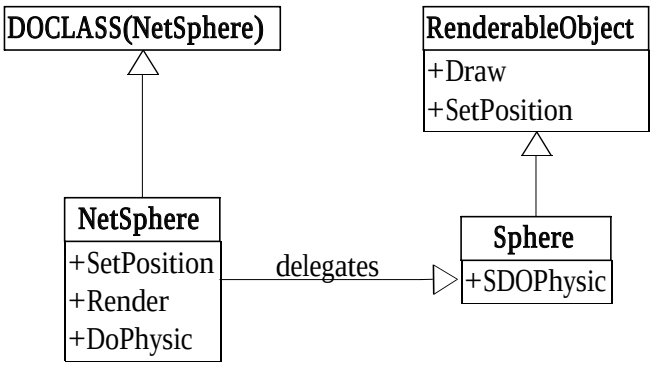


Figure 3.7: Class hierarchy of the Delegation sample program.

There are many ways to implement delegation, the details of which are too varied to be described here. Refer to any good book on design patterns for object-oriented programming for further details.

3.5 Developer Responsibilities

Before a game built with Net-Z can be compiled, several files are required to be written by the developer. It is the developer's responsibility to write the DDL and DDF files in the manner described previously in this chapter. In addition, for each `doclass`, `dupspace`, and `dataset` declaration made in the DDL a user implemented C++ source file and header file is required. Within the header file the inheritance and the variables of a class are defined, as well as any non-dataset variables that are used for local processing.

You also need to ensure that the files generated by the DDL compiler are correctly linked to the project. To do this you must make sure that all DDL generated header files (*ClassNameDDL.h*) are included before the class is defined, and that all DDL generated source files (*ClassNameDDL.cpp*, *DDFFileNameDDF.cpp*) are linked and compiled with the project. The only generated file that does not need to be included is the *DDFFileNameDDF.h* file as Net-Z automatically includes this file where needed.

You must implement any class that is declared in the DDL file and also the methods associated with any datasets, remote method calls, or actions and the match functions associated with any duplication spaces. The syntax used to declare the class in the header file for duplicated object, duplication space, and dataset classes is as follows:

```
class DuplicatedObjectName : public DOCLASS(DuplicatedObjectName) {
     // User specified RMCs, actions, and methods.
};

class DuplicationSpaceName : public DUPSPACE(DuplicationSpaceName) {
     // User specified match functions and methods.
};

class DataSetName : public DATASET(DataSetName) {
     // User specified methods.
};
```

For example, to declare the duplicated object class Delegate in the Delegate header file the following syntax is used:

```
class Delegate : public DOCLASS(Delegate) {
     // User specified RMCs, actions and methods.
};
```

You have the option of overriding the Net-Z system defaults by implementing the appropriate code in the source and header files. For example, you can decide if objects are fault-tolerant as well as customize the implementation of load balancing and duplication spaces. The details of how to change and optimize the default settings of Net-Z to suit your game are detailed in Chapter 4.

4 IMPLEMENTING NET-Z FEATURES

This chapter describes how to implement the various features of Net-Z. The implementation of the duplicated object, dataset, and duplication space classes and RMCs, actions and user-implemented methods is detailed, along with well-known objects, system operations, load balancing, and fault tolerance. The mechanisms used by Net-Z to measure time are described and finally, how to use the plug-in architecture, performance counters, and the fine-tuning devices is detailed.

4.1 Session Management

The status of a session is controlled via the `Session` class. The main function of this class is to enable a session to be created and joined. Once a station is part of a session, specific information about the station may be retrieved via the `Station` class.

A session is created by the session master via the method `Session::CreateSession`, which has the following signatures:

```
static bool Session::CreateSession(TCHAR *szSessionName, bool
bNetworked);
static bool Session::CreateSession(TCHAR *szSessionName);
static bool Session::CreateSession(unsigned short usWKPortNumber);
static bool Session::CreateSession(TCHAR *szSessionName, unsigned
short usWKPortNumber);
```

where *szSessionName* is the name of the session being created, *bNetworked* is a Boolean that determines if the session is networked or not, and *usWKPortNumber* is the IP port number where the connection is established. If no port number is supplied then Net-Z uses the default port number 9103. The definition of a session name is optional. If supplied, it is included in the session description and is useful for identification of the particular session when using the method `Session::FindLANSession`. A non-networked session is created by specifying the *bNetwork* parameter as false. A non-networked session is designed for a single user and therefore, once it is created no other participant may join the session. In addition, the session will not be enumerated by the call `Session::EnumerateLANSessions`.

For example, to create a version of the *RoomZ* application called *My RoomZ Session* the following code is used:

```
Session::GetInstanceHandle()->CreateSession("My RoomZ Session")
```

To join a session that is currently in progress the method `Session::JoinSession` is used. The following syntaxes may be used with this method:

```
bool Session::JoinSession(StationURL *pStationURL);
bool Session::JoinSession(TCHAR *szMachine);
bool Session::JoinSession(TCHAR *szMachine, unsigned short
usWKPortNumber);
bool Session::JoinSession(SessionDescription *pSessionDescription);
```

where *pStationURL* is a pointer to the URL of the station via which we wish to join the session, *szMachine* is the name of a station that is connected to the session, *usWKPortNumber* is the IP port number where the connection is established, and *pSessionDescription* is a pointer to the session description. The latter 3 syntaxes assume that the transport plug-in used is PRUDP.

For example, to join a session that is running on Bob's station we only need to specify the name of a station that is connected to the session, as given below. Alternatively we could specify the station URL, the IP address, or the session description.

```
NetZ *g_pNetZ = new NetZ;
Session::GetInstanceHandle()->JoinSession(_T("BobsStation.Acompany.com"));
```

If you know the specific session on the local area network (LAN) that you want to join, then you can find its session description via the method `FindLANSession`. Alternatively, you can enumerate all the sessions that are currently running on the LAN at the given port number by using the method `EnumerateLANSessions`. This method enumerates all the sessions that are running on the LAN at the supplied port number and calls a user-specified function on each of the session descriptions found. Typically the user-specified function would make a list of the session descriptions found. These methods must be called before the session is created or joined.

The information contained in the session description is held by the session master and only the session master can return this information. The initial session master is simply the station that initiates the session. If this station leaves the session Net-Z automatically elects another station as the new session master. The session description includes the application title, session name, the GUIDs of the compression and encryption plug-ins used, and the library version and build numbers of the session. This information can be accessed via the various methods of the `SessionDescription` class. Once you know a session description you can specify it as a parameter of the methods `JoinSession`, `FindLANSession`, and `EnumerateLANSession` of the `Session` class in order to join or find a given session.

Once the session master has created a session, it has control over the system status and may change the state of the system at any time via the method `Session::SetSystemState`. Only the session master may change the status of the session and therefore, this method is only valid on the session master. Any station can obtain the current system state via the call `Session::GetSystemState`. This method will return the system state as defined on the session master. The system state is one of paused, initializing, running, or terminating.

The `Station` class is used to return specific information about a station once the station is part of the session. The `DOHandle` and ID of the station can be retrieved respectively via the methods `GetLocalStation` and `GetStationID` of the `Station` class. The current state of the station can be obtained via the method `GetState`. The station state is one of: `Unknown`, `JoiningSession`, `CreatingSession`, `Participating`, `Leaving`, or `LeavingOnFault`. It can also be determined if the station is the local station or a peer station via the methods `IsLocal` and `IsAPeer`. The local station is typically the duplication master of the `Station` object, although this is not always the case. The peers of a given station depend on the implementation of the distributed broadcast. If a distributed broadcast is implemented the peers of a given station will consist of: the stations where the duplication masters of the duplicas on the local station reside, the stations where the duplicas of the duplication masters of the local station reside, and the station where the duplication master of the `RootMulticastGroup` object resides. In addition, if the local station contains a duplication master of a `MulticastGroup`, its peers will also include the stations of all its `MulticastGroup` members. In contrast, if a distributed broadcast is disabled, the peers will consist of all the stations in the session.

4.2 Duplicated Object Classes

The user must implement any duplicated object class that is declared in the DDL file via the following declaration:

```
class ObjectName : public DOCLASS(ObjectName) {
     // User specified variables and methods.
};
```

Once implemented, a duplicated object is created and published directly within its duplicated object class, via the methods `Create` and `Publish`, to instantiate the DDL compiler generated duplicated object classes for the object. Net-Z generates the methods `Create` and `Publish` for each duplicated object class declared in the DDL. The method `Create` creates the duplication master of an object and may be called before either of the methods `CreateSession` or `JoinSession` of the `Session` class, but until the object is published it can only be referenced using the pointer returned by the method `Create`. Once a duplicated object is published, it will automatically be dis-

covered by all remote stations, which require the object, connected to the session or any station that joins the session, and may be referenced using a `DOHandle` or `Ref`. The syntax to directly create and publish a duplicated object of the class UserClass is as follows:

```
UserClass* pDuplicatedObject = UserClass::Create();
     pDuplicatedObject->Publish();
```

For example, to create a delegate in the *RoomZ* example we first instantiate the `NetZ` object, create a session, and then create and publish the Delegate duplicated object, as shown below:

```
// Initialize NetZ.
NetZ* pNetZ = new NetZ();

// Create a session.
Session::GetInstanceHandle()->CreateSession();

// Create and publish a Delegate duplicated object.
Delegate* pNewDelegate = Delegate::Create();
pNewDelegate->Publish();
```

The Net-Z Application Programming Interface (API) contains methods to instantiate or destroy duplicated objects. When a duplicated object is instantiated, the local object is initialized as a duplication master, while the duplicated objects created on the remote stations are initialized as duplicas. If a participant joins a session that is already in progress, duplicas of all the existing duplicated objects that the participant requires will be automatically created on the new participant's station. The duplication of a duplicated object is controlled via several mechanisms that the developer can use to ensure that each station connected to a session only has copies of the objects that it really needs. This enables a game to be scaled and to operate efficiently, and ensures that the use of resources such as CPU and bandwidth are minimized.

The four mechanisms used to duplicate objects to the stations where they are required are: a duplication master may create a duplica on a particular station, a station may fetch a duplica of a specific duplication master, an object may be automatically duplicated to all participating stations, or duplication spaces may be used to dictate where objects should be duplicated to. The flexibility of the duplication model allows the developer to use any combination of these four mechanisms to control the duplication of objects across the stations connected to the session. If an object is required to be known globally, the easiest way to be certain that it is duplicated to all stations participating in the session is to ensure that the user-defined class inherits from the `GlobalDO` class. This guarantees that the object will be duplicated to each station connected to the session. If an object is not required to be known globally, then the developer must dictate the conditions under which an object is duplicated. This can be done by simply directly calling the method `DuplicatedObject::Create` to create a duplica on another station or `DuplicatedObject::FetchDuplica` to fetch a duplica from another station. In certain situations, this may be the simplest manner in which to dupli-

cate objects. However, if objects are simultaneously created, migrated, or deleted, due to operations such as fault tolerance and load balancing, object duplication can become very complex and the implementation of duplication spaces will usually present an easier solution to the management of object duplication. Duplication spaces are detailed in §4.5. In addition, you can migrate objects between stations via the methods `Emigrate` and `AttemptEmigration` of the `DuplicatedObject` class.

Duplicated objects are referenced across the network using a `DOHandle`, which refers to the local instance of the duplicated object. Net-Z generates a `DOHandle` when either a duplicated object is created or published, which ever occurs first after a session is created or joined. The `DOHandle` can be passed across the network either as a RMC or action argument or as a dataset. The method `DuplicatedObject::GetHandle` is used to return a `DOHandle` to the local instance of a duplicated object but may only be called after the object has been published. To determine if the object referenced by the handle is a well-known object the method `IsAWKHandle` is used. To obtain information about the class of the object that a particular handle refers to, the methods `IsA` and `IsAKindOf` can be used to determine if the object is a member of or inherits from a particular class. In addition, the method `DOHandle::GetClassNameString` may be used to obtain a string that represents the class name of the duplicated object to which a particular `DOHandle` refers.

To reference an object only on a local station a `Ref` may be created. This creates a direct reference to an object that may be used to call any method or group of methods on the object. The reference is created before the methods are performed and will be destroyed after the block has been exited. A block is a section of code delimited by a `{ }` pair. To ensure that we can never have a reference to an object that does not exist, Net-Z guarantees that the object to which we have a reference cannot be deleted until the block is exited. Therefore, it is important to ensure that a reference to an object is kept for as short a period of time as possible. There are two manners in which to create a reference to an object: by specifying a `DOHandle` to an object or by specifying a reference to an object. The default constructor may also be used and then an object assigned to the `Ref` via the assignment operator. The method `DORef::IsValid` should then be used to verify if the construction was performed correctly. The methods `DORef::Poll` and `DORef::Wait` may be used to verify if a reference is valid. `Wait` will continually check for validity until the time out is exceeded. The methods `DORef::GetDOHandle` and `DORef::GetDOPtr` can be used to return respectively the `DOHandle` used to construct the reference or a pointer to a specific duplicated object. For example, to perform a method locally on the object MyDelegate by specifying the `DOHandle` of the object the following code is used:

```
{
     Delegate::Ref refMyDelegate(hMyDelegate);
           if (refMyDelegate.IsValid()){
                 // call a method
                 refMyDelegate->method();
           }
}
```

Alternatively, we may create a reference by specifying a reference to an object in either of the two following manners:

```
// create a Ref using the default constructor and
// then assign a value to it
Delegate::Ref refToADelegate;
RefToADelegate=refMyDelegate;

// or create a Ref by specifying a Ref to an object
Delegate::Ref refToADelegate(refMyDelegate);
```

Within the `DuplicatedObject` class there are several methods that you can use to determine the properties of a particular object. To determine if a duplicated object is a duplication master or duplica the methods `IsADuplicationMaster` and `IsADuplica` can be used. To determine if the object is a well-known object the method `IsAWellKnownDO` can be used. To obtain information about the class a particular object belongs to, the methods `IsA` and `IsAKindOf` can be used to determine if an object is a member of or inherits from a particular class. In addition, the methods `GetClassID` and `GetClassNameString` will return respectively the ID and name of the object's class.

The deletion of a duplicated object is performed via the method `DuplicatedObject::DeleteMainRef`, which can only be called on a duplication master. This method deletes the main reference to the duplicated object held in the duplicated object store. If this was the only reference to the object then the destructor will be called and the duplication master and all of its duplicas will be deleted by this call. If there are other references to the object in existence then the destructor will not be called until the last reference is released. For example, if a reference to the object created via the `RefTemplate` exists, the object's destructor will not be called until this reference to the object is deleted. The method `DuplicatedObject::MainRefIsDeleted` may be used to determine if the main reference to the duplicated object has been deleted. The complete sequence of events that occurs when an object is deleted is described under the RemoveFromStore operation in §4.14.1. For example, if we wish to destroy the object MyDelegate, then the following code is used:

```
{
     Delegate::Ref refMyDelegate(hMyDelegate);
     refMyDelegate->DeleteMainRef();
}
```

When a station leaves the session, Net-Z calls the method `DuplicatedObject::ApproveEmigration` on all duplication masters on the station to determine if the objects should immigrate to another station and therefore, survive. If this method returns false for a duplicated object, then Net-Z will automatically delete the main reference to the object. Similarly, if a station leaves the session due to the occurrence of a fault then the method `DuplicatedObject::ApproveFaultRecovery` will be called on the duplicated objects on that station. If this method returns false for a duplicated object, then Net-Z will automatically delete the main reference to the object..

4.3 Well-known Objects

A well-known object is effectively a global object that is common to all stations that are participating in the session. Typically they are such things as the game environment and other objects that need to be discovered before a participant can partake in the session. Well-known objects are typically computer-controlled objects and when they are created their duplication master resides on the session master.

The difference between a duplicated object, whether it is a `GlobalDO` or not, and a well-known duplicated object is that a well-known object is created before the session is created. When a station joins a session, it is guaranteed that the station will discover all well-known objects before the method `Session::JoinSession` returns on the station. In addition, well-known objects are fault-tolerant by default and will migrate to a new station if the one where its duplication master resides fails or leaves the session.

As described in §3.2, well-known duplicated objects must be defined in the DDF which ensures that the number of well-known instances in a session is known before the session is created. For example, in the *RoomZ* example the well-known objects are defined in the DDF as follows:

```
define          Room;
wellknown       Room g_hLobbby;
wellknown       SageBalancing g_hSageBalancing;
```

When a well-known duplicated object is declared in the DDF, a global variable of the type `WKHandle` is generated by the DDL compiler. The `WKHandle` variable refers to a well-known object. In *RoomZ*, *g_hLobby* and *g_hSageBalancing* are the generated `WKHandle` global variables. All `WKHandle` instances inherit from the `DOHandle` class.

All well-known duplicated objects specified in the DDF must be constructed and published before the session is created, *i.e.* before `Session::CreateSession` is called. Well-known duplicated objects are constructed in a similar manner to duplicated objects via the method `DuplicatedObject::CreateWellKnown`. This method returns a pointer to the newly created duplicated object. The pointer can be used until `DuplicatedObject::Publish` is

called, after which the pointer belongs to Net-Z and may be invalidated at any time as objects can be created and destroyed during a session. The duplication master of a well-known duplicated object must reside on the station that is the session master. Therefore, the method `CreateWellKnown` is only valid on the station that calls the method `CreateSession`. In *RoomZ* to create and publish the Lobby the following code is used:

```
{
     Room *pLobby = Room::CreateWellKnown(g_hLobby);
     pLobby->Publish();
}
```

To determine if a duplicated object is well-known, the method `DuplicatedObject::IsAWellKnownDO` can be used. This method will return true if the object that it is called on is well-known and false if it is not.

4.4 Datasets

The user must implement any dataset class that is declared in the DDL file and also declare that the dataset inherits from the DDL generated `DATASET` class as follows:

```
class DatasetName : public DATASET(DatasetName) {
     // User specified variables and methods.
};
```

For example, the datasets of the *RoomZ* example are declared in the DDL as follows:

```
dataset StringInfo {
     char Info[64];
};

dataset Location {
     dohandle room;
};

dataset SageKnowledge {
     string m_sQuote[512];
};
```

Therefore, the user specified classes StringInfo, Location, and SageKnowledge would be implemented as follows:

```
class StringInfo: public DATASET(StringInfo) {
     // methods of the class
};

class Location : public DATASET(Location) {
     // methods of the class
};

class SageKnowledge : public DATASET(SageKnowledge) {
     // methods of the class
};
```

A dataset may take a container as a parameter. The kinds of containers used in Net-Z are *vector*, *list*, and *queue* and the supported types for the containers are: *char, uchar, short, ushort, long, ulong, longlong, ulonglong, int, uint, float, double, bool,* and *dohandle*. If a container is used, depending on its kind, operations such as remove, add, insert, and iterate over may be performed on the dataset members of the container. In Net-Z, the methods available to perform such operations are found in the classes `MemberVector`, `MemberList`, and `MemberQueue` and are similar to the methods available for standard containers in the C++ standard library. For example, the Net-Z method `Pop_Back` removes the last element in the container and functions in a similar manner to the standard C++ method *pop_back*. The manner in which an iterator for a container is declared is slightly different to standard techniques, the syntax of which is shown below. Assuming that the dataset MyContainer is declared in the DDL as follows:

```
dataset MyContainer {
     vector<double> vec;
} reliable;
```

then to create an iterator for this container the following syntax is used:

```
MemberVector<double>::Iterator it;
```

4.4.1 Data Propagation

To propagate the contents of the datasets of a duplicated object from the duplication master to its duplicas, the method `DuplicatedObject::Update` is used. This method is called on the duplication master of an object and may be called on either a specific dataset or on all datasets of the duplicated object. The method must be called on a duplication master because changes to a dataset are always made from the duplication master to the duplica. The method `Update` has the semantic of a virtual method. Therefore, if an object is cast into a pointer of a base class and `Update` is called on the base class, the datasets of the object in both the base and inheriting classes will be updated.

When `Update` is called, if the dataset is unbuffered then the dataset variables on the duplica will be directly updated whereas, if the dataset is buffered then the buffer on the duplica will be updated. If a dataset is buffered, by specifying the update qualifier `buffered` or `extrapolation_filter`, then the method `DataSet::Refresh` or `DuplicatedObject::Refresh` must be called on the duplica in order to transfer the contents of the buffer to the duplica of the duplicated object. The method `DuplicatedObject::Refresh` may be called on either a specific dataset or on all buffered datasets of a duplica.

If a dataset of a duplicated object uses an extrapolation filter, then to maintain a coherent prediction model for the extrapolated data, the method `Update` must be called even if the contents of the dataset does not change. The frequency at which the method needs to be called will depend on how often the dataset changes, but for most games calling the method each physic loop will suffice. If the dataset uses an `upon_request_filter`, a call to update the dataset using the `Update` method will not generate a network message unless the method `DataSet::RequestUpdate` is called before it. Therefore, it is necessary to call the method `RequestUpdate` each time an update of the dataset is required. When an `upon_request_filter` is used it is up to the user to define the conditions under which the dataset will be updated. For example, the user can define that a specific variable must change a certain magnitude before it is updated.

Containers may be updated in either of two modes: content or operation via the methods `MemberContainer::SetContentUpdate` and `MemberContainer::SetOperationUpdate` respectively. In the content mode, each time `Update` is called the total content of the container is updated on each duplica, whereas in the operation mode the operations that were performed since the last update are sent to and reapplied on the duplica. It should be noted that when the operation mode is used it is the user's responsibility to ensure that data is accessed via standard container operations so that the operation can be reapplied on the duplica. When the method `Update` is called on a dataset that consists of a container, to reduce bandwidth usage, in general, only the changes made to the dataset members are propagated from the duplication master to its duplicas. The situations where this is not true and all dataset members are updated is when a station joins the session where upon discovery of the duplicated object a complete update of the container is sent to all duplicas. In addition, if the list of operations performed on the container members is greater than the number of members in the container, then a complete update will be sent.

To deal with the problem of different transport types, Net-Z performs the routing of messages automatically when required. Therefore, if a station cannot directly communicate with a station to which it wishes to send a message, Net-Z will automatically route the message via another station. This is useful to circumvent firewalls and to enable two stations that do not have a common transport type to communicate.

4.4.2 Hierarchical Messaging

To cope with station capacity and bandwidth limitations Net-Z instigates hierarchical messaging. To update information on its duplicas, a duplication master needs to send a message to each duplica of itself that exists throughout the session. If only a few duplicas exist, the updates can be sent directly from the station where the duplication master resides to each station that holds a duplica. However, as the size of a session increases, for an object to send a message directly to each of its duplicas becomes unrealistic in terms of the available bandwidth. Therefore, to permit a game to scale Net-Z implements hierarchical updates where `MulticastGroup` objects are used to distribute update messages sent from a duplication master to its duplicas. This enables a station to update all of its duplicas without using an excessive amount of bandwidth as the update message is forwarded to several duplicas via other stations in the game rather than the duplica receiving a message directly from the duplication master.

Hierarchical messaging can be configured by the user via the `MulticastPolicy` class, which is accessed via the `GetInstance` method. `MulticastGroup` objects are arranged in a tree like configuration with a `RootMulticastGroup` object being at the top of the hierarchy. Each `MulticastGroup` object contains a certain number of members, which may be either other `MulticastGroup` objects or stations if we are at the lowest level of the tree. The maximum number of members is set via the method `MulticastPolicy::SetMaximumGroupSize` and effectively determines the spread of the tree. If no maximum group size is set, a default value of 16 is used. The maximum group size and the number of stations connected to the session determine the number of levels in the hierarchy. For example, if we set the maximum group size to 10 as given below, the `RootMulticastGroup` object could contain 10 `MulticastGroup` objects, which in turn contain another 10 `MulticastGroup` objects, which then contain up to 10 individual stations.

```
MulticastPolicy::GetInstance()->SetMaximumGroupSize(10);
```

When a duplication master wishes to send a message to its duplicas, it has a choice of sending the message directly to a duplica or to the `RootMulticastGroup`, which is a global object and is therefore, known by all the stations in the session. The `RootMulticastGroup` will then forward the message to its members, who will then forward the message to their members and so on, until at the lowest level of the hierarchy the `MulticastGroup` will forward the message to a station who has a duplica of the object that is being updated. The maximum number of stations that a particular station will send a message to directly is defined as the message sending capacity of the station. It is set for all the stations in the session via the method `MulticastPolicy::SetDefaultMsgSendingCapacity`. If no message sending capacity is set, a default value of 16 is used. For example, if the default message sending capacity is set to 10, a station will send messages directly to 10 stations. If there are more than 10 stations that the

station needs to send a message to, then 9 messages will be sent directly to the duplicas and 1 message to the `RootMulticastGroup`, which will then forward the message via the multicast group hierarchy to all the other duplicas.

The message sending capacity of an individual duplicated object can be set via the method `DuplicatedObject::MsgSendingCapacity`. This method works in a similar manner to `MulticastPolicy::SetDefaultMsgSendingCapacity` but rather than setting the sending capacity of a station, this method sets the maximum number of messages that a particular duplication master can send. The group size and sending capacities can be configured dynamically throughout a session for all the stations or for each particular object. Therefore, if the frequency of updates for a particular duplicated object increases, the number of stations directly updated can be decreased to avoid increasing the bandwidth usage, and vice versa. The configuration used will be a trade off between the extra time taken for a message to reach its desired destination if the number of levels is high, compared to the number of messages each station is required to forward if the spread of the tree is wide.

By default a distributed broadcast is enabled however, it may be disabled and re-enabled while a game is running via the method `DisableDistributedBroadcast` and `EnableDistributedBroadcast`. When a distributed broadcast is disabled, each duplication master will send an update message directly to each of its duplicas. Therefore, if there are 20 duplicas of a particular object, the duplication master will send 20 individual messages each time the duplicas need to be updated.

4.5 Duplication Spaces

A duplication space is a space where specific duplicated objects are grouped together and are related by a user-defined function, which is termed the match policy. All the duplicated objects within a duplication space are split into two categories: subscriber and publisher. An object in a duplication space must be assigned the role of a subscriber, a publisher, or both, and may belong to more than one duplication space. A subscriber is an object that can discover other objects within the duplication space, and a publisher is an object that can be discovered within a duplication space. More specifically, a subscriber will discover the duplica of a publisher on the station where the duplication master of the subscriber resides, and a publisher will create a duplica of itself on the station where the duplication master of the subscriber resides. The duplication space concept is illustrated in Figure 4.1 where P and S refer to publishers and subscribers respectively.

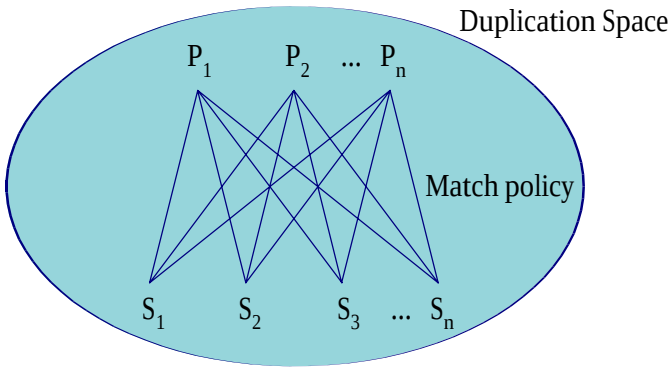


Figure 4.1: Conceptual diagram of a duplication space.

The decision of whether a subscriber discovers an object and if the publisher publishes a duplica of itself depends on the `PSMatch` function, which is the specific match policy that relates publishers to subscribers. The `PSMatch` function, which is explained in detail later in this section, is a user-defined Boolean function that determines if a particular publisher and a particular subscriber correspond in that particular duplication space. The `PSMatch` function offers the developer significant flexibility as it is up to the developer to define the conditions that constitute a match in the context of the game.

Duplication spaces ensure that objects are duplicated as defined by the `PSMatch` function however, due to various other requirements of the system it is possible for objects to be duplicated to additional stations not defined by the `PSMatch` function. For example, if an object is defined as fault tolerant and there are only two stations connected to the session, a duplica of the object will automatically be discovered by the second station, regardless of the `PSMatch` conditions, because by definition a fault tolerant object requires at least one duplica to be in existence. Therefore, for certain games it may be necessary to implement various filters to ensure that particular users cannot access given objects even if a copy of that object resides on the user's station.

As detailed in §3.1.3, duplicated objects are defined as publishers, subscribers, or both of a given duplication space. In *RoomZ* we define the duplication space RoomSpace and then the duplicated object class Delegate both as a publisher and a subscriber in the duplication space RoomSpace in the following manner:

```
dupspace RoomSpace;

doclass Delegate {
     // other declarations
     publisher in RoomSpace;
     subscriber in RoomSpace;
};
```

If no cells are declared in the duplication space, then all the publisher-subscriber matching computations are done on one station. Obviously, as the size of a session increases the number of computations required will increase. To reduce the resource usage on a particular station we can distribute the processing of the matching computations between many stations. To do this we divide the duplication space into cells, where each cell is responsible for a certain portion of the matching computations. A cell is a subset of a duplication space and typically groups publishers and subscribers of a similar "type". A cell may also inherit from the `GlobalDO` class to ensure that it is known across the session, if this is necessary in the context of the game. To determine if a particular publisher or subscriber belongs to a particular cell, a `CellMatch` function is used, which is described in detail later in this section. The concept of cells within a duplication space is illustrated in Figure 4.2.

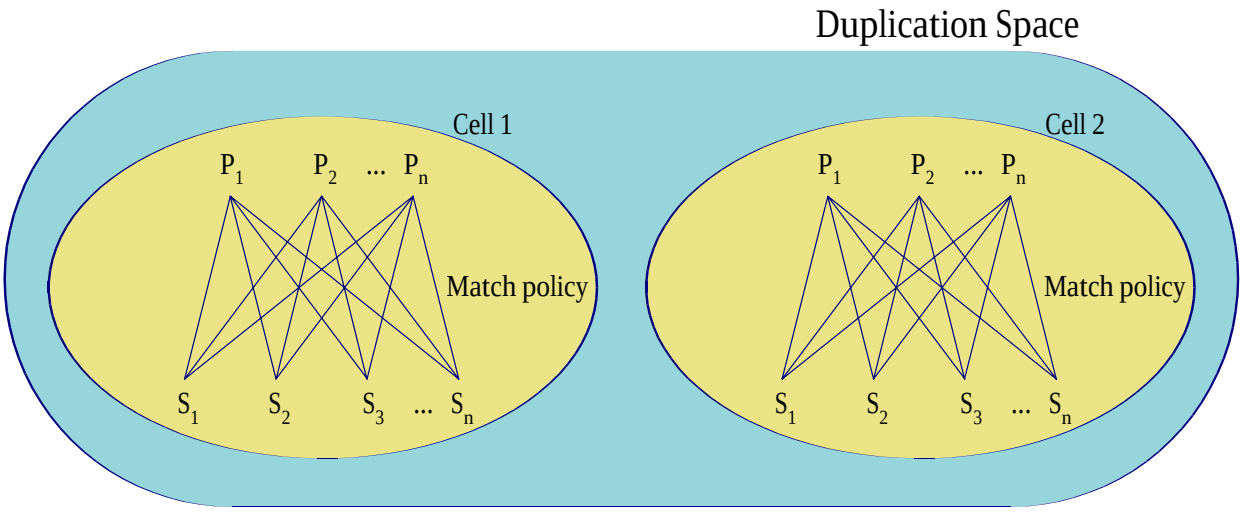


Figure 4.2: Conceptual diagram of a duplication space with cells.

The creation of cells enables the `PSMatch` computations to be distributed between the stations connected to the session. The station where the duplication master of a cell resides is where the computations are performed. A cell is a duplicated object that is dynamic and can be created and destroyed while a session is running. It is possible for the developer to define where the duplication master of a cell resides and cells may be load balanced to optimize their distribution. For example, if more stations join the session, then more cells can be created to take advantage of the additional processing power that the new stations bring to the session. Similarly, if a station leaves the session, the cells can be modified to redistribute the computations previously done by the leaving station. However, if a cell is not a global object, it is up to the developer to ensure that cells are discovered on the stations where they are required.

When a cell is defined in the DDL, the developer has the choice of using the default implementation or implementing cells himself in order to exercise greater control over the game. The default implementation divides the number of publishers and subscribers evenly into a suitable number of cells, the number of which will depend on the size of the session. Default cells automatically inherit from the `GlobalDO` class and are fault tolerant. However, in order to optimize the game, the developer can define the individual publishers and subscribers that belong to each cell.

In *RoomZ* we designate each Room duplicated object as a separate cell, which is define d in the DDL as follows:

```
doclass Room : GlobalDO {
     // other declarations;
     cell in RoomSpace;
};
```

4.5.1 PS Match Function

The `PSMatch` function is what determines for any given pair of publishers and subscribers in a duplication space whether or not an object should be discovered on a particular station. The match function is a user-implemented method that returns a Boolean value for the pairing of two duplicated objects. The function is iterated over the possible pairs formed by the publishers and the subscribers in the duplication space to determine if a discovery should take place. The match decision will typically be based on a computation involving the datasets of the two duplicated objects involved and can be based on such things as their topic, their location, the distance between two objects, or any other variable that makes sense in the context of the game.

When a `PSMatch` function is called by a duplicated object, it returns a Boolean value to determine if a certain subscriber should discover a certain publisher in a given duplication space. Therefore, on every station each duplication master and each duplica of a publisher is compared to each duplica of a subscriber. If the `PSMatch` function returns true, a duplica of the publisher is created on the station where the duplication master of the subscriber resides, if the publisher object is not yet known on that station. The subscriber will then discover the publisher. If the method returns false, a duplica of the publisher object will not be created nor will it be discovered by the subscriber. The behaviour of the `PSMatch` function is defined by the system but it is up to the developer to define the match condition, which is the conditions under which the function will return true or false. The match condition must be written so that it will return the same value if it is called on either the duplication master or duplica of a publisher or subscriber, i.e. independent of where it is called. The match function is dynamic and the match condition may change while a session is running.

The very simple case of three stations within one duplication space is illustrated in Figure 4.3. In this example Station 1 and Station 2 contain a Publisher, P_1, and Subscriber, S_2, respectively and Station 3 contains no duplication master objects. Station 3 is the station that created the session and therefore, in this simple case all objects are automatically duplicated to this station and the `PSMatch` function is performed on this station. In effect, Station 3 acts as a server. Each publisher and subscriber is duplicated, as denoted by a prime, to Station 3 to ensure that all the pertinent publishers and subscribers are discovered on the station that performs the `PSMatch` so that the `PSMatch` function can be performed correctly. The `PSMatch` function is then executed to match each duplica or duplication master of a publisher on Station 3 with each duplica of a subscriber on that station (Step 1). In this case, we assume that the relation returns true and therefore, a duplica of P_1 is discovered on Station 2 (Step 2).

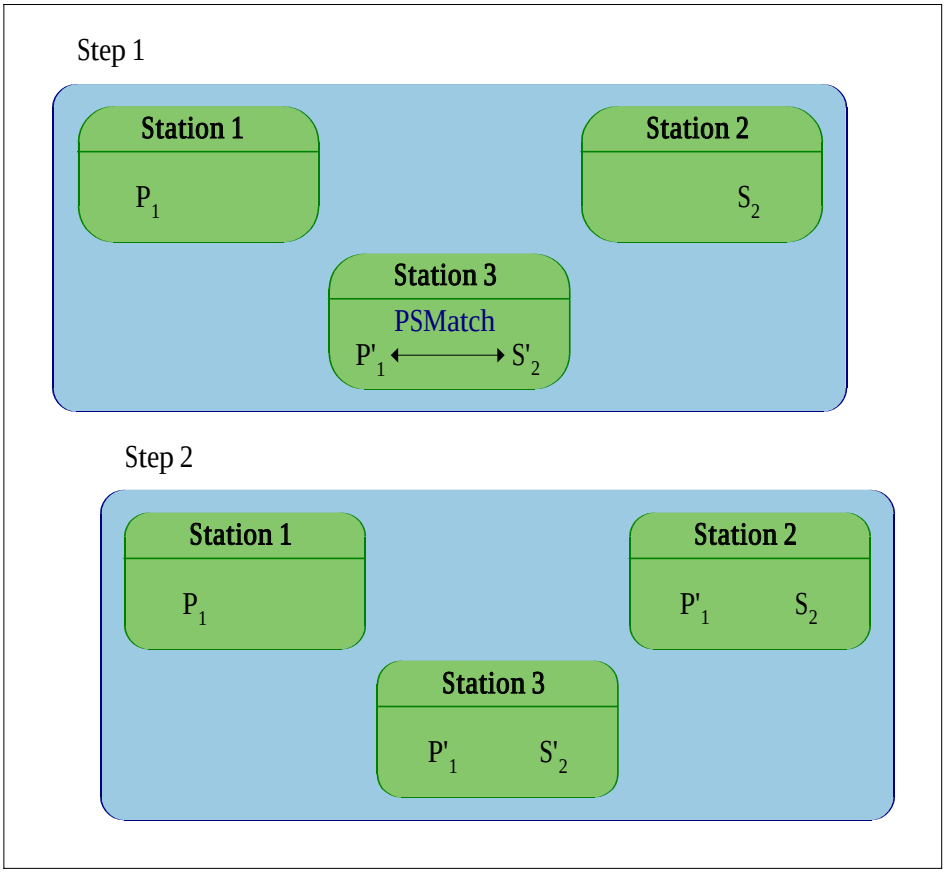


Figure 4.3: Illustration of the `PSMatch` mechanism.

The `PSMatch` function is implemented as a virtual function of the duplication space and returns a Boolean value. The syntax for the function is as follows:

```
virtual bool PSMatch(DuplicatedObject* pPublisher, DuplicatedObject*
pSubscriber);
```

For example, to implement the `PSMatch` for *RoomZ* we must implement a match condition that checks all publishers against all subscribers. This method verifies that the delegates are in the same room in order for them to discover each other, except for the moderator who discovers all the delegates in the session. Firstly we check if a subscriber is of the Room and Sage classes and return true if they are. Then we match all publisher objects of the type Delegate with all Moderator subscribers and return true if the objects exist. Next we match all Moderator publishers with all Delegate subscribers and return true if the objects exist. Finally we match all Delegate publishers with Delegate subscribers and return true if they are in the same room. If the `PSMatch` function returns true the publisher object will be discovered on the station where the duplication master of the subscriber resides. If it returns false, the publisher object will not be discovered.

```
bool RoomSpace::PSMatch(DuplicatedObject* pPublisher,
DuplicatedObject* pSubscriber){
     if (pSubscriber->IsA(DOCLASSID(Room)) && pSubscriber->
IsA(DOCLASSID(Sage))) {
           return true;
     }

     if (pSubscriber->IsA(DOCLASSID(Moderator))){
           if (pPublisher->IsAKindOf(DOCLASSID(Delegate)) ||
pPublisher->IsAKindOf(DOCLASSID(Sage))){
                 return(true);
           }
           return(false);
     }
     else if (pSubscriber->IsA(DOCLASSID (Delegate))) {
           DOHandle dohPublisher = pPublisher->GetHandle();
           if (pPublisher->IsAKindOf(DOCLASSID(Moderator))) {
                 return (true);
           }
           else if(((Delegate *)pSubscriber)->
IsInSameRoomAs(dohPublisher)) {
                 return (true);
           }
     }
     return (false);
}
```

4.5.1 Cell Match Function

A duplication space may be partitioned into a certain number of cells. The creation of cells subdivides the task of matching each publisher and subscriber and therefore, we must be able to determine if a particular duplicated object will be involved in a particular subtask. This is defined by the `CellMatch` function, which is published on all stations connected to the session. It is via this relation that the developer ensures that the publisher and subscriber which the `PSMatch` is called on are in the same cell.

To determine if an object should be discovered in a cell, the `CellMatch` function is used. This function compares all possible pairs formed by the publishers and subscribers and the cell to determine if a discovery should take place. Therefore, on each station each duplication master or duplica of a publisher or subscriber is compared to each cell duplica. When the `CellMatch` function returns true, a duplica of the publisher or subscriber is created on the station where the duplication master of the cell resides if the duplicated object is not yet known on that station. As for the `PSMatch` function, it is up to the developer to define the match condition for when the `CellMatch` function will return true or false. After the `CellMatch` is performed, the `PSMatch` is performed on the publishers and subscribers on the station where the duplication master of the cell resides.

For example, Figure 4.4 illustrates the simple case of three stations and one cell in the one duplication space. The cell is located on Station 3 and the duplication master of the cell object, C_3, which contains the `CellMatch` function and other pertinent cell information, is located on Station 3. Station 1 and Station 2 contain a Publisher, P_1, and Subscriber, S_2, respectively. A duplica, denoted by a prime, of the cell object is found on all stations in the session. The `CellMatch` function is called on the duplication space by the system and matches the duplication masters of each publisher and subscriber object with each duplica of the cell (Step 1). The `CellMatch` function ensures that all the pertinent publisher and subscriber objects are discovered on the station where the duplication master of the cell is located so that the `PSMatch` function can be performed correctly. We assume that the `CellMatch` function returns true in both cases shown here and therefore, duplicas of P_1 and S_1 are discovered on Station 3 (Step 2). The `PSMatch` function is then performed to match each duplica or duplication master of a publisher on Station 3 with each duplica of a subscriber on that station. We assume that the relation returns true and therefore, a duplica of P_1 is discovered on Station 2 (Step 3).

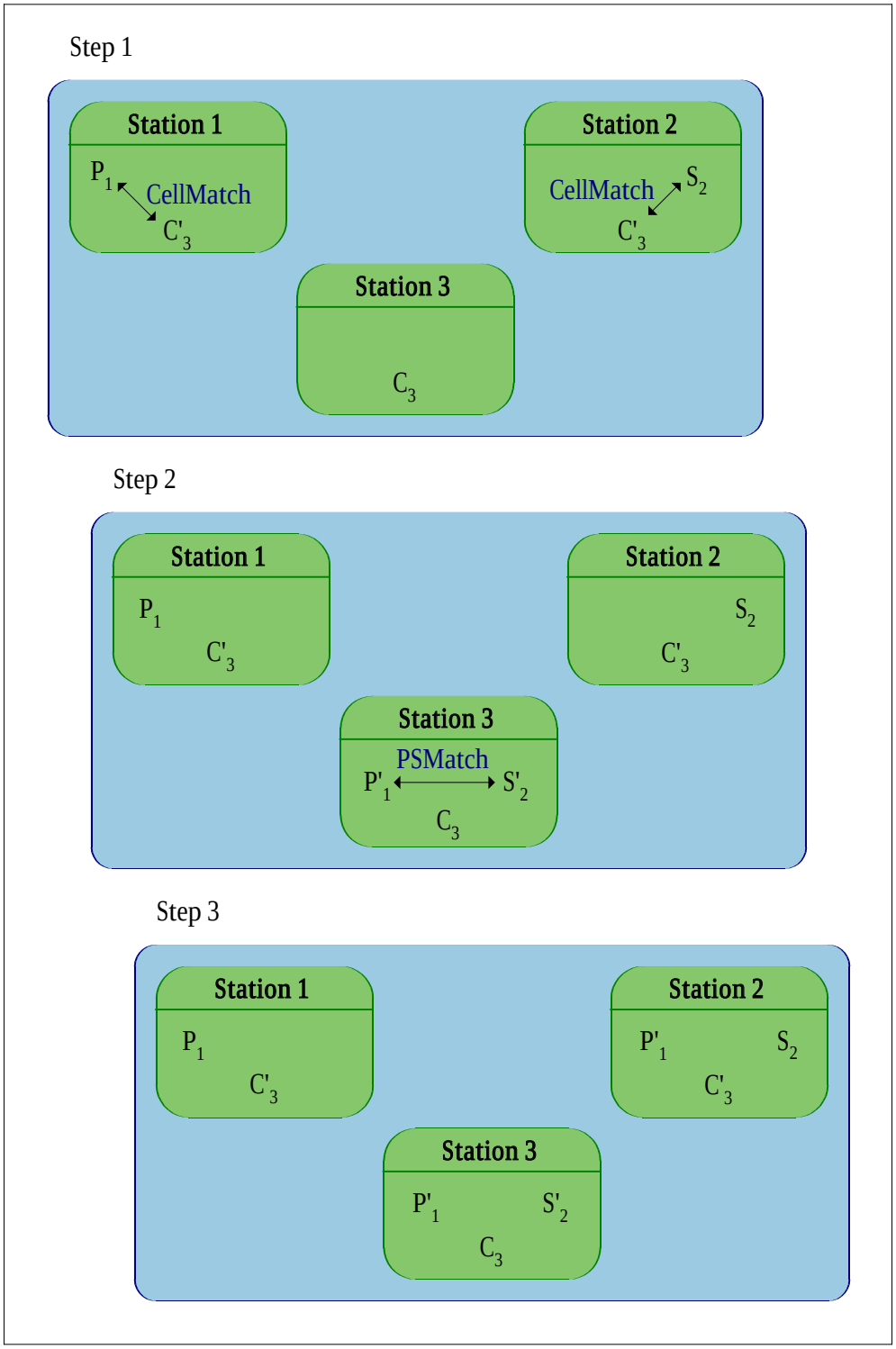


Figure 4.4: Illustration of the `CellMatch` mechanism.

When a cell is defined in the DDL, the developer must implement the `CellMatch` function in a similar manner to the `PSMatch` function. The `CellMatch` function should be written so that it compares all publishers and subscribers to a cell to determine if a particular publisher or subscriber should be discovered by that cell. After the `CellMatch` function is executed the `PSMatch` function will be called by the system on the station where the duplication master of the cell resides. When cells are defined by the developer it is necessary to call `DuplicationSpace::CreateCells` when the duplication space is constructed.

The `CellMatch` function is implemented as a virtual function of the duplication space and returns a Boolean value. The syntax for the function is as follows:

```
virtual bool CellMatch(DuplicatedObject* pPS, DuplicatedObject*
pCell);
```

As described in §3.1.3, in *RoomZ* we defined each Room duplicated object as a cell. To check if each publisher or subscriber should be discovered in a cell of the RoomSpace we verify that each publisher and each subscriber object in the duplication space is in the particular room of interest. Therefore, we match all Delegate objects with all Room cells and return true if they are in the same room. If the `CellMatch` function returns true the publisher or subscriber object will be discovered on the station where the duplication master of the cell resides. If it returns false, the objects will not be discovered.

```
bool RoomSpace::CellMatch(DuplicatedObject* pPS, DuplicatedObject*
pCell){
     return(((Delegate*) pPorS)->GetCurrentRoom()->GetHandle() ==
((Room*) pCell)->GetHandle());
}
```

4.5.3 Calling Match Functions

To update a duplication space the method `DuplicationSpace::Match` is called periodically. By default, this is called every 250ms, but the match period may be redefined by the developer via `MatchOperationTriggers::EnablePeriodicMatch`. The `Match` method calls the methods `MatchAcrossCellRelation` and `MatchAcrossPSRelation`, which respectively call the `CellMatch` and `PSMatch` functions. This ensures that all the publishers, subscribers, and cells of the duplication space are correctly duplicated. In general, the developer should not need to specifically call the match functions on a duplication space. However, several match methods of the `DuplicationSpace` class are provided that may be used to locally match specific objects. For example, `MatchSingleObject` can be used to call the appropriate match function on a given object and `MatchAcrossPSRelations` will match the publishers and

subscribers for the given cell. The method `CheckRelevance` is used to determine if duplicas are still relevant in the duplication space. If this method returns false, the duplica will be removed from the store via the `RemoveFromStoreOperation`.

4.5.4 Multiple Duplication Spaces

As previously mentioned, it is possible to define several duplication spaces within a game. The advantages of doing this are that in some cases it can simplify the implementation of the `PSMatch` and `CellMatch` functions, or it may be preferable or easier for the developer to design the game using multiple duplication spaces. The flexibility inherent in the design of duplication spaces allows the developer to create the game in the most convenient manner.

To illustrate the implementation of multiple duplication spaces we can redesign the *RoomZ* sample to use two duplication spaces. For simplicity we do not define any cells, but if required they would be defined as described in the previous section. Suppose the chat region now consists of several different zones where each zone is differentiated according to its category, which could include for example, the general subject, the age group, or the geographical location that the chat is limited to. Each zone is then divided into several rooms or sub-zones where the chat is limited to a more specific topic of the general category. As in the *RoomZ* sample, it is within each of these rooms that the chat actually takes place. Therefore, to modify the *RoomZ* sample we maintain our RoomSpace and duplicated object class Room but also define a ZoneSpace and the duplicated object class Zone, which inherits from the `GlobalDO` class. To the ZoneSpace we add the Delegate and Room classes as subscribers and publishers respectively. There are no restrictions on how many duplication spaces a particular duplicated object may belong to. Therefore, an object may be declared as a publisher and as a subscriber in several duplication spaces, as is the case in this example. Hence, the DDL for the modified *RoomZ* sample would be as follows:

```
dupspace RoomSpace;

dupspace ZoneSpace;

dataset StringInfo {
     char Info[64];
};

dataset Location {
     dohandle room;
};
```

```
doclass Delegate {
     Location m_dsLocation;
     StringInfo m_dsName;
     subscriber in RoomSpace;
     publisher in RoomSpace;
     subscriber in ZoneSpace;
     action PrintMsg(dohandle hSpeaker, string strMessage[128], bool
bWhisper);
     void SetCurrentRoom(in dohandle hRoom);
};

dataset SageKnowledge {
     string m_sQuote[512];
};

doclass Sage : Delegate {
     SageKnowledge m_dsKnowledge;
};
doclass Moderator : Delegate {
};

doclass Room {
     StringInfo m_dsTopic;
     publisher in RoomSpace;
     publisher in ZoneSpace;
};

doclass Zone : GlobalDO {
     StringInfo m_dsTopic;
};

doclass SageBalancing : LoadBalancingDomain {
};
```

By defining the duplication spaces as such, we need to modify the implementation of the `PSMatch` function. The implementation is very similar to that performed in *RoomZ* as described previously except that we have to implement the match conditions for the ZoneSpace in addition to the RoomSpace. The match conditions for the RoomSpace would be implemented in a similar manner as described in the previous section and the match conditions for the ZoneSpace would verify that the publishers and subscribers of the ZoneSpace, which are the Room and Delegate objects respectively, are in the same zone.

4.6 Remote Method Calls

The manner in which Net-Z remotely calls a method on a duplicated object is via Remote Method Calls (RMC) or via actions, which are described in §4.7. RMCs are methods that are called on one specific instance of an object, which may be a duplication master or a duplica, and they may return a value. RMCs give the developer significantly more control over the method than actions with respect to the manner in which they are called and the information the developer may obtain about the `State` and the `Outcome` of the call. A RMC may be performed both locally and remotely with respect to the station where it is called and since a RMC is implemented as a C++ method, it may also be called directly on the local station.

RMCs are declared in the DDL, as described in §3.1.4, and are implemented using the following signature. The valid arguments are the same as those valid for datasets which are: *char, uchar, short, ushort, long, ulong, longlong, ulonglong, int, uint, float, double, bool, dohandle*, an *array* of any of these types, and *string*. A RMC cannot include a container as a parameter.

```
returntype RMCName( [Argument List] );
```

For example, continuing with *RoomZ* the RMC SetCurrentRoom is implemented as a method of the Delegate duplicated object class in the following manner:

```
class Delegate : public DOCLASS(Delegate) {
     virtual void SetCurrentRoom(DOHandle dohRoom);
};
```

When a RMC is declared in the DDL, the Net-Z DDL compiler automatically generates a stub to perform the RMC. The stub is used to call the method on the duplicated object. The name of the stub is obtained by adding the prefix `Call` to the name of the RMC. The syntax of the RMC stub is given below where ***RMCParameters*** are the arguments specified in the RMC declaration. The mapping of the arguments between the RMC and the stub are given in Table 4.1. If the RMC specifies the return type rather than `void`, the stub contains an additional pointer to the `Result`.

```
// RMC invocation stub for a method that specifies the return type
bool CallRMCName(RMCContext* pCallContext, type* pResult,
RMCParameters)
//RMC invocation stub for a void method
bool CallRMCName(RMCContext* pCallContext, RMCParameters)
```

Therefore, in the above example the generated DOCLASS(Delegate) defines the following stub:

```
bool CallSetCurrentRoom(RMCContext* pCallContext, int dohRoom)
```

RMC Argument Type	Modifier	RMC Stub Argument Type
Simple type	in	Simple type
Simple type	out	Pointer to a simple type
Simple type	in out	Pointer to a simple type
Array of simple types	in	Pointer to the first element of a sequence of simple type
Array of simple types	out	Pointer to the first element of a sequence of simple type
Array of simple types	in out	Pointer to the first element of a sequence of simple type
String	in	Pointer to the first element of a sequence of TCHAR
String	out	Pointer to the first element of a sequence of TCHAR
String	in out	Pointer to the first element of a sequence of TCHAR

Table 4.1: Mapping of argument types for remote method calls.

To invoke a RMC the context of the RMC must be defined via the `CallContext` class. Via this class you set the station you wish the method to be called on and the appropriate flags. You can also obtain information about the `State` and the `Outcome` of the call and have the ability to cancel or reset a call. Call contexts are detailed in §4.11. They are used to specify the context of a RMC and the fetching and migration of objects.

To complete the example, part of the event loop for the duplicated object class Delegate is as follows. In this case we clear the flag `SynchronousCall` to ensure that the RMC is made asynchronously and set the target station of the call to where the duplication master of the object resides.

```
void Delegate::SetCurrentRoom(DOHandle dohRoom) {
     SYSTEMCHECK(dohRoom != INVALID_DOHANDLE);
     if(IsADuplicationMaster()) {
           m_dsLocation.room = dohRoom;
           Update();
     }
     else {
           RMCContext oContext;
           oContext.ClearFlag(RMCContext::SynchronousCall);
           oContext.SetTargetStation(GetMasterStation());
           CallSetCurrentRoom(&oContext, dohRoom);
     }
}
```

4.7 Actions

The manner in which Net-Z remotely calls a method on a duplicated object is via Remote Method Calls, as described above, or via actions. Actions are the easiest way to call a method on all the duplica of a duplication master and to ensure that the method is executed on each station participating in the session that has a copy of the duplica. An action is always performed within a duplicated object class and may be performed both locally and remotely with respect to the station where the action is called. An action may be called either on a duplica, which will then call the method on its duplication master, or called directly on a duplication master. Since an action is implemented as a C++ method, it may also be called directly on the local station.

Actions are declared in the DDL, as described in §3.1.5, and are implemented using the following signature. The valid arguments are the same as those valid for datasets which are: *char, uchar, short, ushort, long, ulong, longlong, ulonglong, int, uint, float, double, bool, dohandle*, an *array* of any of these types, and *string*. An action cannot include a container as a parameter.

```
action ActionName( [Argument List] );
```

For example, in *RoomZ* the action PrintMsg is implemented as a method of the Delegate duplicated object class in the following manner:

```
class Delegate : public DOCLASS(Delegate) {
     void PrintMsg(dohandle hSpeaker, string strMessage[128], bool
bWhisper);
};
```

When an action is declared in the DDL, the Net-Z DDL compiler automatically generates two stubs to perform the action, although only one is used at a time. A stub is used to call the action on the duplication master or on the duplicas. The first stub calls the action on the duplication master of an object and it may be invoked on any one of the duplicas of the object. The name of the stub is obtained by appending `_OnMaster` to the name of the action. In the above example, the generated DOCLASS(Delegate) defines the stub PrintMsg_OnMaster(dohandle hSpeaker, string strMessage[128], bool bWhisper). The second stub calls the action on every duplica of an object and may be invoked on the duplication master of an object. The name of the stub is obtained by appending `_OnDuplicas` to the name of the action. For the above example, the generated DOCLASS(Delegate) defines the stub PrintMsg_OnDuplicas(dohandle hSpeaker, string strMessage[128], bool bWhisper). The mapping of the arguments between the action and the stub are given in Table 4.2.

Action Argument Type	Action Stub Argument Type
Simple type	Simple type
Array of simple types	Pointer to the first element of a sequence of simple type
String	Pointer to the first element of a sequence of TCHAR

Table 4.2: Mapping of argument types for actions.

To complete the example, part of the event loop for the duplicated object class Delegate is as follows. To call the method PrintMsg on the master of each participant in the chat who is in the same room as the caller the stub `_OnMaster` is used. In this case we use an iterator to iterate over the Delegates that are discovered in the RoomSpace by the caller. As a result, each duplication master will call the method PrintMsg, which is implemented in the Delegate class.

```
void TalkToEveryone() {
     TCHAR  strSpeak[LINE_LENGTH];
     _tprintf(_T("What do you want to say (1 line)?\n\n"));
     _fgetts(strSpeak, LINE_LENGTH, stdin);
     TCHAR* str = _tcschr(strSpeak, _T('\n'));
     if(str!=NULL) *str = _T('\0');
     Delegate::SelectionIterator itCoDelegates;
     Delegate::Ref refMyDelegate(g_hMyDelegate);
           while(! (itCoDelegates.EndReached())){
           Delegate::Ref itRef(itCoDelegates->GetHandle());
           if(RoomSpace::GetInstance()->PSMatch(itRef.GetDOPtr(),
refMyDelegate.GetDOPtr())) {
                 itCoDelegates->PrintMsg_OnMaster(g_hMyDelegate,
strSpeak, false);
           }
           itCoDelegates++;
     }
```

4.8 Load Balancing

In order to optimize a system it is necessary to make the best use of all available resources. To permit a system to scale in the most efficient manner we need to distribute the workload between the stations on the network. Load balancing is the process whereby specific work quantities may be transferred from one station to another. This enables the available processing power to be exploited and reduces bandwidth usage by distributing the workload across the game. By implementing load balancing the developer can ensure that each station on the network performs a portion of the total system workload that corresponds to the station's capacity. When a new participant joins the session, or a current participant leaves the session, the distribution of workload will change to optimize the available processing power and bandwidth.

With Net-Z, you have complete control over how load balancing is performed in the game. Virtually any quantity can be load balanced as you define the load balancing policies: the load metrics, the capacity metrics, and how the load is to be distributed across the stations. Therefore, you may choose to load balance such things as CPU load, communication bandwidth, memory load, or duplicated objects.

All load balancing performed by Net-Z is controlled via the `LoadBalancingDomain` class. This class may be sub-classed, as detailed in §4.8.2, in order for a user-defined load balancing policy to be implemented. As the developer defines what constitutes a load and a capacity, and how to transfer the load, game specific load metrics and balancing policies can be easily implemented so that what is load balanced makes sense in the context of the game. The load and capacity are defined via the methods `GetLocalLoad`, `GetLocalCapacity`, and `SetLocalCapacity`, and `TansferLoad` is used to specify how to transfer load between stations.

It is up to the developer to decide which stations will participate in load balancing. By default, Net-Z assumes that all stations will participate in load balancing. However, the participation of a given station can be permitted and cancelled while a game is running via the methods `Participate` and `CancelParticipation` respectively. For example, load balancing could be limited to a group of servers and exclude the individual stations that are connected to them.

The default behaviour of Net-Z is to perform load balancing only when requested to do so by a call to the method `RequestLoadBalancingEpisode` as follows:

```
pLoadBalancingDomain->RequestLoadBalancingEpisode();
```

However, while a game is running, load balancing may be toggled between the manual and automatic implementation via the methods `SelectAutomaticLoadBalancing` and `SelectManualLoadBalancing`. The former method will instruct Net-Z to perform load balancing automatically at a set time interval, which is set via the method `SetEpisodePeriod`.

To revert to manual load balancing after automatic load balancing has been invoked, a call to `SelectManualLoadBalancing` is necessary. Load balancing may then be performed when requested by calling the method `RequestLoadBalancingEpisode`.

Net-Z determines when to transfer load between stations via the load ratio and the equilibrium deviation threshold. The load ratio is the ratio of the load to the capacity on a given station and is set via the method `SetLoadRatioThreshold`. It has a default value of 0. The equilibrium deviation is the load ratio of the local station minus the ideal load ratio and is set via the method `SetEquilibriumDeviationThreshold`. It has a default value of 0.05 units. The ideal load ratio is the total load divided by the total capacity of all the stations that participate in load balancing. If the load ratio on a given station is greater than the load ratio threshold, then load balancing will be performed across all the participating stations to try to reduce the load ratio of that station to the ideal ratio. If the equilibrium deviation on a given station is greater than the equilibrium deviation threshold, then load balancing will be performed to transfer some load from that station in order to restore equilibrium between all the participating stations.

4.8.1 Object Load Balancing

With Net-Z, user-defined duplicated objects may be subjected to load balancing. Therefore, rather than one single station acting as the duplication master for all objects, the duplication masters can be dynamically distributed across the stations connected to the session. When load balancing is performed, Net-Z promotes a duplica to the role of duplication master while on another station the previous duplication master is demoted to a duplica. The load balancing policy is based on object weight and station capacity, and the migration of objects ensures that the total weight of the duplication masters on a station corresponds to its capacity. The developer may use the default implementation of object load balancing or may define certain characteristics such as what defines the weight of an object, the capacity of a station, which objects are load balanced, and which stations participate in the load balancing.

For an object to be subject to load balancing, control of the duplication master must be able to migrate between stations. Migration is the process whereby control of a duplicated object is transferred to another station. Migration involves the two operations emigration and immigration. Emigration is the process whereby a station reassigns control of a duplicated object to another station, and immigration is the process whereby a station accepts control of a duplicated object that was previously controlled by another station. In Net-Z, object migration is achieved via the `DuplicatedObject::ApproveEmigration` system callback. If the callback returns true, then the object can migrate between stations and if it returns false then the object cannot migrate and will cease to exist when the station where its duplication master resides leaves the session. The default behaviour of an object is to return false.

The following code shows implementation of the `ApproveEmigration` system callback that allows Room objects to migrate from one station to another to perform load balancing.

```
class Room : public DOCLASS(Room) {
     public:
           Room();
           ~Room();
           bool ApproveFaultRecovery() {
                 return true;
           }
           bool ApproveEmigration(unsigned long ulReason) {
                 return true;
           }
           // other game specific methods and variables
};
```

To distribute specific workloads in Net-Z, the load balancing of particular system objects is implemented internally. In particular, the multicast group objects used in hierarchical messaging are automatically load balanced to ensure that updates are performed in the most efficient manner possible.

4.8.2 Sub Classing Load Balancing

To add custom functionality you may sub class the `LoadBalancingDomain` class. Naturally, the sub classed class inherits all methods from its base class. Any inherited methods that you wish to redefine, must be redefined in the source file of the new class, along with any user-defined methods for the class. The user-defined class must be defined in the DDF and in the DDL defined as inheriting from the `LoadBalancingDomain` class.

In *RoomZ*, we sub class the `LoadBalancingDomain` in order to implement the user-defined load balancing scheme, SageBalancing. In the DDL we define the SageBalancing class as inheriting from the `LoadBalancingDomain` class and also define the SageBalancing class in the DDF. In the SageBalancing class, we define what constitutes a local load and capacity and how to transfer the load. The appropriate methods are defined in the header file of the sub classed load balancing domain as given below. In this case, we define the local load as the number of Sages in the duplication master Rooms on the local station.

```
class SageBalancing : public DOCLASS(SageBalancing) {
     public:
           SageBalancing();
           ~SageBalancing();
           // Redefine virtual methods of the LoadBalancingDomain
           // class.
           double GetLocalLoad() {
                 double dLoad=0.0;
                 Room::SelectionIterator it(DUPLICATION_MASTER);
                 while(!it.EndReached()) {
                 dLoad+=1.0*it->GetNbSages();
                 it++;
                 }
                 return dLoad;
           }
           double GetLocalCapacity() {return 1.0;}
           void TransferLoad(DOHandle hStation, double dLoad);
};
```

The methods are then implemented in the source file of the sub classed load balancing domain. In this example, how the load is transferred is defined via the method `TransferLoad`. Here, we define that in order to perform load balancing of Sage objects, we move them from Room to Room.

```
void SageBalancing::TransferLoad(DOHandle hStation, double dLoad) {
     double dLoadToTransfer=dLoad;
     TRACE(TRACE_LOADBALANCING, _T("SageBalancing::TransferLoad(%f) is
called!!"), dLoadToTransfer);
     //Migrate as many Sage Masters as possible without surpassing
dLoad
     while(dLoadToTransfer >= 1.0) {
           Room::Ref refRoomM = FindRoomWithMostSagesOnLocalStation();
           Room::Ref refRoomL =
FindRoomWithLeastSagesOnStation(hStation);
           if(!refRoomM.IsValid() || !refRoomL.IsValid()) {
                 return;
           }

           Sage::Ref refSage(refRoomM->ChooseSageHandle());
           TRACE(TRACE_LOADBALANCING, _T("Trying to move Sage %x from
Room %s to Room %s"), refSage->GetHandle(),
                 refRoomM->GetTopic(), refRoomL->GetTopic());
           if(refSage.IsValid()) {
                 refSage->SetCurrentRoom(refRoomL->GetHandle());
                 dLoadToTransfer-=1.0;
           }
     }
}
```

4.9 Fault Tolerance

Fault tolerance is the ability of a program to recover from faults, such as the disconnection of a station, with little or no effect on the ongoing session. A station may fail, but the duplicated objects will continue to exist as if nothing had happened. The disconnection of a station may be due to a participant quitting the session or due to the failure of the network, power supply, or some other fault. Net-Z implements fault tolerance by giving the station that performs the role of session master the ability to assume control of a fault-tolerant duplicated object whose duplication master has failed. When a station fails, all the duplica of the fault-tolerant objects from the failed station are promoted to duplication masters on the session master. If the session master fails, then Net-Z automatically elects a new session master. When failure of a particular station occurs, load balancing may be implemented to redistribute the total load over all currently connected stations. Therefore, stations may fail but duplicated objects will continue to exist as if nothing had happened.

Duplicated objects are not fault-tolerant by default. It is up to you to decide if an object is fault-tolerant. An object is defined as fault-tolerant via the `DuplicatedObject::ApproveFaultRecovery` system callback. If the callback returns true, then the object is fault-tolerant. If it returns false then the object is not fault-tolerant and will cease to exist if and when the station where its duplication master resides fails.

The following code shows implementation of the `ApproveFaultRecovery` system callback, which dictates that Room objects are fault-tolerant:

```
class Room : public DOCLASS(Room) {
     public:
           Room();
           ~Room();
           bool ApproveFaultRecovery() {
                 return true;
           }
           // other game specific methods and variables
};
```

On the other hand, if a station quits the session, rather then leaving due to the occurrence of a fault, then the duplication masters on the station will be destroyed when the station quits the session, unless the `DuplicatedObject::ApproveEmigration` system callback returns true. This callback allows the control of a duplication master to migrate between stations. It must return a reason for the object to migrate, which may be either migration due to leaving the session or load balancing. If the callback returns true, then the object can migrate between stations and if it returns false then the object cannot migrate and will cease to exist when the station where its duplication master resides leaves the session. Therefore, if we wish an object to survive regardless of whether the station where its duplication master resides leaves the session due to a fault or simply quits the session, then both the `ApproveFaultRecovery` and `ApproveEmigration` system callbacks must

return true. For example, if we wish the Room objects to survive when the station where their duplication masters reside quits the session, as well as when a fault occurs, the following code is implemented:

```
class Room : public DOCLASS(Room) {
     public:
           Room();
           ~Room();
           bool ApproveFaultRecovery() {
                 return true;
           }
           bool ApproveEmigration(unsigned long ulReason) {
                 return true;
           }
           // other game specific methods and variables
};
```

If due to a fault a station only loses the connection to the session it is participating in rather than losing all functionality, such as would occur if the LAN connection to the station was disabled as opposed to a loss of power, it is possible that the session will continue to run on the faulting station. When this situation occurs, we effectively have two instances of the same session. Both sessions can be joined by other stations, however the two sessions cannot rejoin each other's session.

4.10 Dead Reckoning

Net-Z provides easy extrapolation of dataset values by the implementation of a customized algorithm. The algorithm is simple to use and requires no a-priori knowledge of changes in the value of a variable over time. To efficiently use the available bandwidth, the precision of the extrapolation may be either constant or based on the distance between the object and a user-defined observer. These two cases are explained in detail later in this chapter.

When implemented, dead reckoning is performed automatically by Net-Z to determine when to update the dataset of a duplica. For datasets that change frequently, to update the dataset of a duplica every time the duplication master's dataset was updated would use up precious bandwidth. Therefore, it makes more sense to use an extrapolation filter to update the dataset of the duplicas and to set a maximum error between the dataset values of the duplication master and its duplicas. This error is termed the dead reckoning extrapolation error and when it exceeds the error tolerance the dataset of a duplica will be updated from the duplication master.

The extrapolation error is defined as the difference between the current dataset values of an object, *i.e.* that of the duplication master, and the extrapolated dataset values of the object, *i.e.* that of a duplica. The error tolerance may be constant or a function of the distance between the observer and an object, depending on the parameters chosen. For example, the error tolerance function can be set such that a

larger error is tolerated for objects displayed a long distance from the observer, *i.e.* those objects in the background of a scene, and a smaller error is tolerated for those objects closer to the observer, *i.e.* those in the foreground. When this is the case, extrapolation of the datasets is more precise and therefore, a greater portion of the bandwidth is used for objects that are close to the observer. For objects that appear further away from the observer, extrapolation of the datasets is less precise and proportionally less bandwidth is used. Net-Z does not impose a definition of close or far. It is up to you to implement a system callback that calculates the distance between two duplicated objects via the method `DuplicatedObject::ComputeDistance`, which is detailed later in this chapter.

Dead reckoning can only be used on datasets that are continuous, although they may contain infrequent continuity breaks. Continuity breaks are discussed later in this chapter. To implement dead reckoning, you are required to perform the following three basic operations:

- The definition of the dataset in the DDL must indicate that an `extrapolation_filter` will be used, as described in §3.1.2.
- The extrapolation precision must be defined using the `SetParameters` system callback, or one of the appropriate wrappers, as described in this chapter.
- The dataset variables must be updated on the duplication master and refreshed on all the related duplicas, as described below.

It should be noted that if in the DDL it is indicated that a dataset will use an extrapolation filter and no error tolerance is defined, then a default value of zero is used for the error tolerance.

The method `DuplicatedObject::Update` is used to propagate the contents of the datasets of a duplication master to its duplicas and `DataSet::Refresh` is used to refresh the values of the datasets of the duplicas of a buffered dataset. These methods need to be called periodically in every game. For example, the methods may be called via the methods DoPhysic and Render as follows:

```
void Sphere :: DoPhysics() {
     // compute the physic and then update the duplication
     // master's position.
     Update();
}

void Sphere :: Render(unsigned long ulDummy) {
     if (IsADuplica()) {
           // Refresh the position.
           m_dsPos.Refresh();
     }
// render the object at the correct position.
}
```

When this code runs, the method `Update` will check if the current extrapolation error is greater than that defined by the error tolerance function. If the error is greater than the tolerance, then a message is sent to correct the extrapolation model. On the duplica side, the dataset extrapolation model generated by Net-Z is used to refresh the dataset variables whenever required. The methods `Update` and `Refresh` do not have to be synchronized. If the duplica runs faster than the duplication master, Net-Z will automatically interpolate the values of a dataset.

4.10.1 Constant Extrapolation Error

When a constant dead reckoning extrapolation error is chosen, the extrapolation error will not vary regardless of the distance between the observer and the object. Therefore, once the error is exceeded the number of position updates sent for the object and the amount of bandwidth used remains the same, regardless of the relative positions of the observer and object.

To implement dead reckoning, the values of the dataset must be updated using an extrapolation filter. This must be defined in the DDL as described in §3.1.2.

Current Extrapolation Error

The current extrapolation error is computed as the difference between the current value and the extrapolated value of a dataset. The error is calculated according to the following equation, where the x_i values are the values of different variables contained in the same dataset, and the prime refers to the extrapolated value of the variable.

$$Extrapolation\ Error = \left(\sum_{i=1}^{n} (x_i - x_i')^2 \right)^{1/2}$$

In the calculation of the extrapolation error, the final value of the error is equally dependant on each variable of the dataset. Therefore, to avoid one variable having a dominant effect on the extrapolation error ensure that the datasets are consistent, *i.e.* each of the variables of a dataset should have a similar range of values.

Error Tolerance

The dead reckoning error tolerance function is calculated according to the following equation:

$$\textit{Error tolerance} = \textit{dConstantFactor} + \textit{dLinearFactor} * z + \textit{dQuadraticFactor} * z^2$$

where the values *dConstantFactor*, *dLinearFactor*, and *dQuadraticFactor* are the parameters of the error tolerance function, as defined using the method `ErrorToleranceFunction::SetParameters`, and *z* is the distance between the observer and an object. When a constant extrapolation error is used the linear and quadratic factors should be set to zero. Therefore, it is not necessary to define the distance as the error tolerance does not depend on it.

The syntax of the method `SetParameters` is:

```
ErrorToleranceFunction::SetParameters(double dConstantFactor, double
dLinearFactor, double dQuadraticFactor, double dMaximumError)
```

Rather than using the method `SetParameters` the easiest way to set a constant error is by a call to the method `SetConstantError` as follows:

```
ErrorToleranceFunction::SetConstantError(double dError)
```

The method `SetConstantError` calls the method `SetParameters` and is equivalent to the call:

```
ErrorToleranceFunction::SetParameters(double dError, 0, 0)
```

4.10.2 Distanced-based Extrapolation Error

When a distance-based extrapolation error is used, the tolerated error will change according to the distance between the observer and the object. The observer is the duplicated object from which the distance is calculated and is defined on each process by the user via the method `Station::RegisterObserver`. The observer may be redefined as required and if no observer is defined then Net-Z uses a default distance to calculate the error tolerance.

When distance-based dead reckoning is used, the precision of the extrapolation adjusts dynamically, based on the distance between the observer and the object, to efficiently use the available bandwidth. For example, the user can define that the extrapolation error decreases as the distance between the observer and object decreases. Therefore, as an object moves closer to the observer, the error decreases, and the object's dataset will be refreshed more frequently so that the duplica is shown more accurately. Hence, the closer an object is to the observer, the more frequently the position is refreshed and greater bandwidth is used. Similarly, when the object is far away from the observer, the allowed error

is greater and therefore, the datasets are refreshed less frequently and less bandwidth is used. Net-Z does not impose a definition of close or far. It is up to you to implement the system callback `DuplicatedObject::ComputeDistance`, which calculates the distance between two duplicated objects. The implementation of this callback is explained later in this chapter.

If distance-based dead reckoning is used for a dataset, whenever the method `DuplicatedObject::Update` is called on the duplication master that holds the dataset, the error tolerance function is used by Net-Z to determine if the extrapolation error on the dataset is acceptable or not. The general idea of the algorithm is to compute two values: the current extrapolation error and the error tolerance. If the current extrapolation error is greater than the error tolerance, the method `Update` refreshes the contents of all the duplicated object datasets to their duplicas. If the extrapolation error is less, then the datasets are not updated. Net-Z is optimized to efficiently perform these computations. As the distance between the observer on each station and the object can be different, dataset updates may be sent to the various remote stations at different frequencies.

Current Extrapolation Error

The current extrapolation error is computed as the difference between the current value and the extrapolated value of a dataset as detailed in §4.10.1.

Error Tolerance

The error tolerance is a function of the distance between the observer and an object. Therefore, depending on the parameters chosen, the tolerated error may change according to the distance between the observer and the object. For example, the error tolerance function can be set such that a larger error is tolerated for objects displayed a large distance from the observer, *i.e.* those objects in the background of a scene, and a smaller error is tolerated for those objects closer to the observer, *i.e.* those in the foreground. The error tolerance is computed in two steps. Firstly, the distance between the duplicated object that is to be updated and the observer on the station where the duplicated object will be updated is computed. To compute the distance, Net-Z calls the method `DuplicatedObject::ComputeDistance` on the local instance of the observer on the remote station. This system callback should be redefined for each duplicated object class that may be used as the observer. If the value returned by this method is `UNKNOWN_DISTANCE`, a default distance is used. This default distance is defined using the method `ErrorToleranceFunction::SetDefaultDistance`. Net-Z then uses this value if the distance between two duplicated objects is unknown, or if no observer is defined on the station being updated. If no default distance is defined then Net-Z uses a value of zero and therefore, the error tolerance will be constant. The distance between the duplicated objects is then used to compute the error tolerance, via the following equation:

$$Error\ tolerance = dConstantFactor + dLinearFactor * z + dQuadraticFactor * z^2 \qquad \textbf{Equation 4.1}$$

where the values *dConstantFactor*, *dLinearFactor* and *dQuadraticFactor* are the parameters of the error tolerance function, as defined using the `ErrorToleranceFunction::SetParameters` method, and *z* is the distance between the object being updated and the observer.

The error tolerance can be defined by using the method `SetParameters` or more simply by using the methods `SetLinearError` or `SetQuadraticError`. The method `SetLinearError` defines a linear relationship between the error tolerance and the distance. The syntax is as follows:

```
bool ErrorToleranceFunction::SetLinearError(double dMinimumValue,
double dMaximumValue, double dDistance)
```

The variables *dMinimumValue* and *dMaximumValue* are the minimum and maximum tolerated error, and *dDistance* is the distance at which the maximum tolerated error is reached. The linear coefficient for the relationship between the distance and the error tolerance is given below and the relationship is shown graphically in Figure 4.5.

$$linear\ coefficient = \frac{dMaximumValue - dMinimumValue}{dDistance}$$

The method `SetLinearError` calls the method `SetParameters` and is equivalent to the call:

```
ErrorToleranceFunction::SetParameters(double dMinimumValue, linear
coefficient, 0, double dMaximumValue)
```

The method `SetQuadraticError` defines a quadratic relationship between the error tolerance and the distance, where there is no linear dependence. The syntax is as follows:

```
bool ErrorToleranceFunction::SetQuadraticError(double dMinimumValue,
double dMaximumValue, double dDistance)
```

The variables *dMinimumValue* and *dMaximumValue* are the minimum and maximum tolerated error, and *dDistance* is the distance at which the maximum tolerated error is reached. The quadratic coefficient for the relationship between the distance and the error tolerance is given below and the relationship is shown graphically in Figure 4.5.

$$quadratic\ coefficient = \frac{dMaximumValue - dMinimumValue}{(dDistance)^2}$$

The method `SetQuadraticError` calls the method `SetParameters` and is equivalent to the call:

```
ErrorToleranceFunction::SetParameters(double dMinimumValue, 0,
quadratic coefficient, double dMaximumValue)
```

As opposed to the method `SetParameters`, a maximum value for the tolerated error must be defined when using the `SetLinearError` or `SetQuadraticError` methods. If you wish to have an unlimited maximum error then it is necessary to use the method `SetParameters` to describe the error tolerance. Similarly, if you wish to define an error tolerance that exhibits both a linear and quadratic dependence on the distance, then it is necessary to use the method `SetParameters` and define values for both the variables *dLinearFactor* and *dQuadraticFactor*.

Calculation of Distance

When distance-based dead reckoning is used the developer must define the observer on each station via the method `Station::RegisterObserver`. The syntax for this method is:

```
bool Station::RegisterObserver(DOHandle idObserver)
```

If desired, the observer can be changed by unregistering the current observer and registering a different observer. An observer is unregistered via the method `Station::UnregisterObserver`.

The system callback to compute the distance between duplicated objects must be defined using the following syntax:

```
double DuplicatedObject::ComputeDistance(DuplicatedObject*
pObservedDuplicatedObject)
```

This call is always made from the duplication master of the object that is to be updated to the local instance of the observer on the remote station where the duplication master will be updated. Therefore, if on the remote station a duplication master is registered as the observer, the call will be made on a duplica, but if a duplica is the observer depending on the station where the call is made, the call may be made on a duplica or duplication master.

When Net-Z calculates the distance between two objects, z, it is assumed that the datasets of the two objects contain the same variables. Therefore, the distance is typically computed as the difference between their dataset values and is calculated in much the same manner as the extrapolation error. The distance is calculated according to the following equation, where the x_i values are the values of different variables contained in the same dataset, and the subscripts duplicated object and observer refer to the values of the variables for the different objects.

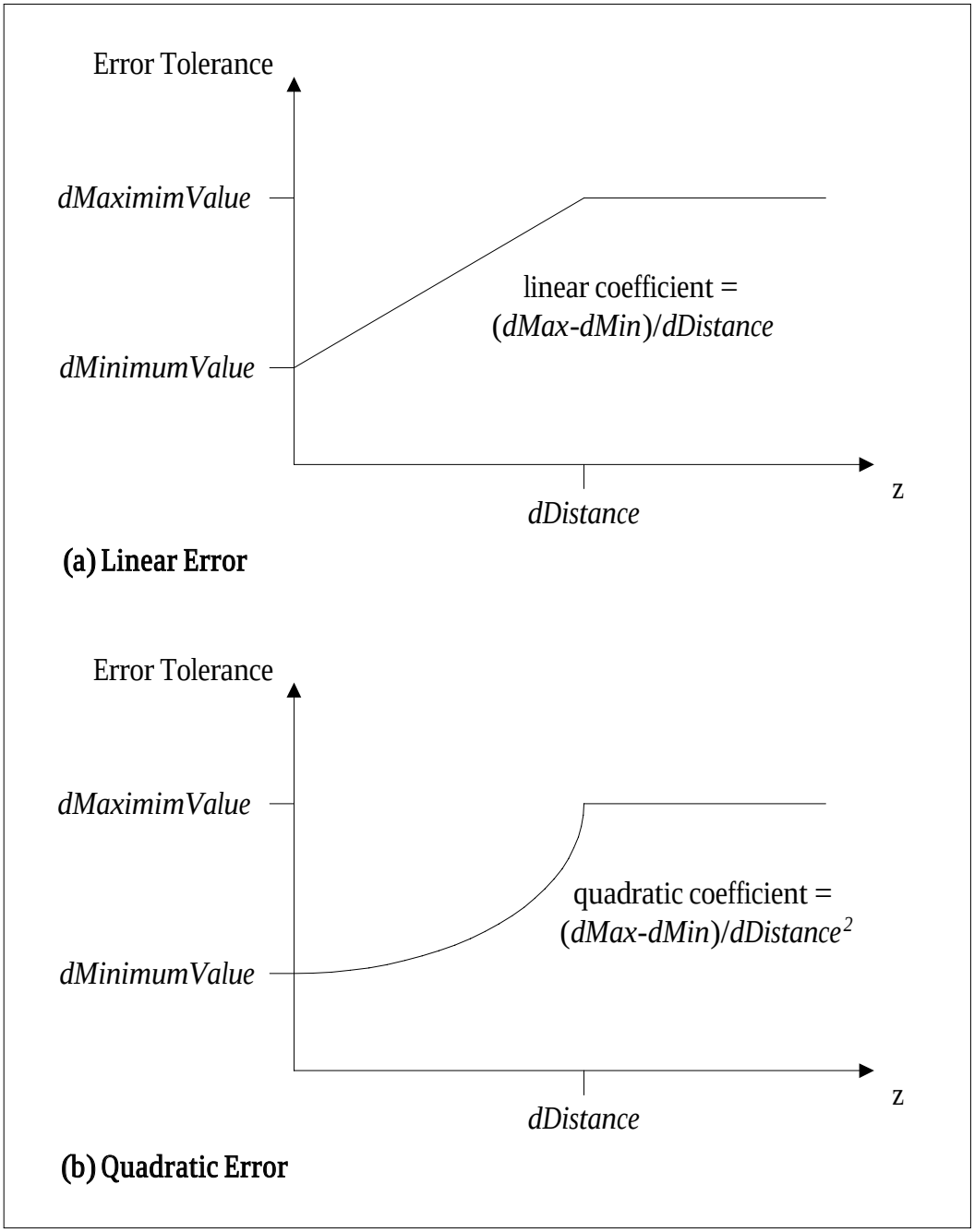


Figure 4.5: Graphical representation of how the methods (a) SetLinearError and (b) SetQuadraticError affect the error tolerance.

$$z = \left(\sum_{i=1}^{n} \left(x_{i\ (DuplicatedObject)} - x'_{i\ (Observer)} \right)^2 \right)^{1/2}$$

Net-Z does not impose any restrictions on how the distance between two objects is calculated. Therefore, if the datasets of the two objects do not contain the same variables it is up to the user to determine how the distance should be computed. For example, if the position of one object is given in Cartesian coordinates (x, y, z) and another object in polar coordinates (r, ϑ), the developer must implement the method used to calculate the distance between the two objects. This could be achieved by converting one set of position coordinates to the other when the distance calculation is performed, by returning a set value dependant on particular values of the dataset, or by any other means that makes sense in the context of the game.

The distance calculation does not depend on the field of view of a player. Therefore, regardless of whether the player can see the object, as the object moves closer to the observer the datasets of the object will be updated more frequently, which naturally uses more bandwidth. Therefore, if a player does a complete circle, as objects come into his field of view they will be shown accurately. On the other hand, if a player is not interested in the objects outside of his field of view, then having them updated frequently uses unnecessary bandwidth. For example, this would occur in a car racing game where the field of view of the observer, which is the driver, is usually in front. When this is the case, the developer can implement methods to customize the distance calculation to take into account the field of view so that objects are not updated frequently when it is not necessary, thus reducing bandwidth usage.

Example of the Implementation of Distance-based Dead Reckoning

The implementation of distance-based dead reckoning is explained in the context of the *SphereZ* sample program. In this example we assume that every process that joins the *SphereZ* session creates two sphere instances: one sphere that is controlled by the user and one sphere that is AI (computer)-controlled. The user-controlled sphere on each station is defined as the observer. Therefore, the distance between the object being updated and the user-controlled sphere on the station where the update is to occur determines how precise an extrapolation is required.

On each station the sphere on which the camera is focused is defined as the observer on that station via the following method:

```
if (m_bHasFocus){
     if (!Station::RegisterObserver(GetHandle())) {
           // Error handling
     }
}
```

Then to compute the distance between the sphere and another object in the Sphere class the syntax below is used. In this example the distance between two objects of the Sphere class is calculated as the distance between their positions. The distance between a Sphere and an object in another class is unknown.

```
double Sphere::ComputeDistance(DuplicatedObject* pDO) {
     switch (pDO->GetClassID()) {
           case DOCLASSID(Sphere):
                 return m_dsPos.Distance(((Sphere*)pDO)->m_dsPos);
           default:
                 return UNKNOWN_DISTANCE;
     }
}
```

The method Distance is implemented separately as part of the class Position as follows:

```
double Position::Distance(const Position& oPosition) {
     return sqrt((x-oPosition.x)*(x-oPosition.x) +
                 (y-oPosition.y)*(y-oPosition.y) +
                 (z-oPosition.z)*(z-oPosition.z));
}
```

The error tolerance is set to vary with the square of the distance between the limits of 0.05 and 1 with the maximum error being reached at a distance of 30 by calling the method `GetErrorTolerance::SetQuadraticError` as follows:

```
Position::GetErrorToleranceFunction()->SetQuadraticError(0.05, 1, 30);
```

Say that a *SphereZ* session is created by Station A and joined by Station B. On every station a total of four duplicated objects are found, two spheres that are controlled by the users and two that are AI-controlled. Of the four duplicated objects two are duplication masters that were created by the station and two are duplicas that were discovered when the session was joined. For example, the duplication masters on Station A are the user-controlled and the AI-controlled spheres that were created by that station, and the duplicas are the two spheres created by Station B. The situation is represented schematically in Figure 4.6 where the spheres controlled by Stations A and B are red and green respectively. When a duplication master is updated on a station by invoking the method `DuplicatedObject::Update`, the following operations are carried out. The given example is for when the method `Update` is called on the AI-controlled sphere on Station A.

i The extrapolation error is computed. Net-Z computes the difference between the real (updated) position of the AI-controlled sphere on Station A (x, y, z) and the position of its duplica on Station B (x´, y´, z´), which is extrapolated. This value is the extrapolation error and is calculated according to the following equation;

$$Extrapolation\ Error = \sqrt{(x - x')^2 + (y - y')^2 + (z - z')^2}$$

ii The distance, *z*, is then computed. On Station A, the duplica of the observer on Station B, in this case the user-controlled sphere on Station B, is located. The distance between the duplica of the observer on B ($x'_{User\text{-}cont(B)}$, $y'_{User\text{-}cont(B)}$, $z'_{User\text{-}cont(B)}$) and the duplication master of the AI-controlled sphere on Station A ($x_{AI\text{-}cont(A)}$, $y_{AI\text{-}cont(A)}$, $z_{AI\text{-}cont(A)}$) is then computed by Net-Z by calling the method Sphere::ComputeDistance, as defined above. Note that when an update occurs on a station, it is the observer that was set on the remote station that determines from which object the distance is calculated.

$$z = \sqrt{\left(x_{AI-cont(A)} - x'_{User-cont(B)}\right)^2 + \left(y_{AI-cont(A)} - y'_{User-cont(B)}\right)^2 + \left(z_{AI-cont(A)} - z'_{User-cont(B)}\right)^2}$$

iii The error tolerance function is then used to compute the tolerated error. Net-Z computes the error tolerance based on the parameters that were set for the error tolerance function as described above. In this example, we called SetQuadraticError(0.05, 1, 30). Therefore, the error tolerance is given by the formula *Error tolerance = 0.05 + [(1-0.05)/900] * z^2*. The error tolerance increases with the square of the distance and has minimum and maximum values of 0.05 and 1 respectively. The minimum error tolerance of 0.05 occurs when *x*=0, *i.e.* when the AI-controlled sphere of A is very close to the user-controlled sphere of B. As the distance increases the error increases until at a distance of 30, the error tolerance reaches its maximum value of 1. If the distance increases further the error tolerance remains constant at its maximum value.

iv Finally, the extrapolation error and the error tolerance are compared. If the extrapolation error is greater than the error tolerance, a message is sent from Station A to Station B to update the position of the duplica of the AI-controlled sphere of Station A (AI-control (A´)) on Station B.

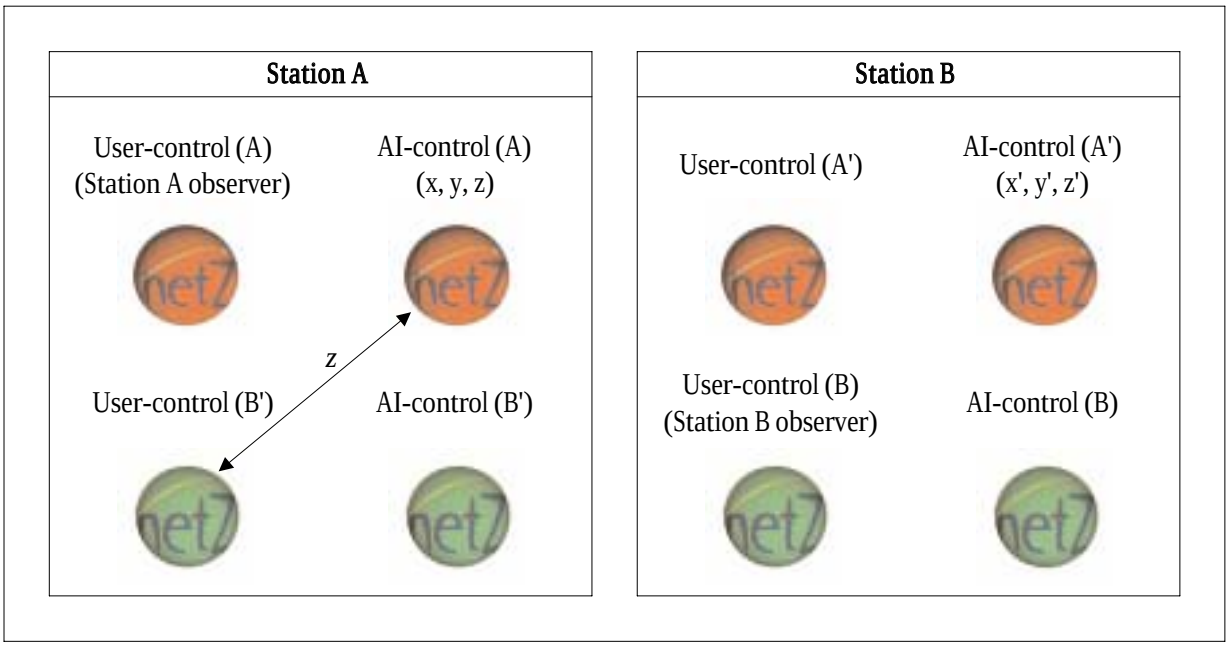


Figure 4.6: Schematic diagram of an update of the position of the sphere AI-control (A) using distance-based dead reckoning. No prime indicates a duplication master and a prime indicates a duplica. The duplication masters have positions (x, y, z) and the duplicas (x´, y´, z´).

4.10.3 Quality Factor

The quality factor is a mechanism used by Net-Z to automatically optimize the appearance of the game on each player's station, as long as there is sufficient bandwidth available to do so. Net-Z achieves this by adjusting the error tolerance, which in turn adjusts the frequency at which the dataset of a duplica is refreshed and therefore, the accuracy at which the duplica is shown.

To use the quality factor function, distance-based dead reckoning must be enabled. Therefore, how accurately a duplica is shown depends on the distance between the object and the observer. When the quality factor is enabled, Net-Z automatically adjusts the quality factor according to the available bandwidth. If there is available bandwidth, then Net-Z will increase the quality factor, which leads to a decrease in the error tolerance. As the error tolerance has decreased, the datasets of the duplica will be refreshed more frequently, resulting in an increased use of bandwidth. If there is a reduction in the available bandwidth the opposite will occur. Net-Z will decrease the quality factor, which leads to an increase in the error tolerance. As the error tolerance has increased, the datasets of the duplica will be refreshed less frequently, resulting in a decreased use of bandwidth. This ensures that as long as bandwidth is available, Net-Z will maximize the quality of the appearance of the duplica. The quality factor set on the local station will affect how accurately the duplicas, which are on a remote station, of

all the duplication masters on the local station are shown. The quality factor should be set the same on all connected stations to maintain consistency in the accuracy at which objects of the same class are shown on each station.

A distance-based error tolerance is set using either of the methods `SetLinearError`, `SetQuadraticError`, or `SetParameters` of the `ErrorToleranceFunction` class. The error tolerance is usually calculated as a function of the distance between the observer and object, *z*, as given in Equation 4.1. In contrast, when the quality factor is invoked the error tolerance is calculated as a function of *z* divided by the quality factor, *q*, as shown below.

$$Error\ tolerance = dConstantFactor + dLinearFactor\left(\frac{z}{q}\right) + dQuadraticFactor\left(\frac{z}{q}\right)^2$$

where the values *dConstantFactor*, *dLinearFactor*, and *dQuadraticFactor* are the parameters of the error tolerance function, as defined via the method `SetParameters`. The range of the quality factor is set via the method `QualityFactor::SetRange`.

As the quality factor increases, the accuracy at which a duplica is shown increases because the error tolerance decreases. As the quality factor decreases, the inverse occurs and the error tolerance increases resulting in the duplica being shown less accurately. The quality factor is set by a call to the `QualityFactor::SetRange` method. This defines the range of values of the quality factor, *dMin*- *dMax*, within which Net-Z will adjust the error tolerance. A quality factor of 1 has no effect on the error tolerance and is the default value for both *dMin* and *dMax*. The syntax for the method is:

```
static bool QualityFactor::SetRange(double dMin, double dMax);
```

For example, the following line of code sets the quality factor to the range 0.01-100:

```
static bool QualityFactor::SetRange(0.01,100);
```

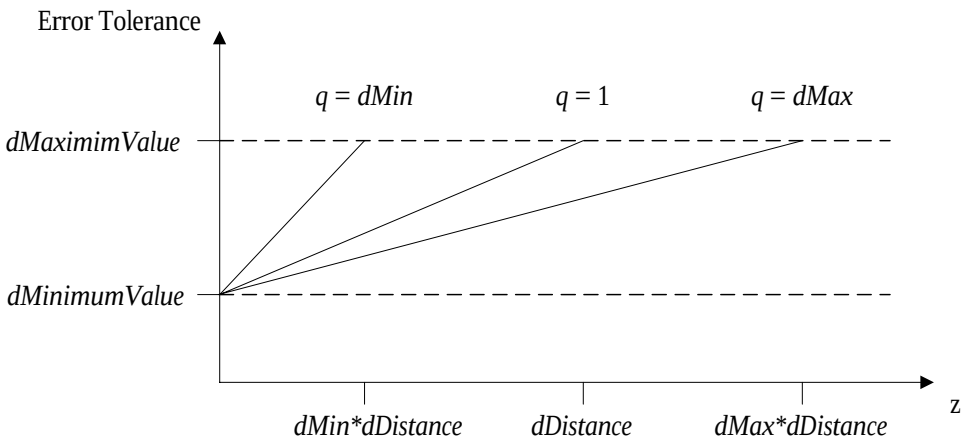
Figure 4.7 illustrates the effect of the quality factor and the distance between objects on the error tolerance. When the quality factor is equal to 1, it has no effect on the error tolerance. Therefore, the error tolerance behaves as described in §4.10.2 and the maximum error is reached at a distance of *dDistance*. When the quality factor is invoked, the error tolerance is proportional to the distance between objects and inversely proportional to the quality factor. Therefore, when using either a linear or quadratic error tolerance function, the maximum error is reached when the distance between the object and the observer is equal to *q*dDistance*. If the error tolerance is set with both a linear and quadratic dependence on the distance between the observer and object, then the error tolerance will be given by the following equation. When this is this case, the maximum error will be reached when the distance, *z*, is approximately equal to 0.618*q*dDistance*.

$$Error\ tolerance = dMinimimValue + \left(\frac{dMaximumValue - dMinimumValue}{dDistance}\right)\left(\frac{z}{q}\right) + \left(\frac{dMaximumValue - dMinimumValue}{dDistance^2}\right)\left(\frac{z}{q}\right)^2$$

When automatically adjusting the quality factor, Net-Z uses the packet drop probability to determine if a remote station is being flooded and therefore, dropping packets. It is set via the method `InputEmulationDevice::SetPacketDropProbability` and is the probability that a given packet will not be received by the remote station. When the packet loss ratio increases above the tolerated level, Net-Z will decrease the quality factor, and therefore the number of packets being sent to a station, until the ratio decreases below the set value.

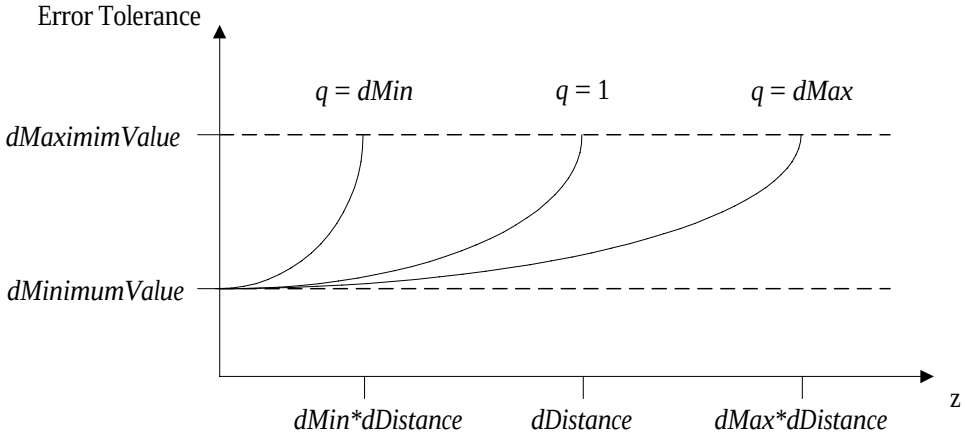
The choice of the maximum and minimum quality factor is determined largely by how your game functions under different network conditions. For example, if your game will not function satisfactorily on say a 33K modem, then you can set the quality factor such that the maximum tolerated error corresponds to a connection of this speed. If on the other hand, players with very fast connections gain an unfair advantage over those with slower connections, then you can set the minimum tolerated error such that the unfair advantage is eliminated.

When determining the quality factor it is beneficial to define the base line conditions at which you wish your game to function and then optimize by trial and error using the performance counters that are accessed via the `Transport` class. The game can be tested at various network conditions using the `EmulationDevice` class.



$$Error\ tolerance = dMinimimValue + \left(\frac{dMaximumValue - dMinimumValue}{dDistance} \right) \left(\frac{z}{q} \right)$$

(a) Linear Error



$$Error\ tolerance = dMinimimValue + \left(\frac{dMaximumValue - dMinimumValue}{dDistance^2} \right) \left(\frac{z}{q} \right)^2$$

(b) Quadratic Error

Figure 4.7: Graphical representation of the error tolerance when (a) a linear error is set using SetLinearError, and (b) a quadratic error is set using SetQuadraticError, and the quality factor is invoked.

4.10.4 Fine-tuning Dead Reckoning

You can fine-tune the dead reckoning algorithm used by Net-Z in various ways. In general, the default values used by Net-Z will produce a satisfactory dead reckoning performance, but you may change how dataset updates are treated by Net-Z in order to optimize game performance. The choice of the various fine-tuning mechanisms will depend on the type of dataset, but the dataset must use an extrapolation filter.

Dataset Updates

When a remote station receives a dataset update from a duplication master two steps are performed to update the dataset model: a tracking step and then a convergence step. The tracking step, which is simply the extrapolation of the dataset, uses the dataset values from previous updates to predict the values of the dataset until the next update is received. The convergence step adjusts the local values of the dataset so that the values converge smoothly to the predicted path.

When an extrapolation filter is used on a dataset, prediction of each of the variables of the dataset is made using a tracking algorithm. The algorithm used may be either linear or non-linear. When a linear algorithm is employed the two most recent updates are used to predict the variable, whereas for a non-linear, or parabolic, algorithm the three most recent updates are used. The choice of which algorithm is more accurate depends on how much the variable changes between updates, which is defined by the *angle of embrace,* ϑ. As illustrated in Figure 4.8, if the values of the variable remain relatively unchanged over time then the angle of embrace is large and non-linear tracking is used. However, if the variable undergoes a sudden change over time then the angle is small and linear tracking is used.

The tracking angle threshold is used to determine if a linear or non-linear prediction algorithm is used. The threshold, which may be specified in radians or degrees, is set for each dataset using the method `PHBDRParameters::SetTrackingAngleThreshold`. The default unit for the angle is radians. If the angle of embrace is less than the tracking threshold angle, then a linear tracking algorithm is used. If the angle of embrace is larger, then a non-linear tracking algorithm is used. To ensure coherence between the datasets of objects on different stations, the tracking angle threshold must be set the same on each station across the network.

The second step that occurs after a dataset update is the convergence of each individual variable of the dataset on the remote station to the values given by the update from the duplication master. If no convergence is implemented then when a duplica receives an update message the values of the dataset will change directly to that of the update. Therefore, if the dataset is a position, the object will appear to jump from one position to another. On the other hand, when a convergence algorithm is used then the values of the dataset will smoothly converge to the path predicted by the tracking algorithm. By default, Net-Z implements the convergence step. If you do not wish to use the convergence step then the method `PHBDRParameters::ApplyConvergence` should return false.

As for tracking, a linear or non-linear (parabolic) algorithm may be used to determine the convergence path, the choice of which is based on the angle of embrace. As illustrated in Figure 4.9, a large angle of embrace indicates that the values of the variable are almost constant and a linear convergence is used. On the other hand, a small angle of embrace indicates that the variable has changed significantly and therefore, a non-linear convergence algorithm is used. Once the variable has converged to the path predicted by the tracking algorithm, it follows this path until the next update is received, after which the process is repeated.

The time taken for the variable to converge from the old values to a value on the predicted path is called the convergence delay, *CPDelay*. Net-Z uses a heuristic approach to compute the convergence delay and the minimum and maximum delay for the convergence can be defined by using the method `PHBDParameters::SetConvergenceDelay`. The convergence delay should be set so that the variable converges to the predicted path before the next update is received. If this is not the case the extrapolation will appear irregular.

The convergence angle threshold is used to determine if a linear or non-linear convergence algorithm is used. The threshold, which may be specified in radians or degrees, is set for each dataset using the method `PHBDRParameters::SetConvergenceAngleThreshold`. The default unit for the angle is radians. If the angle of embrace is less than the convergence threshold angle, then a non-linear tracking algorithm is used. If the angle of embrace is larger, then a linear tracking algorithm is used. As the convergence algorithm is only implemented on the local station, each station across the network may set a different convergence angle threshold.

The choice of appropriate values for the tracking angle and convergence angle thresholds and the convergence delay depends on the dynamics of the particular dataset. A summary of the general criteria for the selection of the parameters is given in Table 4.1. For example, assume that the dataset relates to the position of an object. If the object undergoes frequent changes in its motion, such as a ball, then the tracking angle threshold should be set to a high value to ensure that linear tracking is usually used, and the convergence delay should be set to a low value to ensure that the object quickly converges to the predicted path. If a high convergence delay was set it is likely that the next update would be received before the object had converged to the predicted path, which would result in the object having an irregular trajectory. However, if the object exhibits a relatively smooth motion, such as an aeroplane, then the tracking angle threshold should be low so that non-linear tracking is used and the convergence delay set high so that the motion of the object remains smooth. The value of the convergence angle threshold largely depends on how smooth you wish the motion of the duplica on the local station to be. If the convergence angle threshold is high then non-linear convergence is used more frequently on updates and therefore, the path of the object is smooth. If a low convergence angle threshold is used then linear convergence is used and the object is more likely to have an irregular trajectory.

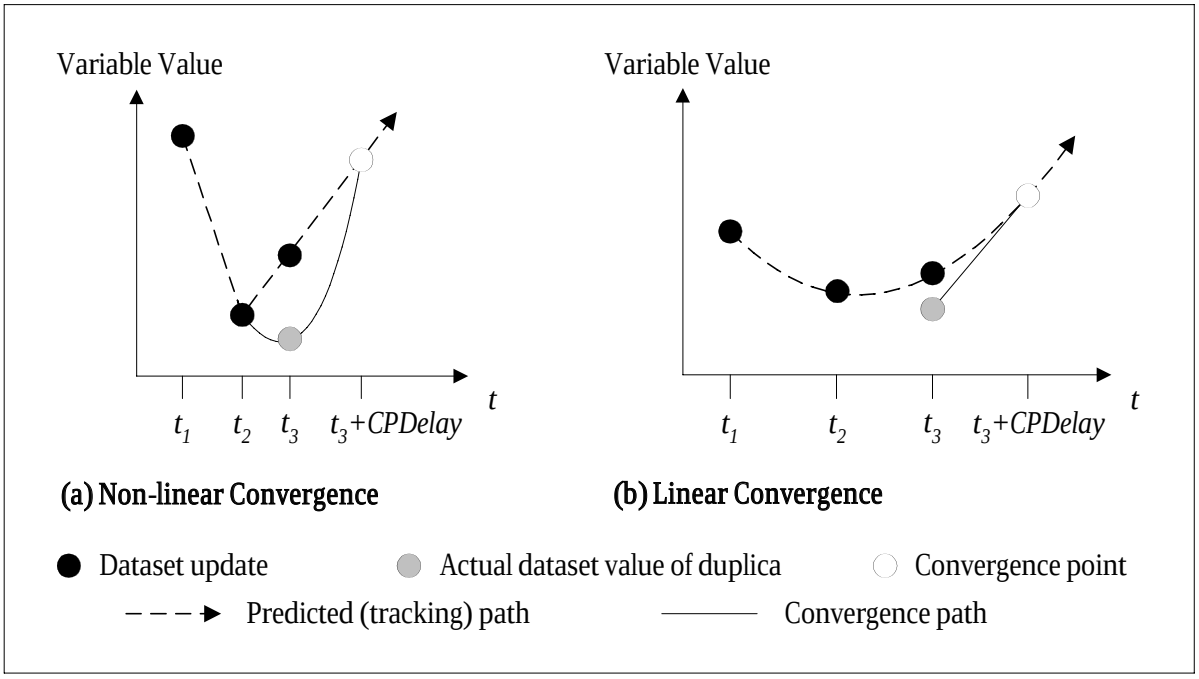


Figure 4.8: Tracking angle for (a) linear and (b) non-linear tracking algorithm for a single variable of a dataset.

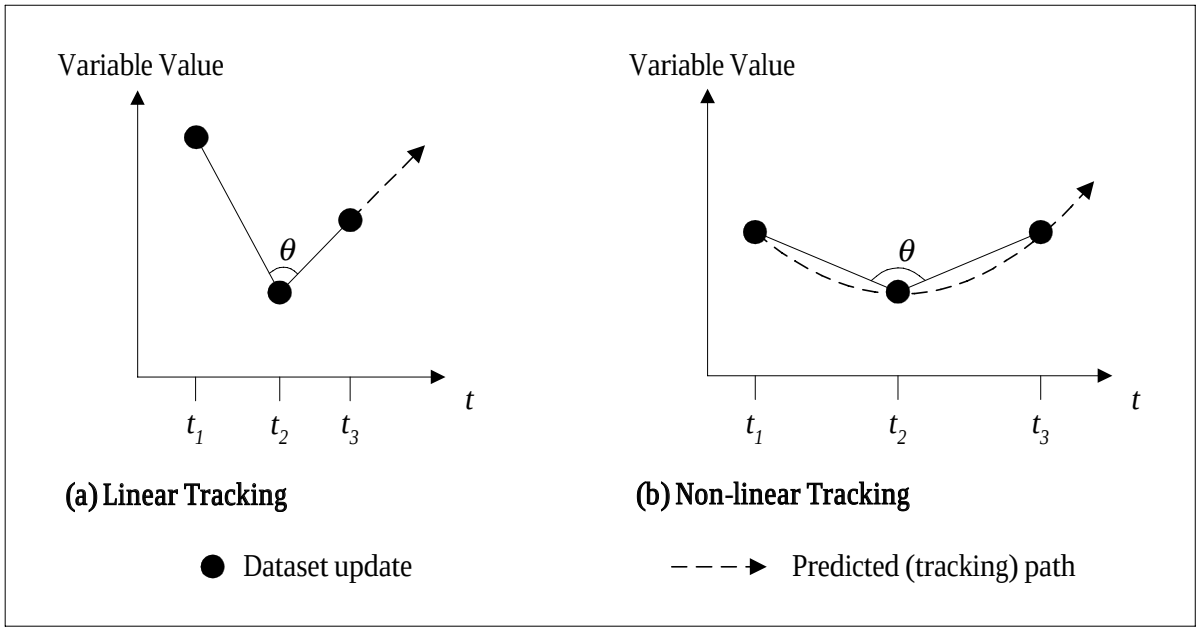


Figure 4.9: Convergence path for (a) non-linear and (b) linear convergence algorithm for a single variable of a dataset.

Parameter	Characteristic	Setting
Tracking Angle Threshold	Dataset values change frequently	High
	Dataset values relatively constant	Low
Convergence Angle Threshold	Smooth trajectory required	High
	Smoothness of trajectory unimportant	Low
Convergence Delay	Dataset values change smoothly	High
	Dataset values can change suddenly	Low

Table 4.1: General criteria for the selection of tracking and convergence parameters.

For most datasets the default values of the tracking angle and convergence angle thresholds and the convergence delay supplied by Net-Z will be adequate and it will be unnecessary for the user to adjust these values. If you do wish to fine-tune the updates of datasets, then the criteria given in Table 4.1 should be used as a guide and then optimization performed via trial and error for each individual dataset. It should be noted that the default values for the tracking angle and convergence angle are large because it is assumed that the time scale is much greater than the variable scale.

Postdate Updates

The latency between two stations causes a delay between the time at which an update is sent by the duplication master and when the duplicas receive the update. Therefore, when the dataset of a duplication master is changed, there is a time lag before the duplicas are updated. This can lead to an inaccurate extrapolation of the datasets of the duplicas, especially if the values of the dataset change significantly. To handle this situation, Net-Z can postdate all updates to the duplicas by changing the timestamp on the message. This enables Net-Z to accurately predict the dataset of the duplicas as there is a time lag between display of the duplication master and its duplicas. Naturally postdating updates results in a trade-off between the duplica displaying the correct, but delayed dataset values, and incorrect values that are time synchronized with the duplication master.

Postdating of dataset updates is implemented by the method `PHBDRParameters::PostdateUpdates`, as shown below where the time is given in milliseconds.

```
PHBDRParameters::PostdateUpdates(TimeInterval tiDelay);
```

For example, the following line of code would postdate updates by 100ms:

```
PHBDRParameters::PostdateUpdates(100);
```

It should be noted that only dataset updates from a duplication master are postdated and that other messages such as RMCs, actions, and fault recovery operations are not postdated. Therefore, in some situations this may lead to the inaccurate display of a duplica or to runtime errors.

Continuity Breaks

Continuity breaks occur when the values of a dataset change in a non-continuous way, such as when an object is teleported or bounces off a wall. For example, if an avatar is teleported from one position to another, it makes no sense to interpolate between the two positions. The indication of continuity breaks helps Net-Z perform a smoother dead reckoning by allowing it to disregard the extrapolation error. When a continuity break occurs the dataset will be updated a certain number times, which is dependant on the type of continuity break, regardless of the extrapolation error. As explained in §4.10.2, the extrapolation error is the difference between the dataset values of a duplication master on the local station and the extrapolated dataset values of the duplica on a remote station.

A continuity break is indicated by the method `DataSet::IndicateContinuityBreak`, which is declared in the following fashion, where *byBreak* indicates the type of continuity break:

```
DataSet::IndicateContinuityBreak(unsigned char byBreak);
```

The valid types of continuity breaks are stop, sudden change, and teleport. As their names suggest they are used to indicate when the values of a dataset suddenly stop, change, or jump from one set of values to another. As well as indicating a continuity break, if we know how the dataset will change when a continuity break occurs, we can use a continuity break update model to provide Net-Z with additional information about how a particular type of continuity break will affect the dataset. An update model may be used if the continuity break indicated is a teleport or sudden change. This allows Net-Z to react more quickly to the sudden change in dataset values and to predict the dataset values more accurately. For example, the spheres in our *SphereZ* game regularly collide with the walls and undergo a sudden change in the values of the Position dataset. Therefore, to indicate the continuity break when a collision occurs the following code is used:

```
m_dsPos.IndicateContinuityBreak(CONTINUITY_BREAK_SUDDEN_CHANGE |
CONTINUITY_BREAK_UPDATED_MODEL);
```

In this case we have also indicated that an update model, which contains additional information about the continuity break, is associated with the break. The update model is set via the method `DataSet::SetModel` and accessed via the macro `MODEL`. For example, the update model for the collision of a sphere with the wall is as follows. It defines the new position, speed, and acceleration of the sphere in the *x*, *y*, and *z* directions.

```
m_dsPos.SetModel(MODEL(m_dsPos,x), m_dsPos.x, m_Speed.sx/1000, 0);
m_dsPos.SetModel(MODEL(m_dsPos,y), m_dsPos.y, m_Speed.sy/1000, 0);
m_dsPos.SetModel(MODEL(m_dsPos,z), m_dsPos.z, m_Speed.sz/1000, -
9.0/1000000);
```

The method `SetModel` is called on the duplication master, when the change occurs, to provide Net-Z with additional information about a sudden change to an extrapolated dataset. Therefore, rather than the duplica requiring to wait for three updates to be received so that it has enough information to be able to extrapolate the dataset, information about how the dataset changes immediately after the continuity break can be sent from the duplication master to the duplica immediately after the occurrence of the continuity break. This enables the dataset of the duplica to be shown more accurately when a continuity break occurs.

Local Corrections

Local corrections are performed on the local station and are used to correct the inaccurate extrapolation of the dataset of a duplica before the subsequent update is received from the duplication master. For instance, on each station we know that the duplica of a sphere should not pass through the walls. Although, due to network latency extrapolation of the position of the duplica of a sphere may lead to a sphere being displayed in or past a wall. If this occurs we can correct an inaccurate extrapolation of the position of the duplica on the local station before an update is received from the duplication master. This correction does not have to be exact, as the position of the duplica will converge to that of the duplication master when the position is updated. Therefore, by implementing a local correction the positions of the duplicas of the spheres are shown more accurately. The local correction on the duplica is made via the method `DataSet::SetLocalCorrection` and is called on the duplica. When this method is invoked, updates from the duplication master are ignored and the local extrapolation model is used for the amount of time specified in the method `PHBDRParameters::SetLocalCorrectionExpirationDelay`. When setting local corrections the values of a dataset of a duplica can be predicted using the methods of the `DataSet` class `PredictValue`, `PredictRateOfChange`, and `PredictAcceleration`. These methods predict the values of a dataset based on the current values. For example, we can set a local correction for the duplicas of all spheres to ensure they are not displayed in a wall. The code below indicates that when a sphere collides with a wall its position remains the same, but its velocity and acceleration changes. The velocity in the *x* and *y* directions is the negative of the current value predicted by the method `PredictRateOfChange`. In the *z* direction the sphere is given an arbitrary velocity and a negative acceleration to account for gravity.

```
if (ComputeXWallCollision()) {
     double dCurrentSpeed=
m_dsPos.PredictRateOfChange(MODEL(m_dsPos,x));
     m_dsPos.SetLocalCorrection(MODEL(m_dsPos,x), m_dsPos.x, -
dCurrentSpeed, 0);
     m_dsPos.SetLocalCorrection(MODEL(m_dsPos,z), m_dsPos.z,
10.0/1000, -9.0/1000000);
}
if (ComputeYWallCollision()) {
     double dCurrentSpeed=
m_dsPos.PredictRateOfChange(MODEL(m_dsPos,y));
     m_dsPos.SetLocalCorrection(MODEL(m_dsPos, y), m_dsPos.y, -
dCurrentSpeed, 0);
     m_dsPos.SetLocalCorrection(MODEL(m_dsPos, z), m_dsPos.z,
10.0/1000, -9.0/1000000);
```

4.11 Call Contexts

To invoke a RMC or to fetch or migrate objects the context of the call must be defined via the `RMCContext`, `FetchContext`, and `MigrationContext` classes. These three classes inherit from the `CallContext` class. Via these classes you set the target station of the call context and the appropriate flags. You can also obtain information about the `State` and the `Outcome` of the call context and have the ability to cancel or reset a call context.

Via the `CallContext` class the flags `TargetObjectMustBeMaster`, `SynchronousCall`, and `CancellationNotAllowed` are set. These flags respectively ensure that the target object that the call context is called on is a duplication master, make the call context synchronous, and ensure that the call context cannot be cancelled. If the flag `SynchronousCall` is not set the system will operate in an asynchronous manner. Therefore, the thread from which the call context was made will not block while it waits for a response from the call context. When a response is received the `State` and the `Outcome` of the call context will change. When a call context is made asynchronously, the method `Wait` can be used to set a timeout to specify the length of time that the thread from which `Wait` was called will wait for the call context to be processed before proceeding. If a response is received before the end of the timeout then the `Wait` call will terminate. When the call context is made in a synchronous manner, the thread where the call context was initiated will wait until it receives a response from the call context before proceeding. Effectively, this is equivalent to setting the wait time to infinite on an asynchronous call. The flag may be set and cleared via the methods `SetFlag` and `ClearFlag` respectively and the method `FlagIsSet` can be used to determine if a particular flag is set. Also, the methods `SetTargetStation` and `GetTargetStation` may be used to respectively set and obtain the target station of the call context. The significance of the target station depends on the specific call

context. For a `RMCContext`, the target station is the station that the RMC is called on, for a `FetchContext` it is the station from which we wish to fetch a duplica, and for a `MigrationContext` it is the station to which we wish to migrate and object.

A call context may be in one of five states: Available, CallInProgress, CallSuccess, CallFailure, or CallCancelled as illustrated in Figure 4.10. A call context is initially in the Available state. When it is called by the user it passes to the CallInProgress state from which it may pass to one of the states CallSuccess, CallFailure, or CallCancelled. When a call context is in the CallInProgress state the user may cancel the call context via the method `Cancel` and the state will pass to CallCancelled. When the state is one of CallSuccess, CallFailure, or CallCancelled the method `Reset` may be called which will reset the state to Available. The method `GetState` will return the current state of the call context. If `Cancel` is called and the `CallContext` is not allowed to be cancelled, as is the case for a migration call, the method will lock until the call context has terminated.

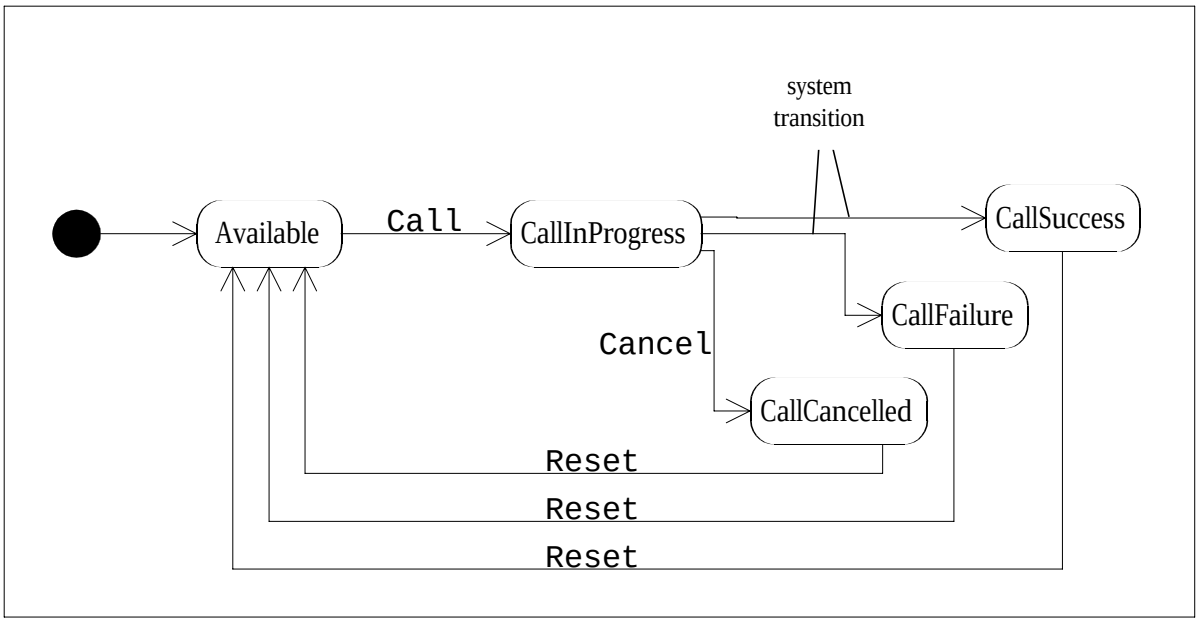


Figure 4.10: State diagram for call contexts.

The manner in which a call context terminates can be obtained via the method `GetOutcome`. It may be one of nine outcomes: Success, ErrorInvalidRole, ErrorObjectNotFound, ErrorLocalStationLeaving, ErrorStationNotReached, ErrorTargetStationDisconnect, ErrorRMCDispatchFailed, ErrorMigrationInProgress, or UnknownOutcome. Success will be returned if the call context terminated successfully, ErrorInvalidRole if the call context was called on a duplica when the flag `TargetObjectMustBeMaster` was set, ErrorObjectNotFound if the object could not be found on the target station which may be due to the object being deleted or not currently existing on that station, ErrorLocalStationLeaving if the local station is leaving the session,

ErrorStationNotReached if the request was sent but it did not reach the target station, ErrorTargetStationDisconnect if the request was sent but the target station was disconnected before a response was received, and UnknownOutcome if the outcome of the call context is as yet unknown as it has not returned a response. ErrorRMCDispatchFailed is only valid for a RMC and will be returned if the request could not be sent from the initiating station. Similarly, ErrorMigrationInProgress is only valid for a migration call and will be returned if a migration call is in progress.

4.12 User-defined Contexts

The user-defined value passed via the `UserContext` class may be either an unsigned long, double, Boolean, or pointer to an object. It is typically used to pass a value that the user wishes to retain, which is not automatically retained by Net-Z. The `UserContext` class would typically be used in conjunction with the `Operation` class, however it may be used in relation to any other class in the game. For example, when a MasterIDChange operation occurs the original MasterID of the object is not retained by Net-Z. However, if you wish to retain this information it can be passed as user-defined data before the operation begins and retrieved after the operation ends. When user-defined information is passed in the context of an operation the methods `SetUserData` and `GetUserData` of the `Operation` class should be used as detailed in §4.14.

4.13 Iterators

Methods called and performed locally on duplicated objects are implemented using standard C++ programming techniques. To perform a method on a particular object you simply create a reference to the object and then call the method on it directly as described in §4.2. To call a method over a group of duplicated objects iterators are used.

On each station Net-Z keeps a list of all duplicated objects and two subsets of this list containing all duplication masters and all duplicas. These lists may be accessed via the `SelectionIteratorTemplate` class. This class enables you to iterate over a specific class of objects. Three different selection iterators can be constructed: an iterator that holds all duplicated objects, all duplication masters, or all duplicas. The default construction creates an iterator that holds all duplicated objects. The methods `GotoStart`, `GotoEnd`, `EndReached`, and `Count` may be used within an iterator loop to set the pointer of the iterator to point to the first or last object in the object list, determine when the end of the list is reached, and to return the total number of objects in the list respectively.

This example is from the *RoomZ* sample. If we wish to call the method FindByName on all duplicated objects in the Delegate class we create a selection iterator that holds all duplicated objects, point to the first duplicated object in the iterator list and then call the method on each object in the list until the end of the iteration list is reached as follows;

```
DOHandle Delegate::FindByName(TCHAR* strName) {
     Delegate::SelectionIterator itALLDelegate;
     itALLDelegate.GotoStart();
     while(! (itALLDelegate.EndReached())) {
           if (_tcscmp(itALLDelegate->GetName(), strName)==0 {
                 return(itALLDelegate->GetHandle());
           }
           itALLDelegate++;
     }
     return (INVALID_DOHANDLE);
}
```

4.14 Net-Z System Operations

An operation is simply an event that occurs within the system and typically results from a network message. The various types of operations that can occur in Net-Z are sub classed into duplicated object operations, dataset operations, and duplication space operations, which in turn are sub classed into each type of system operation. A duplicated object operation, a dataset operation, and a duplication space operation are operations that are performed on respectively a particular duplicated object, a particular dataset, and a particular duplication space. The hierarchy of the operation classes is detailed in Figure 4.11. Details of the sequence of events that occurs during each operation and the locks used by each operation are described later in this section. The methods `OperationBegin` and `OperationEnd` of the `DuplicatedObject`, `DataSet`, and `DupSpace` classes are called with a pointer to the operation class as a parameter.

The methods of the `Operation` class may be used to gain further information about a specific operation and are particularly useful when implementing traces for debugging purposes. The implementation of system traces is further discussed in §4.19. To obtain the type of operation performed the methods `GetType` and `GetClassNameString` can be used to return the operation type as a value or as a string respectively. The method `HasALock` can be used to confirm if the operation has a lock associated to it. A context may be attached to an operation via the methods `SetUserData` and `GetUserData`. The user-defined value passed via the `UserContext` class may be either an unsigned long, double, Boolean, or pointer to an object. It is typically used to pass a value that you wish to retain but is not automatically retained by Net-Z. For example, if we wish to know when an object has migrated due to the occurrence of a MasterIDChange operation, we could implement the following code:

```
void Avatar::OperationEnd(DOOperation* pOperation) {
     switch (pOperation->GetType()) {
           case Operation::MasterIDChange:
                 _printf(_T("The Avatar %x has migrated"), pOperation
->GetAssociatedDOHandle);
                 break;
     }
}
```

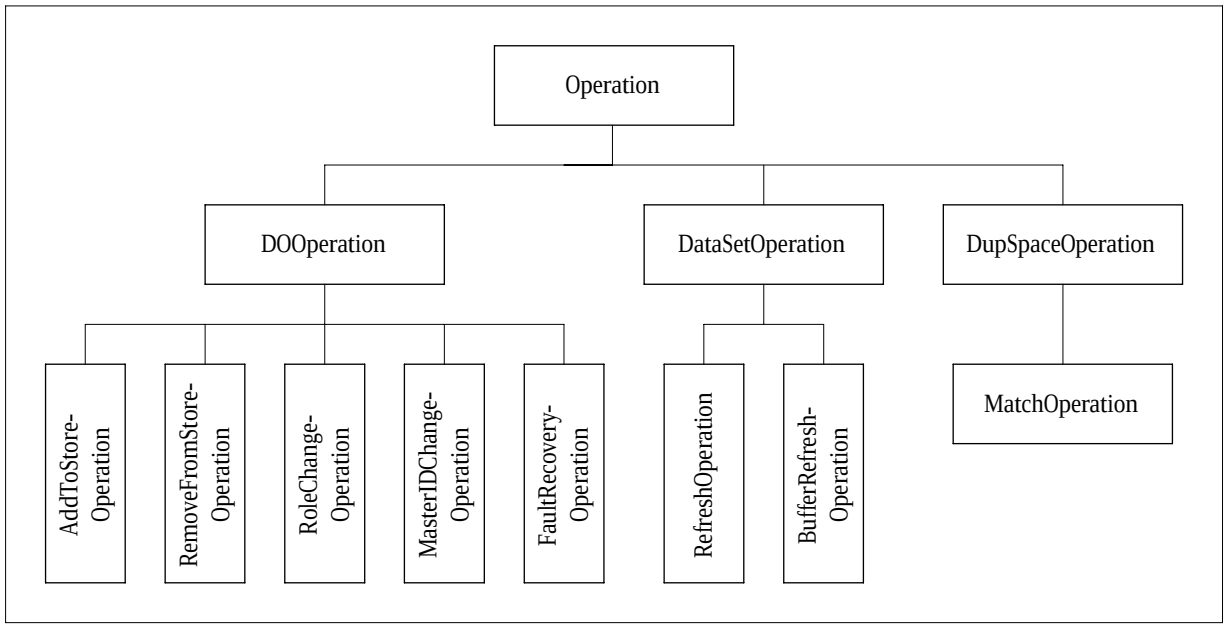


Figure 4.11: Hierarchy of the Operation class

4.14.1 Operation Sequences

The operations that can occur in Net-Z are: AddToStore, RemoveFromStore, RoleChange, MasterIDChange, FaultRecovery, Refresh, BufferRefresh, and Match. When a particular system operation occurs specific events will take place in a predefined sequence, as detailed below. It is important to understand the sequence of events for each operation so that you do not inadvertently deadlock your program. For each of these operations the methods `SetTraceFlag`, `GetTraceFlag`, and `ClearTraceFlag` can be used in order to trace the behaviour of the operations. These methods may be used when the trace flag `TRACE_OPERATION` is set via the method `TraceOutput::SetTraceFlag`. The `TRACE_OPERATION` flag will by default trace all the system operations. However, the trace methods of each operation class can be used to disable and re-enable the tracing of an operation. The implementation of system traces is further discussed in §4.19.

AddToStore

An object can be added to the duplicated object store as either a duplication master or a duplica. When an object is published on a station, either by the user or by Net-Z, the object is always added as a duplication master. On the other hand, when an object is discovered it is added as a duplica.

When an object is added to the duplication master store the following sequence of events occurs:

- The object constructor is called,
- A DOHandle and MasterID are assigned to the object. The methods `DuplicatedObject::IsADuplica` and `DuplicatedObject::IsADuplicationMaster` are now valid,
- The method `DuplicatedObject::Publish` is called,
- The method `DuplicatedObject::OperationBegin` is called and the AddToStore operation begins,
- The method `DuplicatedObject::InitDO` is called,
- The duplicated object is added to the duplication master store,
- The method `DuplicatedObject::OperationEnd` is called and the AddToStore operation ends.

When an object is added to the duplica store the following sequence of events occurs:

- The object constructor is called,
- A DOHandle and MasterID are assigned to the object. The methods `DuplicatedObject::IsADuplica` and `DuplicatedObject::IsADuplicationMaster` are now valid,
- The method `DuplicatedObject::OperationBegin` is called and the AddToStore operation begins,
- The datasets of the object are initialized,
- The method `DuplicatedObject::InitDO` is called,
- The duplicated object is added to the duplica store,
- The method `DuplicatedObject::OperationEnd` is called and the AddToStore operation ends.

RemoveFromStore

When an object is removed from the duplicated object store it may be either a duplication master or a duplica. A duplication master will be removed as a result of the object being destroyed, whereas a duplica can be removed as a result of its duplication master being destroyed or the duplica becoming irrelevant on a particular station, as may occur when duplication spaces are implemented. It should be noted that an object will not be deleted until all the valid references to it are released.

An object will be removed from the duplication master store if the method `DuplicatedObject::ApproveFaultRecovery` returns false or if the object is deleted via the method `DuplicatedObject::DeleteMainRef`. When an object is removed from the duplication master store, the following sequence of events occurs:

- The method `DuplicatedObject::OperationBegin` is called and the RemoveFromStore operation begins,
- The duplicas of the duplication master object are notified of the deletion via a multicasted message,
- The reference to the duplication master in the duplication master store is deleted,
- The method `DuplicatedObject::OperationEnd` is called and the RemoveFromStore operation ends. Iterations can no longer be performed over the duplication master object.
- When the number of references to the object, created via the `RefTemplate`, on the local station equals 0 the object destructor is called.

An object will be removed from the duplica store if the duplica receives a message from its duplication master notifying it that the master will be deleted or if either of the methods `DuplicationSpace::CheckRelevance` or `DuplicationSpace::PSMatch` return false. When an object is removed from the duplica store, the following sequence of events occurs:

- The method `DuplicatedObject::OperationBegin` is called and the RemoveFromStore operation begins,
- The reference to the duplica in the duplica store is deleted,
- The method `DuplicatedObject::OperationEnd` is called and the RemoveFromStore operation ends. Iterations can no longer be performed over the duplica object.
- When the number of references to the object, created via the `RefTemplate`, on the local station equals 0 the object destructor is called.

RoleChange

The role of a duplicated object may be either a duplication master or a duplica and may change during a session via a RoleChange operation, as detailed below. A RoleChange operation is always encapsulated within a MasterIDChange operation to ensure that when a RoleChange occurs it is always accompanied by a MasterID change.

- The method `DuplicatedObject::OperationBegin` is called and the RoleChange operation begins,
- The role of the duplicated object changes from a duplica to a duplication master, or vice versa,
- The MasterID is changed to that of the new duplication master,
- The method `DuplicatedObject::OperationEnd` is called and the RoleChange operation ends.

MasterIDChange

A MasterID change will occur as a result of the migration of the duplication master of a duplicated object. If the object on the local station is a duplica and remains a duplica, only a MasterIDChange operation is performed. On the other hand, if as a result of the migration the duplicated object on the local station changes its role, then a RoleChange operation will also occur.

When only a MasterIDChange operation occurs the following sequence of events occurs:

- The method `DuplicatedObject::OperationBegin` is called and the MasterIDChange operation begins,
- The MasterID is changed to that of the new duplication master,
- The method `DuplicatedObject::OperationEnd` is called and the MasterIDChange operation ends.

When a RoleChange is encapsulated within a MasterID change operation the following sequence of events occurs:

- The method `DuplicatedObject::OperationBegin` is called and the MasterIDChange operation begins,
- The method `DuplicatedObject::OperationBegin` is called and the RoleChange operation begins,
- The role of the duplicated object changes from a duplica to a duplication master, or vice versa,
- The MasterID is changed to that of the new duplication master,

- The method `DuplicatedObject::OperationEnd` is called and the RoleChange operation ends,
- The method `DuplicatedObject::OperationEnd` is called and the MasterIDChange operation ends.

FaultRecovery

When a station fails, a FaultRecovery operation is performed as follows on the session master for each duplicated object whose duplication master resided on the failed station.

- The method `DuplicatedObject::OperationBegin` is called and the FaultRecovery operation starts,
- The method `DuplicatedObject::ApproveFaultRecovery` is called.

If `DuplicatedObject::ApproveFaultRecovery` returns true the session master takes control of the duplicated object via the following sequence of events:

- The MasterIDChange and RoleChange operations are performed,
- The method `DuplicatedObject::OperationEnd` is called and the FaultRecovery operation ends.

If `DuplicatedObject::ApproveFaultRecovery` returns false the object is removed from the store via the following sequence of events:

- The RemoveFromStore operation is performed,
- The method `DuplicatedObject::OperationEnd` is called and the FaultRecovery operation ends,
- When the number of references to the object, created via the `RefTemplate`, on the local station equals 0, the object destructor is called.

Refresh

The Refresh operation can only be performed on a duplica and is the only system operation that does not use a lock. If the operation is performed on a buffered dataset then the BufferRefresh operation is denied while a Refresh operation is in progress.

If the dataset is unbuffered the following sequence of events occurs:

- The method `DataSet::OperationBegin` is called as the result of a network message and the Refresh operation starts,
- The dataset members of the duplica are refreshed from the duplication master,
- The method `DataSet::OperationEnd` is called and the Refresh operation ends.

If the dataset is buffered the following sequence of events occurs:

- The method `DataSet::OperationBegin` is called as the result of the user calling the method `DataSet::Refresh` and the Refresh operation starts,
- The BufferRefresh operation is denied,
- The dataset members of the duplica are refreshed from the buffer,
- The BufferRefresh operation is permitted,
- The method `DataSet::OperationEnd` is called and the Refresh operation ends.

BufferRefresh

The BufferRefresh operation proceeds as follows and can only be performed on a buffered dataset of a duplica.

- The method `DataSet::OperationBegin` is called as the result of a network message and the BufferRefresh operation starts,
- The dataset members of the buffer of the duplica are refreshed from the duplication master,
- The method `DataSet::OperationEnd` is called and the BufferRefresh operation ends.

Match

When the match operation is called on a duplication space the following sequence of events occurs:

- The method `DuplicationSpace::OperationBegin` is called and the Match operation begins,
- The appropriate `PSMatch` or `CellMatch` sequence is called,
- According to the return value of the `PSMatch` or `CellMatch` function the AddToStore or RemoveFromStore operations may be triggered,
- The method `DuplicationSpace::OperationEnd` is called and the Match operation ends.

4.14.2 Operation locks

Except for the Refresh operation, all the system operations of Net-Z are controlled by locks that when enabled prevent an operation from occurring. For example, when a role change occurs the RoleChange operation uses a lock while the operation is in progress and therefore, only one role change may occur at any particular time. If another request for a role change occurs while the lock is in use, then the request is queued until the lock is available upon completion of the operation.

Operation locks can be used by the developer to control duplicated object operations and are instigated via the `OperationLock` class. The two types of locks that may be used by this class are a `ConcurrentDenialOperationLock` or an `ExclusiveDenialOperationLock`. Operations of the classes `DOOperation` and `DupSpaceOperation` take as a parameter a concurrent denial operation lock whereas, operations of the class `DataSetOperation` take as a parameter an exclusive denial operation lock. Therefore, an operation of the class `DOOperation` or `DupSpaceOperation` may be simultaneously denied by several threads, whereas an operation of the class `DataSetOperation` may only be denied by one thread at a time. The system operations of Net-Z use six locks of the type `ConcurrentDenialOperationLock`, one for each of the duplicated object and duplication space operations, and one `ExclusiveDenialOperationLock` for each buffered dataset.

Operation locks may be used to ensure that particular events occur in a specific order, or that an operation or group of methods is executed atomically. Locks may be implemented for a particular duplicated object or a class of objects. The time that a class is locked should always be minimized because any network messages will be queued until the class is unlocked. For example, a typical use for a lock is to prevent the migration of an object while a method or group of methods is being called on the object. In Net-Z, there are two manners in which to implement locks: via the `OperationDenial` class or the `OperationLock` class. Upon construction and destruction, an instance of the `OperationDenial` class calls the method `OperationLock::BeginOperationDenial` and `OperationLock::EndOperationDenial` respectively. The instance of the class is automatically destroyed when a return value is returned or the block within which it is executed is exited. The second way to implement a lock is to call the methods `OperationLock::BeginOperationDenial` and `OperationLock::EndOperationDenial` directly. The advantage of using the first method is that inadvertent deadlocks will be avoided as the lock is automatically destroyed when a return value is returned.

An example of the implementation of an operation denial is given below. In this case we prevent a role change occurring while the results from a completed task are being transferred to the ResultOutput duplicated object.

```
void ResultOutput::ReportResult(unit uiResult) {
     OperationDenial oDenial(RoleChangeOperation::GetOperationLock());
     if (IsADuplica()) {
           ReportResults_OnMaster(Result);
     } else {
           // Process the results on the local station.
     }
}
```

4.15 Time Management

Net-Z provides three time measurements: the session clock, the local clock, and the chrono. The session clock is synchronized across all connected stations and is used to synchronize and compare events on different stations. The local clock and chrono are used to measure the time of events on a single station.

The objects `LocalClock` and `SessionClock` can be used to assign a value to a `Time` object. For example, to set the value of the variable *tNow* to the number of milliseconds elapsed since the commencement of the session the following is used:

```
Time tNow=SessionClock::GetTime();
```

4.15.1 Local Clock

The local clock is used to measure the elapsed time on the local station. The time provided by the local clock is set to zero milliseconds when the process commences on an individual station. The time is not adjusted nor synchronized across the connected stations. Typically the local clock is used to measure the time it takes to perform a set of operations on the local station, such as is performed in the `Chrono` and `Pacer` classes.

4.15.2 Session Clock

The session clock is used to synchronize or compare events on different stations and is the time used to time stamp messages sent across the network. It is automatically set to zero on the session master when the method `Session::CreateSession` is called. Only the session master may adjust the session clock, using the method `SessionClock::SetTime`, and each station connected to the session synchronizes its session clock to that of the session master. The accuracy of the synchronization is determined by the latency and jitter of the network. Latency is the time lag before a message reaches its destination, while jitter is the variation in latency.

Fine-tuning Session Clock Synchronization

The session clock on each station is synchronized to the session clock on the session master. When any station joins the process, the method `Session::JoinSession` automatically synchronizes the session clock between the session master and the joining station. The number of pulses used to determine the initial synchronization is set using the method `SessionClock::SetMinimumNbSyncPulse`. The higher this value the more accurate the initial value of the session clock, although it will increase the amount of bandwidth used and also the amount of time taken to join the session. After the session clock is initialized the local station will send a set number of pulses to the session master at a set time interval to prevent drift and maintain

synchronization of the session clock. The number of pulses and the time interval are set on each local station respectively by the methods `SetSyncPulseSampleSize` and `SetSyncPulseDelay`. The accuracy required from the session clock will determine the settings for these methods, although greater accuracy will come at the expense of greater bandwidth usage.

To calculate the session clock on a local station, the local station sends a message to the session master and the session master returns the value of the session clock on the session master, as illustrated in Figure 4.12. Due to latency, the local station receives the message from the session master, which contains the value for the session clock, a certain amount of time later than the value for the session clock. Therefore, this value cannot be directly used to update the value of the session clock on the local station. Hence, on the local station the value for the session clock received from the session master is adjusted according to the round trip time of the pulse using the following equation, where t_C and t_L are the session and local clock times respectively.

$$t_{C\ estimated,\ game\ master} = t_{C,\ game\ master} + \left(\frac{t_{L2,\ local\ station} - t_{L1,\ local\ station}}{2} \right)$$

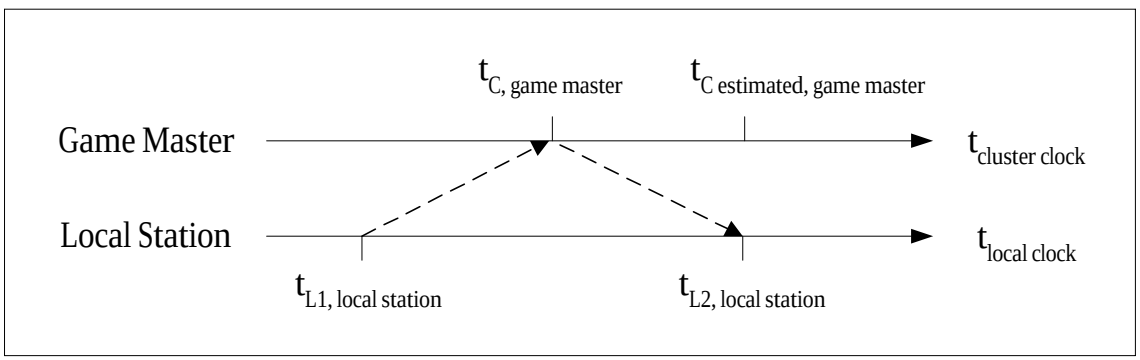


Figure 4.12: Calculation of the session clock on the local station.

The synchronization algorithm assumes that the latency between two stations is symmetrical, *i.e.* it takes the same amount of time for a message to be sent from a particular station to the session master as it does for the session master to send a message to the station. Therefore, the non-symmetry of latency between two stations will cause an error in the session clock on the local station.

Once the session clock time on the session master has been estimated, it is compared to the time of the local clock on the local station. The difference between these two values is called the offset. This offset is then added to the time given by the local clock to determine the new synchronized time for the session clock. In this example only one pulse was used to synchronize the session clock on the local

station. When more pulses are used in the calculation, the averages of the session clock time on the session master and the times on the local station are employed and a more accurate synchronization is obtained.

4.15.3 Chrono

The `Chrono` class measures time on the local station and functions just like a stopwatch. This class enables you to precisely measure the elapsed time, in milliseconds, between given events on the local station. Once an instance of the class `Chrono` is created, it operates in the same manner as a stopwatch using the methods `Start`, `Stop`, `Pause`, `Resume`, and `Reset`. The elapsed time can also be checked at any time using the method `Check`.

4.15.4 Pacer

The `Pacer` class provides an easy method to ensure that a particular event occurs no faster than a specified frequency. Therefore, if an event occurs within the specified frequency, the pacer will sleep for a certain length of time before the next event is allowed to proceed. For example, if the frequency is set to 2Hz the method `Pace` will sleep between the iterations of the paced event to ensure that consecutive events occur with a minimum time difference of 0.5 seconds. Hence, if the delay between two consecutive calls to the method `Pace` is 0.2 seconds, `Pace` will sleep for 0.3 seconds so that the next event occurs 0.5 seconds after the previous. The time between two consecutive calls to the method `Pace` is termed the processing time, which in this case is 0.2 seconds.

The maximum frequency, in Hertz, at which the pacer operates can be defined by the `SetFrequency` method:

```
Pacer::SetFrequency(double dFrequency)
```

or when the class is instantiated:

```
Pacer::Pacer(double dFrequency=0)
```

When the frequency is set to zero the call to the `Pace` function is not delayed *i.e.* the event is not paced.

The method `Pace` can be interrupted by a call to the method `Interrupt`. This call is used to exit from the `Pace` method while the pacer is sleeping. It is called from a separate thread to cause the thread that called the method `Pace` to exit from the method.

A call to the method `GetStats` returns the statistics associated with the class `Pacer`. The statistics are calculated since the creation of the pacer or since the previous call to `ResetStats` and include the total, minimum, and maximum processing times, the number of times the method `Pace` was called, and the total elapsed time.

4.16 Plug-ins

As we cannot predict the needs of all developers, Net-Z provides a plug-in interface that allows the developer to easily extend the capabilities of the product to suit his own needs. Net-Z plug-ins are typically dynamically loadable libraries (DLL) and are therefore, loaded at run time. As such, these plug-ins need to be shipped with the game. When a game built with Net-Z is executed, it will find and then load any Net-Z compatible plug-ins so that each plug-in may be accessed by the game. Plug-ins may also be linked statically to the game for integrity purposes.

The plug-in interface currently allows for three types of plug-ins: the transport protocol, the data compression algorithm, and the encryption algorithm. For each of these interfaces Net-Z provides plug-ins that may be invoked. For the transport protocol, a proprietary reliable UDP protocol plug-in is provided, which is used by default if no other transport plug-in is found. In addition, several compression and encryption plug-ins are available for the compression and encryption interfaces respectively. However, if these plug-ins do not entirely meet the needs of the game the developer may implement any other suitable plug-in.

Control over how Net-Z loads plug-ins is implemented via the `PluginManager` class. This class enables the load path for plug-ins to be changed, as well as controlling the available list of plug-ins. The default path from which plug-ins are loaded is; *c:Program Files\Proksim Software\NetZ\Plugins* however, it may be altered via the methods `ClearPluginsPath` and `LoadFromPath`. Net-Z maintains an activation list that contains all the plug-ins currently loaded that may be used. Specific plug-ins can be added to or removed from the list via the methods `PluginManager::AddToActivationList` and `PluginManager::RemoveFromActivationList`, and a given item may be retrieved via the method `PluginManager::GetActivationListItem`. Once a plug-in is loaded specific details about it may be retrieved via the `Plugin` class. Via this class you can obtain the plug-in type and its name, version, author, and Global Universal ID (GUID) number.

When a station joins a session via `Session::JoinSession`, the transport used is specified via the station URL. The URL may be either used as a parameter of the `JoinSession` method, or if one of the alternative parameters is stated, such as the machine name or port number, this is internally converted to a PRUDP URL. Once a station has joined a session, it will then use all the transports included in the activation list that it supports to communicate with the other stations in the session. Details about the transport protocols and its capabilities used by the game can be accessed via the `Transport` class. This class enables explicit transport specific options of the plug-in to be set via

the method `Transport::SetOption`. It is also via this class that the transport performance counters and emulation device are accessed in order to fine-tune various aspects of the transport protocol.

By default Net-Z enables both packet encryption and compression. Net-Z provides two encryption algorithms, RC4 and a proprietary Proksim Encryption Algorithm, to encrypt all information sent between stations: The default plug-in enabled is RC4, but another plug-in may be specified via the method `Protocol::SelectEncryptionPlugin`. The encryption key may be customized via the method `EncryptionAlgorithm::SetEncryptionKey`. For compression, Net-Z provides you with a choice of three well-known compression algorithms: LZSS, LZRW1, and Splay Tree. For details of these algorithms refer to any good book on data compression algorithms. The default plug-in enabled by Net-Z is LZSS, but another plug-in may be specified via the method `Protocol::SelectCompressionPlugin`. The LZSS algorithm was chosen as the default algorithm because it has a good compression ratio and is relatively fast regardless of the size of buffer to be compressed. However, for your game you may find that one of the other algorithms is more suitable. The LZRW1 algorithm is significantly faster than LZSS, but its compression ratio is lower. On the other hand, the compression ratio of Splay Tree is significantly better than either LZSS or LZRW1, but the rate of compression is approximately constant regardless of the buffer size. For example, for a buffer size of less than about 50 bytes, this algorithm is faster than the other two, but is significantly slower for larger buffers.

4.17 Performance Fine-tuning

Detailed in this section are the classes contained within Net-Z that aid in the fine tuning of a game. These classes enable you to monitor various aspects of the transport protocol and dataset updates. This aids in the fine tuning process and enables the simulation of various network conditions in order to see how your game performs under different conditions.

4.17.1 Performance Counters

To enable a game to be easily fine-tuned, Net-Z has various performance counters built into the system, which enable the user to monitor certain aspects of the transport protocol and dataset updates. The transport and dataset performance counters are commonly employed when using the emulation device to simulate various network conditions.

The counters available for the transport protocol measure the number of bits and packets sent and received. For each of these four counters the values of the maximum frequency, minimum frequency, average frequency, last frequency, and total count can be determined. However, it should be noted that some transport protocol plug-ins may not support all the transport performance counters available. The value to be queried is set using the method `TransportPerfCounters::GetValue`

and then the class is accessed using the method `Transport::GetPerfCounters`. This method returns a pointer to `TransportPerfCounters` for the given transport class. The syntax used is as follows:

```
TransportPerfCounters *Transport::GetPerfCounters()
```

For example, to monitor the bits received per second by a station the transport protocol performance counters could be implemented as follows:

```
Transport *pTransport=Protocol::GetInstance()->
FindTransport(PRUDP_TRANSPORT_TYPE);
TransportPerfCounters *pPerfCounter=pTansport->GetPerfCounters();
UlBPSRecv = pPerfCounter->GetValue(PERF_TRANSPORT_BPS_RECV);
```

The dataset performance counters monitor the frequency of dataset updates, which are made after a call to the method `DuplicatedObject::Update`. The counters available measure the frequency at which calls to update the dataset are made, the frequency of calls that triggered at least one update message, and the frequency at which update messages are sent. For each of the three counters the values of the maximum frequency, minimum frequency, average frequency, last frequency, and total count can be determined. The value to be queried is set using the method `DataSetPerfCounters::GetValue` and then the class is accessed using the method `DataSet::GetPerformanceCounters`. This method returns a pointer to `DataSetPerfCounters` for the given dataset class. The syntax used is as follows:

```
DataSetPerfCounters *DataSet::GetPerformanceCounters()
```

Therefore, if we wish to monitor the dataset Profile the following code would be used:

```
DataSetPerfCounters* pProfilePerfCounters =
Profile::GetPerformanceCounters()
```

4.17.2 Emulation Device

The emulation device enables the simulation of network conditions between stations. Via the `EmulationDevice` class the bandwidth, jitter, and latency, between stations can be set. The `EmulationDevice` class contains the subclasses `InputEmulationDevice`, which enables the router buffer size and the probability that a packet will be lost to be set on the input of a station, and `OutputEmulationDevice`. These classes enable you to test the performance of your game under various network conditions. They are referenced via the `Transport` class using the methods `GetInputEmulationDevice` and `GetOutputEmulationDevice`. It should be noted that some transport protocol plug-ins may not support all the functionality of the emulation device. To use the `EmulationDevice` class, the device must be enabled via the method `EmulationDevice::Enable`.

For example, to emulate a particular bandwidth on a station the following code could be used:

```
Transport *pTransport=Protocol::GetInstance()->
FindTransport(PRUDP_TRANSPORT_TYPE);
pTransport->GetOutputEmulationDevice()->Enable;
pTransport->GetOutputEmulationDevice()->SetBandwidth(ulBandwidth);
pTransport->GetInputEmulationDevice()->Enable;
pTransport->GetInputEmulationDevice()->SetBandwidth(ulBandwidth);
```

4.18 System Errors

The error-handling mechanism used in Net-Z is designed to report and handle errors and exceptional events that occur in the system. The functions in Net-Z that will return an error code generally fall into three categories: functions of the type Boolean, functions of the type pointer, and the `IsValid` methods. The `IsValid` methods are used to verify if a constructor that is directly called by the user and that could fail has successfully instantiated the relevant class. When an error occurs, the `IsValid` methods and the functions of the type Boolean will return false, or 0, and the functions of the type pointer will return a NULL pointer. The return values can be accessed using standard C++ programming methods.

When an error occurs while calling a method from any Net-Z interface, information is stored within the class `SystemError`. The method `GetLast` of this class returns the error code of the most recent Net-Z error that occurred. If the error occurs outside of Net-Z, such as a system error, then the method `GetLastExt` is used to return the system error code of the most recent error that occurred. The method `ClearLast` will remove the stored error and may be called before another method to ensure that the returned error is generated by the most recently called method. In addition, the method `GetErrorString` may be used to add a user-supplied description of the error. If you wish to know the error code and description as well as the file and line number where it occurred then use the method `TraceLast`, which is very useful when debugging a program. For example, the following illustrates one method of how to define the error associated with creating an object:

```
Avatar* pNewAvatar = Avatar::Create();
     if(pNewAvatar == NULL){
           Display(_T("ERROR: Could not create Avatar."));
           TCHAR strError[256]=_T("");
           SystemError::GetErrorString (SystemError::GetLast (),
strError, 256);
           Display(_T("%s!\n"), strError);
           return 0;
     }
```

Errors may be handled in several different ways, depending on the programmer's preference. An example of one manner in which to process errors is given below. This example is for processing any errors that occur when trying to join a session.

```
if (!(Session::GetInstanceHandle()->JoinSession(pStationURL))){
     switch(SystemError::GetLast){
           case SYSTEMERROR_DC_CLUSTER_JOIN_DENIED:
                 // Display join denied message and return
                 break;
           case SYSTEMERROR_DC_CLUSTER_JOIN_FAILED:
                 // Display join failed message and return
                 break;
           default:
                 // Display generic error message and return
     }
}
```

4.19 System Traces

The behaviour of the system can be traced via the `TraceOutput` class. Several trace macros are defined which allow you to trace various aspects of the system such as the object duplication protocol, duplication spaces, fault tolerance, initialization, load balancing, operations, plug-ins, and updates. There is also a `TRACE_USER` macro that you may redefine to implement your own traces and a `TRACE_ALL` flag that allows you to easily trace all of the available information. You can set and clear the various trace flags via the methods `TraceOutput::SetTraceFlag` and `TraceOutput::GetTraceFlag`. The `TRACE_OPERATION` flag is associated with the system operations. To set or clear the flag for tracing a specific operation the `SetTraceFlag` and `ClearTraceFlag` methods of each operation class are used. For example, to trace the fault behaviour of the system the following trace flag would be set:

```
TraceOutput::GetInstance->SetTraceFlag(TRACE_FAULT);
```

The output of the trace is controlled via the `TraceOutput` class. The output device to where the traces are printed can be specified as well as what information is contained in the trace. For example, information such as the thread ID, process ID, the local time, and the trace number can be included in the trace.

To trace the state of a duplicated object the method `DuplicatedObject::Trace` is used. This method may be redefined by the user or the default implementation may be used. The default trace information for a duplicated object consists of its class name, `DOHandle`, duplication role, its ref count, if it is flagged for destruction, as well as the station on which its duplication master instance resides. For example to trace the Room class the following could be used:

```
class Room:DOCLASS(Room) {
     void Trace(unsigned long ulTraceFlag) {
           // Trace base class state
           DOCLASS(Room)::Trace(ulTraceFlag);
           // Trace Room specific state
           TRACE(ulTraceFlag, _T("Room number is %d"),
GetRoomNumber());
     }
};
```

In addition, the contents of the local duplicated object store can be traced via the method `DuplicatedObjectStore::Trace`. This effectively calls the method `DuplicatedObject::Trace` on all the objects currently instantiated on the local station.

5 HOW TO BUILD A NET-Z GAME

5.1 Samples

This chapter contains descriptions of a series of sample programs that progressively demonstrate the capabilities of Net-Z in the context of a simple game. Once you have installed Net-Z, you will find the complete files and built applications for these samples in the *Samples* directory under the Net-Z directory. Detailed comments are written throughout the code for each sample to make it easy for you to modify the samples. In addition, a detailed description for the first sample program, *BasicSample*, is included in this chapter. The samples, along with a brief description, are listed below in order of increasing complexity.

BasicSample	This is a simple console application that shows how to use Net-Z duplicated objects to replicate player-controlled objects to other stations.
EnumerateLANGame	This sample illustrates how to enumerate all games found on the LAN. For each game found on the LAN, all the information from the session description is printed.
Inheritance	This sample illustrates how to easily create Net-Z duplicated objects that inherit from other duplicated objects.
ActionDungeonZ	This sample illustrates the implementation of actions and gives an introduction to the implementation of datasets.
DungeonZ	This sample further illustrates the implementation of datasets and demonstrates the application of various data types and dataset update qualifiers. It also introduces well-known and fault-tolerant objects.
Delegation	This sample illustrates how to delegate operations to enable an existing class hierarchy to be integrated into Net-Z.
Basic3DSample	This sample illustrates how to integrate Net-Z into an existing 3D graphics engine.
RoomZ	This sample illustrates how to implement load balancing and duplication spaces and cells in the context of a chat application.

SphereZ	This sample illustrates the full functionality of Net-Z including fault tolerance and load balancing and how to implement continuity breaks and local corrections.

Note: In each of the samples _t (*e*.g., _tprintf, _T, _getts, _ttoi, *etc*.) is used in the context of string handling to avoid difficulties due to the differences in character storage between UNICODE and ANSI. For more information on this refer to the section on string handling routines in any good C++ book.

How to Run a Sample

From the *Samples* subfolder of Net-Z read the text file *ReadMe.txt* that describes which features of Net-Z each sample illustrates and how to run the samples. Then run the pre-compiled version of *BasicSample*, which can be accessed from the Start menu or from the *ReleaseA* folder. Follow the instructions given in the *ReadMe* file and run the sample concurrently on several stations so that they participate in the same session. Repeat the above steps for each of the remaining samples contained in the *Samples* subfolder. Note that in order to run the samples that generate graphical output, such as *Basic3DSample* and *SphereZ*, DirectX 7.0a or greater is required to be installed on your computerand the 3DEngine, which is in the *3DEngine* folder, is required to be compiled separately.

In order to learn how Net-Z is used to implement a sample, study the C++ source code for *BasicSample* that is found in the *Samples* subfolder. Re-compile *BasicSample* to understand how to build a Net-Z game and so that you can modify it. In order to compile the program refer to the description of the development environment for each supported platform in the online help manual.

Joining a Session

Several computers are considered to be participating in the same session if, in order to communicate with each other, they are concurrently running the same Net-Z-based game.

Except for the *EnumerateLANGame, Inheritance,* and *Delegation* samples, which are single user applications, all the samples are started or joined in the same manner. The main method of the samples uses a small parser to analyse the inputs from the command line. A sample is started by typing any of the following, where *Sample* is the name of the sample you wish to run:

- Sample : Net-Z will search for and join a *Sample* session that is already running on the LAN. If none is found a new *Sample* session is created.
- Sample [IP] : Net-Z will attempt to join a *Sample* sessione at the given IP.
- Sample [MACHINE NAME] : Net-Z will attempt to join a *Sample* session on the station [MACHINE NAME].

- Sample -sessionname [NAME] : Net-Z will search for and join a *Sample* session called [NAME] already running on the LAN.
- Sample -create : Net-Z will create a *Sample* session at the default port number.
- Sample -create -sessionname [NAME] : Net-Z will create a *Sample* session at the default port number and give it the session name [NAME].

The IP address of the computer executing the sample is printed for the text based samples however, the samples that generate graphical output do not print the address. Several operating system commands are available to provide this information. To determine the name of your computer, execute the *hostname* command. To determine its IP address, use the *winipcfg* command for Windows 95/98/2000 or the *ipconfig* command for Windows NT/ME. Note that a station's IP address may change. For example, this is the case for a station that uses DHCP with dial-up Internet access. In this case, you must first establish the Internet connection and then give the command to determine the IP address for the current session.

The station that creates a session is initially the session master. If this station leaves the session, either by choice or via a fault, Net-Z automatically elects a new session master from the connected stations.

5.1.1 BasicSample

This is a very simple console application that creates objects and moves them in 3D space. It illustrates how to implement Net-Z datasets and duplicated objects to replicate an object to other stations. A single Avatar is created on each station and is controlled by that station. The position of the Avatar is continually updated on the duplication master and then rendered on the duplicas.

A *BasicSample* session is started in the manner detailed under Joining a Session in §5.1. You can start a *BasicSample* session by typing any of the following at the command prompt:

- BasicSample : Net-Z will search for and join a *BasicSample* session that is already running on the LAN. If none is found a new *BasicSample* session is created.
- BasicSample [IP] : Net-Z will attempt to join a *BasicSample* session at the given IP.
- BasicSample [MACHINE NAME] : Net-Z will attempt to join a *BasicSample* session on the station [MACHINE NAME].
- BasicSample -sessionname [NAME] : Net-Z will search for and join a *BasicSample* session called [NAME] already running on the LAN.
- BasicSample -create : Net-Z will create a *BasicSample* session at the default port number.
- BasicSample -create -sessionname [NAME] : Net-Z will create a *BasicSample* session at the default port number and give it the session name [NAME].

The *BasicSample* sample consists of six main files: *BasicSample.ddf* which defines the application title and the duplicated object classes; *BasicSample.ddl* which defines the datasets associated with the Avatar duplicated object class; *Position.h* and *Avatar.h* which define the dataset and duplicated object classes; *Avatar.cpp* which defines the methods relevant to the Avatar class; and *BasicSample.cpp* which defines the overall functionality of the sample. A parser is also used to analyse the input from the command line.

BasicSample.ddf

When using the Net-Z libraries, it is always necessary to define the application title and the duplicated object classes inside the DDF file. For *BasicSample*, these are defined as follows:

```
gametitle "BasicSample";
define Avatar;
```

BasicSample.ddl

The DDL file declares the dataset and duplicated object classes of the sample. The dataset Position is defined as consisting of three double variables, which are distributed as part of each Avatar object. The duplicated object class Avatar is defined and associates the dataset Position to the Avatar class.

```
dataset Position {
     double x;
     double y;
     double z;
};
doclass Avatar {
     Position m_dsPosition;
};
```

For each dataset and duplicated object class declared in the DDL, a header and a source file must be written, as detailed below.

Position.h

As this is the user implementation of the dataset Position, it must have the same name as the dataset, *i.e. Position.h*. This file defines the Position class as inheriting from the DDL compiler generated class DATASET(Position). The class includes two methods: Position, which initializes the position to (0, 0, 0) and Move, which updates the x, y, and z position values. This initial position is later used by the implementation given in *Avatar.cpp*.

```
class Position: public DATASET(Position) {
     public:
           Position() {
                 x=0;
                 y=0;
                 z=0;
           }
           void Move(double dx, double dy, double dz) {
                 x+=dx;
                 y+=dy;
                 z+=dz;
           }
};
```

Avatar.h

Avatar.h defines the Avatar class as an inheriting class from the DDL compiler generated DOCLASS(Avatar). The class defines the two iterated methods, Render and DoPhysic, as well as the private double speed variables in the x, y, and z directions. The speed variables are private and need not be a part of the dataset definition because in this example the speed is only required by the method DoPhysic. Since DoPhysic is only iterated over duplication masters, and then the changes are propagated via a call to the method `Update`, the duplica do not need to know their speed.

```
class Avatar: public DOCLASS(Avatar) {
     public:
           Avatar();
           ~Avatar() {};

           void Render();
           void DoPhysic();
     private:
           double m_dSpeedX, m_dSpeedY, m_dSpeedZ;
};
```

Avatar.cpp

Each Avatar has an x, y, and z position, and a speed in the x, y, and z direction. Only the position is part of the Avatar's dataset that is declared in the DDL file. The position is initialized to a random number, and the speed is initialized to 1:

```
Avatar::Avatar() {
     m_dsPosition.x=rand() % WORLD_LIMIT;
     m_dsPosition.y=rand() % WORLD_LIMIT;
     m_dsPosition.z=rand() % WORLD_LIMIT;

     m_dSpeedX = m_dSpeedY = m_dSpeedZ = 1;
}
```

Avatar.cpp shows us an example of the implementation of a user-defined method. DoPhysic calls the method Move found in *Position.h*, and then the method `Update` that automatically propagates the change from a duplication master to all of its duplica. Thus, the DoPhysic method modifies each instance of a duplication master on the station from which it was called, and then propagates the change to other stations with `Update`. If the method is called on a duplica an error will occur and therefore, we also display the error string.

```
Avatar::DoPhysic() {
     m_dsPosition.Move(m_dSpeedX, m_dSpeedY, m_dSpeedZ);
     if (!(Update(m_dsPosition))){
           TCHAR strError[256] = _T("");
           SystemError::GetErrorString(SystemError::GetLast(),
strError, 256);
           Display(_T("ERROR: %s!\n"), strError);
     }
}
```

Render prints the current position of each object, and since it is iterated over all duplicated objects it will print the location and DOHandle of all the distributed objects on the station where it is called.

```
Avatar::Render() {
     Display(_T("Avatar %X, at < %f,%f,%f > \n"), (unsigned
long)(GetHandle()), m_dsPosition.x, m_dsPosition.y, m_dsPosition.z);
}
```

BasicSample.cpp

BasicSample.cpp contains the main method of the sample and defines its overall functionality. Initially we include all the necessary header files, which consist of all of the header files for the user-defined classes.

```
#include <NetZ.h>
#include <time.h>
#include <stdio.h>
#include "Display.h"
#include "Position.h"
#include "Avatar.h"
#include "Parser.h"
```

The main function of *BasicSample* is described by the loop _tmain. Firstly, we create an instance of the class `Pacer`, which is part of the Net-Z library. The `Pacer` class is used to ensure that a given event occurs no faster than a specified frequency.

```
Pacer oPacer;
```

Then, we print the banner of the sample that briefly describes the main features of the sample.

Secondly, we create a new `NetZ` object and check that it is valid.

```
g_poNetZ = new NetZ();
if (!g_poNetZ->IsValid()) {
     Display(_T("ERROR: Could not initialize Net-Z."));
     Error();
     return 0;
}
```

Initialization:

Thirdly, we attempt to initialize the game using the method InitGame. Initialization is performed in the same manner for most of the samples. The initialization uses a small parser, contained in the files *Parser.h* and *Parser.cpp*, to analyze the arguments given at the command line, with which the sample is executed. The method InitSession will either join an existing session, or if none is found will create a new session.

We attempt to initialize the sessione using the method InitSession:

```
bool InitSession(int argc, TCHAR** argv){
```

To parse the command line arguments we use:

```
Parser oParser(argc, argv);
```

Then we attempt to initialize the session with the arguments given at the command line. When a player attempts to join a session, if no [IP], [MACHINE NAME], or [NAME] is given the InitSession method joins an existing session, or if none is found creates a new session. If an [IP], [MACHINE NAME], or [NAME] is given then Net-Z searches for an existing session with the given [IP], [MACHINE NAME], or [NAME]. If an existing session is found then the session is joined, otherwise an error is returned.

If a player states a session name when joining, we extract it with:

```
if(oParser.SessionNameSpecified())
     _tcscpy(g_szLANSessionName, oParser.GetSessionName());
```

Next, we try to find and then join the specified session via FindLANSession. If no [IP], [MACHINE NAME], or [NAME] was specified, a new session with a default name is created.

```
if(Session->FindLANSession(g_szLANSessionName, &oSessionDescription)){
     if (!(Session->JoinSession(&oSessionDescription))){
           return false;
     }
     bCreateSession = false;
}
else{
     bCreateSession = true;
}
```

If an [IP] is given then the session running at that address is joined with:

```
if (!(Session->JoinSession(oParser.GetIPAddress()))){
     return false;
}
```

As soon as the session is created, we create a single instance of the Avatar class, which is the duplication master on the local station, and then publish the Avatar's duplicas via the method `Publish`. If an error occurs during the creation of an Avatar, we return the error.

```
Avatar* pNewAvatar=Avatar::Create();
if (pNewAvatar==NULL){
     Display(_T("ERROR: Could not create Avatar."));
     Error();
     return 0;
}
SYSTEMCHECK(pNewAvatar->Publish());
```

Fourthly, we set the `Pacer` frequency and define the main loop, which is the iteration loop for the user methods. We set the `Pacer` frequency to 2 Hertz via the method `SetFrequency`. This ensures that the minimum time between the start of each iteration loop is 0.5 seconds. We then set the main loop to repeat, until the user quits by hitting Ctrl-C, and call the `Pace` method to pace the activity loop.

```
oPacer.SetFrequency(0.5);

while (true) {
     oPacer.Pace();
```

Inside the loop, we use selection iterators to execute the methods DoPhysic and Render over all duplication masters and all objects respectively on the station where the method is called.

```
Avatar::SelectionIterator itDMAvatar (DUPLICATION_MASTER);
itDMAvatar.GotoStart();
while(! (itDMAvatar.EndReached())){
     itDMAvatar->DoPhysic();
     itDMAvatar++;
}

Avatar::SelectionIterator itALLAvatar;
itALLAvatar.GotoStart();
while(! (itALLAvatar.EndReached())){
     itALLAvatar->Render();
     itALLAvatar++;
}
```

And the last thing to do is to free memory space. After exiting the main loop we destroy the `NetZ` object to free all duplicated objects.

```
delete g_poNetZ;
return 0;
```

5.1.2 EnumerateLANSessions

This sample can be used to display all the network sessions found on the local area network that are using Net-Z. *EnumerateLANSessions* prints all of the game information from the `SessionDescription`, *i.e.* application title, session name, the GUIDs of the compression and encryption plug-ins used, and the connecting station URL.

This application is designed for a single user and is started by simply running the executable file. The *EnumerateLANSessions* sample consists of two files: *EnumerateLANSessions.ddf*, which defines the application title and *EnumerateLANSessions.cpp*, which contains the main method of the sample and defines its overall functionality. The DDL compiler is not used in this sample.

In order to display the network sessions currently running on the LAN, we use a standard C++ template to create a list of session descriptions. We also supply the intermediate user method, AddToSessionList, which will be called by Net-Z each time a `SessionDescription` is found. This method is called using two parameters: *context* and *SessionDescription*. The method converts the parameter *ulContext* back to a SessionList, which is what we pass to the method when we call `EnumerateLANSessions`. We also copy the contents of each session description that is found to a new location, *pDescription*, and store a pointer to a `SessionDescription` at this new location in our SessionList. When we call the method `Session::EnumerateLANSessions`, this method prints the number of sessions found

and, by calling the intermediate user method AddToSessionList for every description it finds, fills the SessionList with the session descriptions that were found. No filtering on application titles is done and we probe on the default port number. The main loop cycles through the session list until it is empty, and uses the following methods of the `SessionDescription` class to print out the relevant information about each session that was found: `GetApplicationTitle`, `GetSessionName`, `GetStationURL`, `GetCompressionGUID`, and `GetEncryptionGUID`.

5.1.3 Inheritance

This sample illustrates how to easily create duplicated objects that inherit from other duplicated objects. In the sample, three duplicated object classes are defined: BasicShape, Triangle, and Square. The BasicShape class is a parent class from which Triangle and Square inherit.

This application is designed for a single user and is started by simply running the executable file. The *Inheritance* sample consists of nine main files: *Inheritance.ddf* which defines the application title and the duplicated object classes; *Inheritance.ddl* which defines the datasets associated with each duplicated object class and the inheritance of the duplicated object classes; *BasicShape.h*, *Triangle.h*, and *Square.h* which define the classes of the same name; *BasicShape.cpp*, *Triangle.cpp*, and *Square.cpp* which define the methods relevant to each respective class; and *Inheritance.cpp* which defines the overall functionality of the sample.

As usual, in the header files for each duplicated object class declared in the DDL we must include the relevant header files, declare the constructor and destructor, and define prototypes of the methods associated with the class. Within the source file for each class we implement the constructor and destructor of an instance of the class and define the methods pertinent to the class. The implementation of the methods simply prints a message informing the user of which function is being executed, along with the DOHandle of the object that the method is executed on.

We create the session with a call to the method `CreateSession` and then create and publish the Triangle and Square objects via the methods `Create` and `Publish`. We then execute the method Display by iterating over all duplication masters. The method Display is declared a virtual method and is declared in both the BasicShape and Triangle classes. Therefore, when the method Display is called the class Triangle executes its own Display method, whereas the class Square inherits the method from the BasicShape class. When the method Display is iterated over all duplication masters we see that in the case of the Triangle, its own Display function is called whereas the Square uses the method from the BasicShape class.

5.1.4 ActionDungeonZ

This sample demonstrates the use of actions to remotely call a method and is an introduction to datasets, where the use of some of the different types of data and dataset update qualifiers available in Net-Z are illustrated.

An ActionDungeonZ session is started in the manner detailed under Joining a Session in §5.1. Upon joining you are prompted to fill in your character's name, and then you can perform one of the following operations that are implemented as actions: display my profiles, display everybody's profiles, hit someone, give an item to someone, heal someone, or talk to someone.

The *ActionDungeonZ* sample consists of 11 main files: *ActionDungeonZ.ddf* which defines the application title and the duplicated object classes; *ActionDungeonZ.ddl* which defines the datasets and actions associated with the duplicated object classes; the header files *profile.h*, *Character.h*, *InventoryObject.h*, and *objectName.h*, which define the dataset and duplicated object classes; the source files of the same name, which initialize the variables and define the methods associated with each particular class; and *ActionDungeonZ.cpp* which defines the overall functionality of the sample. A parser is used to analyse the input from the command line.

The use of actions enables players to interact with each other. A player may fight, heal, trade items with, or talk to another player. Actions are declared in the DDL and then defined in the source and header files of the Character class. Actions are called with the help of a `Ref`, which is used to construct a duplicated object reference from a `DOHandle` to safely access the duplicated object.

In a fight, the action TakeAHit is called, with the hitter's `DOHandle`, on the character that is being attacked. If the attacked character is stronger than the initiator, the initiator calls TakeAHit on his attacker informing the attacker that he is being counter-attacked. At the end of the battle, the loser uses the Scream action to inform the winner as to how much damage he received during the fight and therefore, how much energy he lost. Item trading is performed in a similar way. The giver calls TakeItem on the receiver with information about himself, so that the receiver knows who to take the item from using the LoseItem action, and who to thank using the Say action.

5.1.5 DungeonZ

This simple console application is an extension of *ActionDungeonZ*. It illustrates how to define Net-Z dataset and duplicated object classes in order to replicate player-controlled objects to other stations. The sample demonstrates the different types of data that can make up a dataset, including strings, unsigned int, int, float, and an array of DOHandle, to hold a player's inventory of objects. Furthermore, we use an instance of the class World to demonstrate the use of a well-known object declaration in the DDF and a simple example of a fault-tolerant duplicated object.

A *DungeonZ* session is started in the manner detailed under Joining a Session in §5.1. Upon joining you are prompted to fill in the character's constant profile, and then you can perform one of the following operations: display world, display my profiles, display my inventory, and display everybody's profiles. The session master may also evolve the world.

The *DungeonZ* sample consists of 19 main files: *DungeonZ.ddf* which defines the application title and the duplicated object classes including a well-known duplicated object; *DungeonZ.ddl* which defines the datasets associated with the duplicated object classes; the header files *profile_constant.h*, *profile_inventory.h*, *profile.h*, *weather.h*, and *position.h*, which define the dataset classes and *Character.h*, *InventoryObject.h*, and *World.h*, which define the duplicated object classes; the source files of the same name, which initialize the variables and define the methods associated with each particular class; and *DungeonZ.cpp* which defines the overall functionality of the sample. A parser is used to analyse the input from the command line.

In this sample we introduce well-known duplicated objects. In the DDF, an instance of the duplicated object class World is declared as well-known and therefore, the global variable g_hTheWorld is defined. The global variable is of the type `WKHandle` and can be used to construct a `DOHandle` that refers to the World. The difference between a duplicated object class and a well-known duplicated object class is that the well-known class is created before the session is created and is guaranteed to be discovered by all stations when they join the session.

In the DDL file, all the datasets and duplicated object classes of the sample are defined. We define all the datasets and their relevant update policies that will be associated with the various duplicated object classes. The profile of a character is split into two datasets as the profile consists of constant data and variable data. The data that is constant is contained in the profile_constant dataset and is tagged with a constant update policy as this data is set at the beginning of a session and does not change thereafter. The variable data is defined in the dataset profile and is tagged with an `upon_request_filter` as this data may change from time to time. Similarly the datasets profile_inventory, weather, and position are defined and assigned the relevant update policies. Updates for the dataset position are sent over an unreliable channel as the dataset changes frequently and a missed update is not detrimental to the game. The duplicated object classes are then defined and the relevant datasets assigned to them. In the case of the class InventoryObject, each character holds a list of items that consists of an array of duplicated object handles pointing to instances of the duplicated object class InventoryObject. Also, it should be noted that when we wish to update the datasets tagged with an `upon_request_filter` we need to call the method `DataSet::RequestUpdate`.

Upon creation of a Character object, the user is asked to provide information about his character's profile. This data can only be assigned to the duplication master of the object and is stored in the dataset profile_constant. Before we initialize the profile of the character, we use the method `DuplicatedObject::IsADuplicationMaster` to determine if the object is a duplication master. Upon discovery of a duplicated object it is the user-defined Character constructor that

is called. Therefore, we separate the constructor from the initialization, via implementation of the method `DuplicatedObject::InitDO`, rather than doing all the initialization within the constructor. Upon initialization, the character is randomly given three objects, which can be displayed via the method DisplayProfiles. This method uses two intermediate methods. The first returns the `DOHandle` of the InventoryObject that is stored under the desired index. The second simply returns the number of items in the inventory. The method DisplayInventoryNames iterates through all of the indices of the inventory dohandle array and calls the method PrintItem on each of them. The method PrintItem is implemented as part of the InventoryObject class.

The methods EvolveWorld and DisplayWorld are defined as part of the class World. The method EvolveWorld updates the sun position and the weather depending on the time of day. The height of the sun increases until the *y* variable of the position dataset reaches 100, and then it becomes "After Noon" and the sun starts to set. If it is before noon the temperature and humidity increase, and if it is after noon the temperature and humidity decrease. After the sun has set, the night passes quickly, and a new day begins. Notice that since the SunPosition is implemented with the position dataset, which was tagged variable, by default, in the DDL, we need to call Update for the dataset to be updated. On the other hand, the dataset WorldWeather uses an `upon_request_filter` and thus the `RequestUpdate` call is needed. Only the session master can evolve the world. Therefore, before executing the method EvolveWorld we verify that the request to evolve the world comes from the session master. To do this we check that the Session object is a duplication master via the method `DuplicatedObject::IsADuplicationMaster`. If this method returns true, then the methods EvolveWorld and then DisplayWorld are executed. If the request is made by a station that is not the session master then this station is told that it cannot evolve the world.

The game is initialized in a similar manner as *BasicSample*. The only difference is that we must create all well-known objects before we create a session. Therefore, the well-known duplicated object World is created before the session is created. Refer to the description of Initialization, contained in the detailed description of the *BasicSample.cpp* file, in §5.1.1 for further details. The method CreateWorld creates the well-known duplicated object class World via the method `DuplicatedObject::CreateWellKnown`. When this method is executed, an object identified by the global variable g_hTheWorld of the type `WKHandle` is automatically created and discovered by all stations when they join the session.

As soon as the game space is created, we create a single instance of the Character class that is the duplication master on the local station. The duplicated object is created via the method `DuplicatedObject::Create` and we use the method `DuplicatedObject::GetHandle` to obtain a `DOHandle` to the character object. We use this handle to execute the DisplayProfiles and DisplayInventory user methods on the character. We also randomly create three items from the InventoryObject duplicated object class and give them to the character using the method TakeItem. As we have not as yet published the character object, we can

use the `DuplicatedObject` directly however, after the character has been published, we must use the `DOHandle` or a `Ref` to the object. Finally, we publish the character's duplicas via the method `DuplicatedObject::Publish`.

5.1.6 Delegation

This is a simple example of how to delegate specific functions to classes outside of Net-Z. In this sample the class NetSphere delegates the physics and rendering to an external class hierarchy. The methods of the NetSphere class SetPosition, Render, and DoPhysic delegate the rendering and the physics to the Sphere class. Therefore, the actual rendering and physics is performed by the methods SetPosition, Draw, and SDOPhysic of the Sphere class.

This application is designed for a single user and is started by simply running the executable file. The *Delegation* sample consists of five main Net-Z files: *Delegation.ddf* which defines the application title and the duplicated object classes; *Delegation.ddl* which defines the datasets associated with each duplicated object class; *NetSphere.h* and *Position.h* which define the classes of the same name; *NetSphere.cpp* which defines the methods relevant to the class; and *Delegation.cpp* which defines the overall functionality of the sample. The sample also consists of the files relating to the RenderableObject and Sphere classes that represent a "graphics" engine that is external to Net-Z.

The NetSphere class has a pointer to an instance of the Sphere class and passes information from the Position dataset to the Sphere class via the method SetPosition. This method invokes the method SetPosition of the Sphere class. Similarly, the methods Render and DoPhysic of the NetSphere class invoke the methods Draw and SDOPhysic respectively of the Sphere class. However, in the case of the method DoPhysic, if the object that the method is called on is a duplication master then information is passed to and from, rather than just to, the Sphere class.

The sample is initialized in the same manner as in *BasicSample*. For more details, refer to the description of Initialization, contained in the detailed description of the *BasicSample.cpp* file, in §5.1.1. The main loop of the sample, which is contained in the file *Delegation.cpp*, is paced at 2Hz via the method `Pacer::Pace`. Within the main loop the NetSphere objects are created and published and then the methods DoPhysic and Render are iterated over all duplicated objects. To exit the loop hit Ctrl-C.

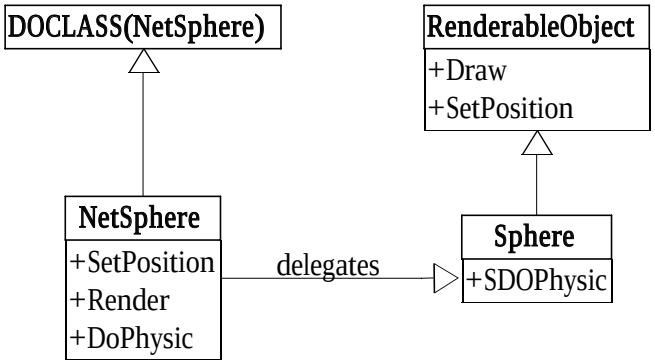


Figure 5.1: Class hierarchy of the Delegation sample program.

5.1.7 Basic3DSample

This sample illustrates how to integrate Net-Z into an existing graphics engine, Direct X, as well as the use of extrapolation filters on datasets. The sample is designed to be run on multiple stations or multiple instances on the same station, although in the latter case the processing power of your station will determine the number of instances that can be run simultaneously on the one station. A Basic3DSample session is started in the manner as detailed under Joining a Session in §5.1. When running this sample, the resource directory (.\res) needs to be at the same level as the executable. The Developer Studio project that comes with the distribution already does this. However, if you run *Basic.exe* from the command prompt, be sure to start it from: *Samples\Basic3DSample*. To compile and run this sample, DirectX 7.0a or later must be installed and configured on your computer and the 3DEngine, which is in the *3DEngine* folder, must also be compiled. For your convenience DirectX is included in the resource directory of the Net-Z CD.

In the sample, each station that joins the session creates a sphere that it is the duplication master of and duplicas of this sphere are created on each of the other connected stations. Each sphere is an instance of the class Sphere that inherits from the class GeometricalObject. In the DDL we state an extrapolation filter is used to extrapolate the datasets Position and SphereData and that the data is sent over an unreliable channel. In the file *Basic.cpp*, for both datasets we then define a constant extrapolation error and set it to the default threshold via the method `ErrorToleranceFunction::SetConstantError`. When a sphere is created it follows the trajectory that is defined in *MathematicalMotion.h*.

In this sample the spheres are not fault-tolerant as both the system callbacks `ApproveFaultRecovery` and `ApproveEmigration` return false. Therefore, when a station quits the session or leaves due to the occurrence of a fault, the sphere that was created by that station and whose duplication master resides on that station is deleted. However, as in any Net-Z game the session master is fault-tolerant. Therefore, if the station that is the session master quits the session

or leaves due to the occurrence of a fault then Net-Z automatically elects a new session master and the session continues although, the sphere that was controlled on the station that was the session master is deleted.

The rendering of the game objects is delegated outside of Net-Z to DirectX. The Net-Z class Sphere has a pointer to an instance of the D3DSphere class of DirectX and passes information from the Position and SphereData datasets to the D3DSphere class via the method Set3DInfo. This method invokes the methods SetPosition and SetZRotation of the D3DSphere class.

The sample is initialized in a similar manner as in *BasicSample*. For more details, refer to the description of Initialization, contained in the detailed description of the *BasicSample.cpp* file, in §5.1.1. The main loop of the sample, which is contained in the file *Basic.cpp*, is paced at 30Hz via the method `Pacer::Pace`. Within the main loop the spheres are created and then published and the methods Move, RefreshDataset, and Set3DInfo are iterated over all duplication masters, duplicas, and all duplicated objects respectively. To customize the termination of the game, the virtual method Terminate of the class SampleCtrlIF is redefined within the class TerminationInterface.

5.1.8 RoomZ

This sample illustrates how to create a scalable chat application via the use of duplication spaces and load balancing. A *RoomZ* session is started in the manner detailed under Joining a Session in §5.1. Upon joining you are prompted to fill in your name whereby initially you find yourself in the lobby. If you are the station that created the chat you are also given the option of becoming the moderator for the chat session. You may then list all the chat rooms available, delegate profiles *i.e*. other users' names and location, your profile, and you may also create a new chat room if desired. You then choose a room you wish to enter from where you can display other delegates' profiles, send messages to all or just one delegate, change rooms, create a new room, or obtain a quote from a Sage.

The *RoomZ* sample consists of 21 main files*: RoomZ.ddf* which defines the application title and the duplicated object classes including a well-known duplicated object; *RoomZ.ddl* which defines the datasets associated with the duplicated object classes and the role that each duplicated object class plays in the duplication space; the header files *Location.h, SageKnowledge.h,* and *StringInfo.h* which define the dataset classes, *Delegate.h, Moderator.h, Room.h, Sage.h,* and *SageBalancing.h* which define the duplicated object classes, and *RoomSpace.h* which defines the duplication space; the source files of the same name, which initialize the variables and define the methods associated with each particular class; and *RoomZ.cpp* which defines the overall functionality of the sample. A parser is used to analyse the input from the command line.

When a delegate joins the *RoomZ* chat, they sign-in and enter the virtual space of rooms and other delegates. When the session is created, a Lobby room is created as a well-known object and it is here that the delegates find themselves upon joining. From the lobby a delegate may choose a room to enter

or create a new room if desired. Once in a chat room, the delegates can transfer from one room to another, create a new room, or send text messages to all or just one specific delegate that is in the same room. The messages are sent via the PrintMsg action. The RMC SetCurrentRoom is used to set the current room that a Delegate object is in.

The Delegate class is the duplicated object class that encapsulates information about the user, including their name and location. Location information is kept in a dataset in the form of a `DOHandle` to the room in which the delegate is presently in. Since users both discover and are discovered by other users, in the DDL the Delegate duplicated object class is declared to be a publisher as well as a subscriber in the RoomSpace. The Moderator class inherits from the Delegate class and therefore, inherits all the functionality of the Delegate class including the publisher and subscriber declarations. The moderator controls the security of the chat and acts as a watchdog to ensure that the code of conduct of the chat is not broken. As such the moderator can listen to and talk to all participants in the chat regardless of the room they are in.

The Room duplicated object class inherits from the `GlobalDO` class and therefore, all Room objects are duplicated to all stations participating in the session. The Room class consists of a description of the room and the class is declared a cell in the RoomSpace. The creation of cells is an efficient mechanism for distributing the matching computations that are required to maintain an accurate view of the session by all of the member stations of the duplication space. The use of a duplication space with cells requires the implementation of the `DuplicationSpace::PSMatch` and `DuplicationSpace::CellMatch` virtual methods. The `PSMatch` method is implemented so that it returns true if a pair of delegates are in the same room or if a moderator/delegate pair exists. Therefore, Delegates will only discover each other if they are in the same room, but Moderator objects will discover all Delegate objects in the session regardless of the particular room that they are in. Thus, when a Delegate enters a particular room it will discover, and thus instantiate duplicas of, only the other delegates that are in that room. Similarly, when a Delegate leaves a room the duplicas of the other delegates in that room will be destroyed on the Delegate's station, as they are no longer relevant to the Delegate. In this way duplication spaces enable an application to be scaled as a station will dynamically create and destroy duplicas of the objects it requires depending on the specifications given by the `PSMatch` function. The `CellMatch` method is implemented to define when a Delegate instance belongs to a cell. Therefore, as each Room is declared a cell the method will return true when a particular delegate is in a particular room. The Room duplicated objects are fault tolerant and therefore, will automatically migrate to another station if the station where their duplication master resides leaves the session.

The session contains a number of Sage objects that a Delegate can request a quote from. Sage objects are associated to a Room and a Delegate can only request a quote from a Sage if they are in the same room. Sage objects reside on the station where the duplication master of the Room that they are associated to resides. Sage objects are automatically load balanced across the application according to the load balancing policy defined in the SageBalancing class and can therefore, change rooms while the

application is running. The SageBalancing class inherits from the Net-Z LoadBalancingDomain class and redefines certain methods of this class. The Sages are load balanced according to the number of Sages in the duplication master Rooms on the local station. The load balancing policy will attempt to evenly distribute the number of Sages between the stations connected to the sessions by moving the Sages from the duplication master of one room to another.

RoomZ DDL File

```
dupspace RoomSpace;

dataset StringInfo {
     string Info[64];
};

dataset Location {
     dohandle room;
};

doclass Delegate {
     Location m_dsLocation;
     StringInfo m_dsName;
     subscriber in RoomSpace;
     publisher in RoomSpace;
     action      PrintMsg(dohandle hSpeaker, string strMessage[128],
bool bWhisper);
     void SetCurrentRoom(in dohandle hRoom);
};

dataset SageKnowledge {
     string m_sQuote[512];
};

doclass Sage : Delegate {
     SageKnowledge m_dsKnowledge;
};

doclass Moderator : Delegate {
};

doclass Room : GlobalDO {
     cell in RoomSpace;
     StringInfo m_dsTopic;
};

doclass SageBalancing : LoadBalancingDomain {
};
```

5.1.9 SphereZ

This sample is a visual demonstration and means of experimenting with the fault tolerance and load balancing capabilities of Net-Z. In a multiplayer session, when a player is disconnected the control of the fault-tolerant objects on that player's station is automatically transferred to another station. In this sample the spheres controlled by each station are a different colour. Thus, when a sphere's colour changes this indicates that a different station has assumed control of it. The spheres whose duplication masters reside on the station that is the session master are always coloured white. Therefore, if the sesssion master leaves the session, a new session master will be elected and the spheres controlled by that station will change from their current colour to white. By disconnecting and reconnecting players, you can observe how the spheres change colour to indicate seamless transfer of control to other stations.

AI-controlled spheres are load-balanced and fault-tolerant. Therefore, the control of these spheres is automatically distributed among all the stations participating in the session. When control of a sphere is transferred to another station, the sphere's colour changes to the colour of the new controlling station. By observing how the spheres change colour as new players join and leave the session, you will observe how the control of AI-controlled spheres is automatically redistributed evenly among all the participating stations. You can switch the camera focus to that of an AI-controlled sphere but you cannot control the sphere's trajectory.

SphereZ also illustrates the implementation of distance-based dead reckoning and the quality factor to optimize bandwidth usage when sending updates for the Position dataset of each sphere. We also implement continuity breaks and use an update model to accurately show the position of the sphere, on both the duplication master and duplica, after the occurrence of a collision. In this sample we allow for collisions with the walls and floor, but for simplicity we do not account for spheres colliding with each other.

A *SphereZ* session is started in the manner detailed under Joining a Session in §5.1. When running this sample, the resource directory (.\res) needs to be at the same level as the executable. The Developer Studio project that comes with the distribution already does this. However, if you run *SphereZ.exe* from the command prompt, be sure to start it from: *Samples\SphereZ*. To compile and run this sample, DirectX 7.0a or later must be installed and configured on your computer and the 3DEngine, which is in the *3DEngine* folder, must also be compiled. For your convenience DirectX is included in the resource directory of the Net-Z CD. The keyboard commands for the sample are given in the table below.

Command	Control
Left arrow	Turn left
Right arrow	Turn right
Up arrow	Accelerate
Down arrow	Decelerate
1	Camera mode: Follow sphere
2	Camera mode: Dive
3	Camera mode: Bird's-eye view
C	Toggle camera mode
S	Create player-controlled sphere
space bar	Switch camera focus to another sphere
W	Toggle the fog on or off. Only the game master can do this.
A	Create an AI-controlled sphere
ESC	Quit

Table 5.1: Keyboard commands for the SphereZ sample.

The *SphereZ* sample consists of 19 main files: *Demo.ddf* which defines the application title and the duplicated object classes; *Demo.ddl* which defines the datasets and actions associated with each duplicated object class; the header files *Position.h, SphereData.h,* and *Weather.h* which define the dataset classes and *GeometricalObject.h, Sphere.h, AISphere.h,* and *World.h* which define the duplicated object classes; the source files of the same name which initialize the variables and define the methods associated with each particular class; the header and source files for the SphereZGameCtrl that is a custom control interface that inherits from the `GameCtrlInterface`, and *SphereZ.cpp* which defines the main loop of the sample and its overall functionality. A parser is also used to analyse the input from the command line.

The dataset and duplicated object classes of the sample are declared in the DDL. The dataset classes are tagged with the appropriate update policy and associated to the relevant duplicated object class. As we wish to extrapolate the position of objects we tag the dataset Position with the update policy `extrapolation_filter`, `unreliable`, and we use an `upon_request_filter` for the dataset SphereData because the only time we wish to change the appearance of a sphere is when it migrates.

The file *Sphere.cpp* implements the constructor and destructor of an instance of the Sphere class, initializes the variables, and defines the methods associated with the class. Initially we include all necessary files and then initialize the relevant variables for when a sphere is created. We set the initial position of a sphere to the to centre of the game area and also start the Chrono and RenderChrono clocks.

The times measured by these two objects are used locally by each station to calculate the velocity and rotation of the spheres respectively. We then define the texture of the spheres on the joining station via the method InitDO.

The two methods RefreshDatasets and DoPhysic are also defined in the file *Sphere.cpp* and are iterated over all duplicas and over all duplication masters respectively. For the method RefreshDatasets, we use the method PredictRateOfChange to calculate the speed of the object in the x, y, and z directions. This method uses the dataset extrapolation model, calculated for the object on the local station, to calculate the local speed of the object. This information is then used to calculate the rotation of the sphere. The method DoPhysic calculates the acceleration, speed, and direction of the object. Also, if the object is bouncing the speed is adjusted in the *z* direction to take into account the effect of gravity. We use the Chrono to determine the time elapsed since the last calculation and then reset the Chrono to zero for the next calculation. The elapsed time is then used to calculate the speed and direction of the sphere.

We then determine if the object collides with the floor or the wall and indicate a continuity break when the collision occurs. When a collision occurs, a continuity break of the type sudden change is indicated. In the case of a wall collision we increase the speed of the sphere in the *z* direction and initiate bouncing of the sphere. After a continuity break occurs, we use a continuity break update model to update the dataset Position on the duplication master. Although it is not essential to use an update model after the occurrence of a continuity break, it enables the position of the duplication master to be shown more accurately immediately after a continuity break occurs. In this case we set the position, speed, and acceleration in the *x*, *y*, and *z* directions using the model of the dataset that Net-Z calculates for the duplication master based on the history of the dataset values. We then compute the new position and speed of the camera and call the method Update, which we must call regularly to maintain a coherent prediction model for the extrapolated datasets.

As we are using distance-based dead reckoning, before we can calculate the error tolerance, we must first calculate the distance between the observer on the remote station and the duplication master of our object on the local station. The distance is calculated as the difference between the positions of the two objects, as defined by the method Distance that is implemented as part of the class Position. When a player creates or joins a game we automatically register the sphere that they create as the observer for that station. When we change the focus of the sphere to another sphere via the method DoChangeFocus, we also change the observer to the new sphere that the camera focuses on. When we wish to change the focus of the camera to another sphere, we iterate over the duplication masters on our station till we find another master. Each duplication master on a station is numbered in the order that it was discovered on the station and we simply change the focus to the next master. When the next master is found we unregister the previous observer and register the new observer as the sphere the camera now focuses on. We then check if the look of the sphere has changed and reset the colour if necessary. The error tolerance, as given by the equation below, is determined from the parameters

defined by the method `SetQuadraticError` and the quality factor. We also set the convergence delay that gives the maximum and minimum time, in milliseconds, which the variables of the dataset will take to converge to the path predicted by the extrapolation algorithm.

$$Error\ Tolerance = 0.05 + \left(\frac{1-0.05}{30^2}\right)\left(\frac{z}{q}\right)^2$$

We then define the methods DoPositioning and DoControl that are iterated over all objects and all duplication masters respectively. In the case of the method DoPositioning, if the object is a duplica we set local corrections for floor and wall collisions. First, using the local extrapolation model for the dataset of the duplica, we predict the rate of change of the dataset variables and use it to update, on the local station, the position, speed, and acceleration of the object in the required direction. For each of the collisions we set a local correction only for those variables that change, *i.e.* when an object hits the floor the values of the x and y variables do not change and therefore, we only set a correction in the z direction. We use the render chrono within the method DoPositioning to calculate the time elapsed since the method was last called. The elapsed time is then used to calculate the rotation of the sphere. For the method DoControl, if a sphere has the camera focused on it and the sphere is not AI-controlled we accept user input to turn or to change the speed of the sphere. The methods TurnRate and DesiredSpeed are part of the Motion class and are detailed in the source and header files of this class.

We instruct Net-Z to automatically perform load balancing every 10 seconds. When we perform load balancing we determine if a sphere may migrate to another station via the system callback `ApproveEmigration`. If the camera focus is on the sphere, then the method returns false and the sphere can not migrate. If the camera focus is not on the sphere and it is an AI-controlled sphere then the method returns true and the sphere can migrate to another station. When a sphere migrates to another station we change its colour and texture to that of the new station where its duplication master now resides and give the sphere an initial speed and direction, which is the same as the last value calculated before migration.

The game is then initialized in a similar manner as in *BasicSample* except that we also set the parameters of the error tolerance function, the convergence delay, and the quality factor that are used in the dead reckoning calculations for the dataset Position. Refer to the description of Initialization, contained in the detailed description of the *BasicSample.cpp* file, in §5.1.1. We then create the game world via the method CreateWorld. When we join or create a session the colour of the spheres on our station is chosen automatically, our sphere is created, the camera focused on it, it is registered as the observer and then published. After initalisation of the session we call the main loop, which is paced to ensure that a complete loop occurs no faster than at 50 frames per second. Within the loop we iterate the methods of the sample over the appropriate duplicated objects.

5.2 Game Scalability

For a game to be scaled without decreasing the quality of the user experience the resource usage of the system needs to be optimized. As more stations join a session the use of resources such as CPU and memory on individual stations and bandwidth and latency between stations needs to be used efficiently so that the session quality perceived by the participants does not decrease. To enable a game to be scaled, the Net-Z API includes several features that can be used to make the most effective use of the available system resources. The object duplication model used by Net-Z gives the developer control over where and how many duplicas of an object are published so that objects are only duplicated to the stations where they are needed. To reduce the bottleneck effect of sending updates to a multitude of stations, Net-Z uses hierarchical updates to forward update messages from a duplication master to its duplicas so that the station where a duplication master resides is not required to send a message to each individual station where a duplica resides. The ability to load balance virtually any quantity, including objects, allows the developer to distribute the resource usage across the game and to optimize performance. These features allow a game to be scaled, but naturally the extent to which a game can be scaled is dependent on the particular game.

The scalability of a game depends on the type of game and on the level of interactivity required between the stations connected to the session. Obviously, as is the case for any networked game, the more information that is required to be sent between stations the lower the number of stations that can be supported simultaneously. The amount of information that is required to be received by a station depends on factors such as how many objects there are, how often their datasets need to be updated, and the quality of the output required on the station. In Net-Z, there are some inherent limitations with respect to the number of duplicated objects and classes that can be created, although in reality these should not prove restrictive. The maximum number of duplicated object classes that can be created in the one game is 2^{10}, or 1024 classes, and the maximum number of duplicated object instances that can be created is 2^{22}, or over 4 million objects. Regardless of how a game is implemented the physical limitations of the hardware, such as the available memory, CPU, and bandwidth, will always be present. Depending on the game, the limitations of the hardware and software may or may not present performance problems, however with the flexibility of Net-Z there are several manners in which to minimise the effect of such restrictions.

Bandwidth use is always a concern over any network but before we can attempt to optimize the bandwidth used by a station, we need to understand the factors that affect it. Depending on the hardware used, a station will have a limitation as to the input and output bandwidth available. The input bandwidth used by a station will depend on the number of duplicas on the station, the size of the message, and the frequency at which the datasets of the object are updated.

$$\textit{input bandwidth} = \textit{nb duplicas} \times \textit{message size} \times \textit{nb messages per second}$$

The message size is equal to the payload plus an additional 88 bytes of overhead that contains information such as the object's ID and location. Compression may be used to decrease the size of a message, which would typically result in a compressed message size of approximately 75 bytes. Assuming we have a 33K connection, this equates to 55 messages per second. Therefore, if we have a very interactive game where each object needs to be updated in the order of 3-4 times per second, we could support approximately 15 duplicas on the station. For a less interactive game, a station could support a significantly greater number of duplicas. To reduce the input bandwidth usage the developer can reduce the message size via compression and should ensure that a station only has copies of the duplicas that it really needs. The object duplication model used by Net-Z enables the developer to control the duplication of objects so that a station does not have duplicas of objects that it does not require.

The output bandwidth used by a station is dependent on the number of duplication masters on the station, the number of stations that it sends updates to directly, as well as the size and frequency of the message.

output bandwidth = *nb masters* **x** *nb direct messages* **x** *message size* **x** *nb messages per second*

The number of messages that a station sends directly to another station will depend on the implementation of hierarchical messaging. If hierarchical messaging is enabled, the developer can configure how many stations will be directly updated, whereas if it is not implemented, the number of messages sent will be equal to the number of duplicas of the object. Hierarchical messaging is one method that the developer can use to decrease the output bandwidth used on a particular station. The developer can dictate which stations use hierarchical messaging and which do not. Therefore, if a particular station has enough bandwidth to update all the duplicas of its objects then it will, whereas another station that does not have sufficient bandwidth can use hierarchical messaging. When hierarchical messaging is implemented, the developer must ensure that the station where the root multicast group duplication master resides has enough bandwidth available to receive messages from all the duplicated objects that are using hierarchical messaging and to send the message to the number of stations in the group. Load balancing may also be implemented to distribute duplication masters across the network to optimize the available resources. Load balancing can be used to decrease the number of duplication masters on a station that has a low capacity and to increase the number on a station that has a higher capacity to ensure that each station only controls the number of duplication masters for which it has sufficient bandwidth.

One method that the developer may use to control where an object is duplicated to is via the implementation of duplication spaces. However, the trade-off is that one or more stations are required to perform the match computations. The number of computations required to be performed is directly related to the number of publishers and subscribers defined in the session and the complexity of the matching functions. In general, the additional computations required by the implementation of duplication spaces should not impact adversely on the overall performance of the station that performs these matches. If the number of computations are particularly numerous or computationally intensive, which may arise when the number of users is very high, then the computations can be distrib-

uted amongst the stations connected to the session via the use of cells. Cells are an effective way in which to distribute the computational load of the session across the connected stations, as they may be load balanced and created and destroyed upon demand while a session is in progress.

Certain processes of Net-Z will use a certain amount of computing resources of the network, which may affect the scalability of some games. Each duplicated object will use a specific amount of memory on the station where it resides that is largely dependent on the size of the user-defined datasets for the duplicated object. Obviously the larger the datasets of a duplicated object, the more memory the object will use in addition to an overhead that contains information such as the duplicated object's ID and location. For a duplica of a duplicated object the overhead will be 32 bytes whereas, a duplication master will require an additional 20 bytes overhead per duplica in order to know the location of each of its duplicas. Certain operations performed while a session is running will use significant amounts of CPU. The main computationally intensive operations are performing the `PSMatch` and `CellMatch` functions and the compression of messages. The amount of CPU used for the matching functions will depend on the number of publishers, subscribers, and cells and if data compression is invoked, the type of compression algorithm used. The bandwidth that a station requires depends principally on the number of messages to be sent or received. The number of messages is in turn dependent on the number of duplicas of an object, as a separate message is required to be sent to each duplica, and also on the frequency at which the datasets of the object are updated. The bandwidth required to receive messages depends on the number of object duplicas that the station receives updates for, and the bandwidth required to send messages depends on the number of duplicas of the duplication master we need to send updates to. If a station has limited bandwidth it is possible to decrease the frequency of updates, although this may decrease the quality of the game that the user experiences. Alternatively, we can route update messages sent from the duplication master objects on the station via a server, which then forwards the update to the other stations in the session. The size of each message depends on the size of the datasets that need to be sent, in addition to the UDP header and a header of 20 bytes that are part of every message. These two headers respectively encompass the transport protocol information and information such as the packet ID and destination. To decrease the size of a message sent across the network the message and header may be compressed, although this will increase the CPU usage.

5.3 Network Configuration Options

The flexible nature of the Net-Z architecture allows it to be run in a true peer-to-peer mode, client-server mode, or a combination of both. In addition, the the API allows the developer to implement restrictions on where duplicated objects are created, where they reside, who may discover them, and who may modify them. This flexibility presents the developer with various options on how to structure the network depending on the needs of the game and can be used to configure a game in various

ways in order to, amongst other things, thwart hackers. The architecture chosen by the developer will be a trade-off between the performance advantages of a peer-to-peer architecture and the additional security measures afforded by a server-like architecture.

In a Net-Z game, as the developer has full control over where the duplication master of each object resides and how data is propagated over the network, it is easy to restrict user access to specific data. For example, certain data can be stored on a secure server to limit its access, without necessarily going to a true client-server architecture. The developer has the option of locating all or some duplicated objects on the secure server. Thereby, sensitive data could be located on a server and non-sensitive data could be located on each user's station. Therefore, by using a hybrid approach a developer can effectively minimise the effects of hackers while still maintaining many of the benefits of a peer-to-peer architecture, such as faster communication, load balancing, and fault tolerance.

To convert the Net-Z architecture from peer-to-peer to server-like is simply a matter of changing the station where the duplication master of each object resides. For a peer-to-peer based architecture, the duplication master of an object that is user-controlled will reside on that user's station. To operate in a server-like mode, the developer can choose to locate all duplication masters on one secure station, which effectively restricts the amount of information that a user can access. When this is the case, a participant would invoke commands to the duplication master of his object by using RMCs or actions. Running Net-Z on a server-like architecture would effectively reduce the opportunities for hackers, but it would also eliminate the peer-to-peer based advantages of Net-Z, such as load balancing, fault tolerance, and latency reduction. However, for certain games it may be beneficial to operate in a pure client-server configuration. When the server actually consists of a group of servers Net-Z provides all the necessary infrastructure to implement load balancing and fault tolerance across the server group. This ensures that the information on the server is still retained even if one server fails, and enables the workload to be distributed across the group of servers via load balancing.

6 TROUBLESHOOTING

6.1 DDL Compiler Errors

The possible DDL compiler errors and warnings and their cause are listed below.

Code	Message	Reason
Error C1	Undeclared base class <identifier>	The class named <identifier> is not declared in the DDL
Error C2	Well-known DOClass <identifier> already used	Two well-known classes cannot have the same name.
Error C3	<identifier> ID already taken	An ID cannot be used more than once.
Error C4	Array size should be greater than 1	The declared size of any array must be greater than 1.
Error C5	DOClass <identifier> undefined in DDF file	The DOClass named <identifier> is declared in the DDL, but not in the DDF. DOClasses must be declared in both files.
Error C7	<identifier> is incompatible with <identifier>	The update qualifiers chosen are incompatible with each other. For details refer to Table 3.1: Net-Z update qualifiers.
Error C8	Only one filter may be specified for update policies	Only one of the two update qualifier filters may be specified.
Error C9	Filters are incompatible with 'constant' for update policies	A dataset cannot be specified as constant when an update filter is also specified.
Error C11	Title is invalid	The application title must be included within quotation marks.

Code	Message	Reason
Error C12	Title must be specified in DDF file	The application title was not specified in the DDF. It is essential to do so.
Error C13	Title defined twice	The application title has been defined twice.
Error C14	Invalid keyword <identifier>	The keyword <identifier> is invalid. Refer to §8 for the correct grammar.
Error C15	Dataset <identifier> already declared	Two datasets cannot have the same name.
Error C16	DOClass <identifier> already declared	Two DOClasses cannot have the same name.
Error C17	A dataset can not contain a container and an extrapolation_filter flag.	A dataset that contains a container can not be extrapolated.
Error C18	Dataset <identifier> can not be unreliable and larger than <max_size> bytes (-v for details)	A dataset sent over an unreliable channel can not contain more than 3584 bytes of data. Recompile using the -v switch to obtain more details about data usage for every dataset class.
Error C19	Redundant duplication space registration : <identifier>	A particular publisher, subscriber, or cell cannot be declared in the same duplication space more than once.
Error C20	<identifier> is already specified.	One of the RMC modifiers (`in`, `out`, or `in out`) was specified twice.
Error C21	Dataset <identifier> contains a type that cannot be extrapolated (only int, float and double can).	An extrapolated dataset may only contain the data types *int*, *float*, or *double*.
Error C22	A doclass can only inherit from another doclass.	A duplicated object class can only inherit from another duplicated object class.
Error C50	<identifier> near <identifier>	Syntax error.
Warning C100	Callbacks are obsolete, use selection iterators	Callbacks are now obsolete. Selection iterators are used to call methods locally.

6.2 Known Bugs

Bug #	Description
81	If a duplication master migrates at virtually the same time as when an action is invoked on the duplication master, the action may be called on the duplica.
82	If a fault occurs at virtually the same time as when an action is invoked on a duplication master, and this duplication master migrates, the action may not be called.
83	Net-Z is not perfectly stable when a combination of faults and migration occurs.
93	If too many updates are sent, for example if the error tolerance function is set to zero, the convergence algorithm does not behave correctly.
123	Since faults are not processed, after a call to TerminateGame the game size does not decrease.
128	If the dead reckoning defined with the error tolerance function depends on the distance, there is a memory leak when a station leaves the session.
136	If an action defined in the DDL is a CPP macro that expands to different values in the generated *DDL.cpp and in the user code, the error SYSTEMERROR_OBJDUP_UNDEFINED_CALLBACK is thrown. For example, if an action is named PlaySound, and if windows.h is included in the user code, then PlaySound expands to PlaySoundW or PlaySoundA, and the action cannot be called. This is not really a bug in Net-Z but a name clash that would occur in any system. This issue can be resolved in two ways: using action names that are not macros, or by making sure that the same include files are used in the user code and in the DDL generated code using the DDL compiler switch -p.
137	The appearance of the sample programs that contain graphics is poor if the screen colour depth is set to a value other than 16bit. This is a problem of the sample programs and may be avoided by setting the depth to 16bit before running these programs.
140	A Net-Z game cannot be joined by a station accessing internet using IP forwarding.
144	When the load balancing scenario in the Atlantis demo is run for an extended period of time, the positions of the duplicas of the cubes are not stable.
158	When a specific bandwidth is set using the method EmulationDevice::SetBandwidth, the bandwidth emulated may be less than that set.
207	If you use a Pocket PC device in its cradle the device will not be able to join a session. This is a bug inherent to the Pocket PC device and not to Net-Z.

226 When an object emigrates, an Update may not be called by the new duplication master on the demoted duplication master. This is a timing problem because for a short period of time two duplication masters exist.

317 A game cannot be joined via a transitive join.

377 2 processes are running on the same machine, with Process 1 listening on the default port (9103) and Process 2 listening on a dynamic port. If the 2 processes do not know each other then when Process 1 leaves the session, Process 2 will not then listen on the default port.

378 In some cases, when a station quits a session a duplica of the Station object may remain in the session.

391 If a RMC is called inside another RMC, a deadlock may occur.

392 When hierarchical messaging is invoked and a station joins a session at the same time as a MulticastGroup is migrating, the station may not be added to the MulticastGroup.

7 GLOSSARY

Action
Actions are user-implemented methods that are called remotely on duplicated objects within the same class. They are the easiest way to call a method on an object to ensure that the method is executed on all duplicas of the object.

AI
Artificial Intelligence

API
Application Programming Interface

Application
A software program.

Bandwidth
The bandwidth of a network is given by the number of bits that can be transmitted over the network in a certain period of time.

Cell
A cell is a duplicated object that is part of a duplication space that enables the matching computations for the PSMatch function to be distributed between a number of stations. The CellMatch function determines if a particular publisher or subscriber object should be discovered by the cell.

CellMatch
The CellMatch function is a user-defined function that determines the conditions under which a particular cell will discover a particular publisher or subscriber duplicated object.

DDF
Duplicated object class Definition File. This file defines the global application information and the duplicated object classes that belong to the application.

DDL
Data Definition Language. The Net-Z Data Definition Language describes the data structures and session objects that are required to be managed by Net-Z.

DHCP
Dynamic Host Configuration Protocol.

Dataset
A dataset describes data relating to a duplicated object that needs to be duplicated to the object's duplicas.

Duplica

A duplica is a copy of a duplicated object that is controlled by a remote station. Its datasets are updated remotely, automatically, and transparently by Net-Z whenever the duplication master performs an update.

Duplication master
A duplication master is a duplicated object that is controlled by the local station. Only a duplication master can update the values of its datasets.

Duplication role
A duplicated object may be either a duplication master or a duplica.

Duplication space
Duplication spaces are the mechanism used by Net-Z to enable a game to be scaled as they ensure that only the objects that a station requires are duplicated to that station. The PSMatch function determines if within the duplication space a particular duplicated object discovers another duplicated object.

Emigration
Emigration is the process via which a station reassigns control of a duplicated object to another station.

Fault tolerance
Fault tolerance is the ability of a program to recover from faults with little or no effect on the ongoing session.

Global duplicated object
A global duplicated object is known globally across the session and is guaranteed to be duplicated to all stations participating in the session.

GUI
Graphical User Interface

GUID
Global Universal ID

Immigration
Immigration is the process through which a station accepts control of a duplicated object that was previously controlled by another station.

IP
Internet Protocol

Iterator
Iterators are used to access the lists of all duplication masters, duplicas, and duplicated objects that are kept on each station that is part of the session.

Jitter
Jitter is the variation in latency.

LAN
Local Area Network

Latency
Latency is the time lag before a message reaches its destination.

Load balancing
Load balancing is the process whereby the total workload of a system is distributed across all the stations connected to the network.

Migration
Migration is the process via which control of a duplicated object is transferred to another station.

Multicasted updates
A hierarchical arrangement of MulticastGroup objects is used to distribute the message sent from a duplication master to its duplicas rather than the duplication master sending a message directly to each of its duplicas.

PSMatch
The PSMatch function is a user-defined function that determines the conditions under which a particular subscriber will discover a particular publisher.

Publisher
A publisher is a duplicated object that is declared in a duplication space that can be discovered by subscriber duplicated objects in that duplication space. The PSMatch function determines if a publisher will be discovered by a particular subscriber.

Remote Method Call
Remote Method Calls (RMC) are user-implemented methods that are called on one specific instance of an object, which may be a duplication master or a duplica, and they may return a value. RMC Remote method call

SDK
Software Developer Kit

Session
A group of stations sharing a common environment via an application.

Session master
The session master is the station that initially created the session. The role of session master is fault-tolerant so if the initial session master leaves the session, then another master is automatically elected.

Subscriber
A subscriber is a duplicated object that is declared in a duplication space that can discover publisher duplicated objects in that duplication space. The PSMatch function determines if a subscriber will discover a particular publisher.

System callbacks
Methods specific to Net-Z that are called and performed locally on duplicated objects.

UML
Unified Modelling Language

Update policy
A dataset update policy consists of one or more update qualifiers and specifies the manner in which a dataset is updated.

Update qualifier
An update qualifier specifies the manner in which a dataset is updated.

URL
Uniform Resource Locator

Well-known duplicated object
A well-known duplicated object is a global object that is common to all stations that are participating in the session. It is created before the session is created and is guaranteed to be discovered by a station before the session commences on that station.

8 GRAMMAR

This chapter details the keywords, lexical conventions, and grammar used in the Net-Z DDL and DDF files.

Net-Z Keywords

action
bool
buffered
cell
char
constant
dataset
define
doclass
dohandle
double
dupspace
extrapolation_filter
float
gametitle
in
int
list
long
longlong
out
publisher
queue
reliable
short
string
subscriber
uchar
uint
ulong
ulonglong
unreliable
upon_request_filter
ushort
variable
vector
wellknown

Lexical Conventions

identifier [a-zA-Z_][0-9a-zA-Z_]*

stringtitle \”[^\n\”]*[\”\n]

decimal_constant [1-9][0-9]*

comments “//”[^\n]*

DDF Grammar

ddf_definition:
    ddf_list

ddf_list:
    sentence
    | *ddf_list sentence*

sentence:
    define_sentence
    | *wellknown_sentence*
    | *title_sentence*
    |

title_sentence:
    `gametitle` *title* ';'

title:
    `string`

wellknown_sentence:
    `wellknown identifier identifier` ';'

define_sentence:
    `define identifier`

DDL Grammar

ddl_definition:
    decl_list

decl_list:
    decl
    | *decl_list decl*

decl:
    dataset_decl
    | *doclass_decl*
    | *dupspace_decl*

dataset_decl:
`dataset identifier` '{' *dataset_member_list* '}' *dataset_update_specifier* ';'

dataset_member_list:
dataset_member
| *dataset_member_list dataset_member*

dataset_member:
attribute_decl ';'

attribute_decl:
simple_type `identifier` *array_size_opt*
| *string_type* `identifier` *array_size*
| *container_type* `'<'` *simple_type* `'>'` `identifier`

dataset_update_specifier:
dataset_updatespec_list
|

dataset_updatespec_list:
qualifier
| *dataset_updatespec_list* ',' *qualifier*

qualifier:
```
reliable
| unreliable
| extrapolation_filter
| upon_request_filter
| buffered
| variable
| constant
```

doclass_decl:
`doclass identifier` *base_doclass_opt* '{' *doclass_member_list* '}' ';'

dupspace_decl:
`dupspace identifier` ';'

base_doclass_opt:
':' `identifier`
|

doclass_member_list:
    doclass_member
    | *doclass_member_list doclass_member*

doclass_member:
    dataset_attribute_decl
    | *action_decl*
    | *rmc_decl*
    | *dupspace_reg_decl*
    |

dataset_attribute_decl:
    declared_dataset `identifier` ';'

dupspace_reg_decl:
    role_type `CONJUCTION INDENTIFIER` ';'

action_decl:
    `action` *action_method_name* '(' *action_parameter_list_opt* ')' ';'

action_method_name:
    `identifier`

action_parameter_list_opt:
    action_parameter_list
    |

action_parameter_list:
    parameter_decl
    | *action_parameter_list* ',' *parameter_decl*

parameter_decl:
    simple_type `identifier` *array_size_opt*
    | *string_type* `identifier` *array_size*
    | *container_type* `'<'` *simple_type* `'>'` `identifier`

rmc_decl_opt:
    rmc_decl
    |

rmc_decl:
    rmc_return_type rmc_method_name '(' rmc_parameter_list_opt ')' ';'

rmc_return_type:
    simple_type
    | `void`

rmc_method_name:
    `identifier`

rmc_parameter_list_opt:
    rmc_parameter_list
    |

rmc_parameter_list:
    rmc_parameter_decl
    | *rmc_parameter_list ',' rmc_parameter_decl*

rmc_parameter_decl:
    rmc_inoutspecifier_list simple_type `identifier` *array_size_opt*
    | *rmc_inoutspecifier_list string_type* `identifier` *array_size*
    | *rmc_inoutspecifier_list container_type '<' simple_type '>'* `identifier`

rmc_inoutspecifier_list:
    rmc_inoutspecifier
    | *rmc_inoutspecifier_list rmc_inoutspecifier*

rmc_inoutspecifier:
    IN
    | *OUT*
    |

array_size:
    `'[' int_value ']'`

array_size_opt:
    array_size
    |

simple_type:

```
char
| uchar
| short
| ushort
| long
| ulong
| longlong
| ulonglong
| int
| uint
| float
| double
| bool
| dohandle
```

string_type:

```
string
```

role_type:

```
cell
publisher
subscriber
```

conjunction:

```
in
```

container_type:

```
list
| queue
| vector
```

declared_dataset:

```
identifier
```

9 TECHNICAL SUPPORT

Proksim Software is dedicated to providing you with the best networking tools for online games. If you have any queries relating to Net-Z please contact us.

The Proksim Software website is regularly updated with the latest information on Net-Z and our other products. A searchable Knowledge Base is also available and is a good place to start if you have any questions about any of Proksim's products. Check out our web site frequently to stay up-to-date with our latest developments at www.proksim.com.

We believe that gathering developer feedback and reacting to their concerns is the best way to improve our product. We will pay attention to every request and do our best to satisfy your needs. If there are additional features you would like to see in Net-Z, please contact us.

We strive to ship the highest quality software however, you may encounter some glitches or bugs in our programs. Please tell us of any problems you encounter and include sufficient details to enable us to reproduce and fix them. Bug reports can be submitted via the Bug Report Form available in the Knowledge Base.

9.1 Contact Details

For assistance with problems or to send reports of bugs, contact us and ask for Proksim Software Product Support.

Email	support@proksim.com
Fax	(514)-940-9091
Telephone	(514)-940-9090

10 INDEX

A

Actions, 10, 17, 53, 111, 131
- Argument mapping, 54
- Arguments, 17

AddToStore operation, 86

Application title, 18

ApproveEmigration, 56, 59

ApproveFaultRecovery, 59

B

BufferRefresh operation, 90

Bugs, 129

C

Call contexts, 81
- Flags, 81
- Outcomes, 82
- States, 82

Cell, 10, 42, 131

Cell match function, 42, 46, 131. See also Match operation

Chrono, 94

Class generation, 20

Compiler errors, 127

Compiler output, 20

Compiler usage, 19

Compression. See Plug-ins

ComputeDistance, 64, 66

Constant extrapolation error. See Dead reckoning

Containers, 37. See also Datasets
- Update mode, 38

Continuity breaks, 79
- Update model, 79

Convergence angle, 76

Create object, 31. See also AddToStoreOperation

Create session, 18, 29
- Non-networked session, 29

Current extrapolation error. See Dead reckoning

D

Data compression. See Plug-ins

Data encryption. See Plug-ins

Data propagation, 37

Datasets, 10, 11, 36. See also Update qualifier, Update policy
- Containers, 13
- Data types, 11

DDF file syntax, 18

DDL compiler usage, 19
- Compiler options, 19

DDL file syntax, 9
- Actions, 17
- Comments, 9

Datasets, 11
Duplicated objects, 9
Duplication spaces, 13
Remote method calls, 16
Dead reckoning, 60
Constant extrapolation error, 62
Current extrapolation error, 62, 64
Distance calculation, 66
Distance-based extrapolation error, 63
Error tolerance, 63, 64
Linear, 65
Quadratic, 65
Fine-tuning, 75
Continuity breaks, 79
Dataset updates, 75
Local corrections, 80
Postdate updates, 78
Setting the observer, 66
Delegation, 26, 114, 115
Delete object, 34. See also RemoveFromStore operation
Developer responsibilities, 27
DOHandle, 33
WKHandle, 33
Duplica, 32, 131
Duplicated object classes, 9, 31
Duplication master, 32, 132
Duplication role, 132
Duplication spaces, 13, 40, 132. See also Cell, Cell match function, PS match munction, Publisher, Subscriber
Calling match functions, 48
Multiple duplication spaces, 49

E

Emulation device, 97
Encryption. See Plug-ins
EnumerateLANSessions, 29, 109
Error tolerance. See Dead reckoning
Errors. See Compiler errors, system errors
Extrapolation error. See Dead reckoning
Extrapolation filter, 14, 61, 115

F

Fault tolerance, 59, 132. See also FaultRecoveryOperation
FaultRecovery operation, 89
FetchContext. See Call contexts
File Syntax
DDF, 18
DDL, 9
FindLANSession, 29
Fine-tuning. See Dead reckoning, Performance fine-tuning

G

Global object, 10, 15, 32, 132
Grammar, 135
DDF, 136
DDL, 136
Keywords, 135
Lexical conventions, 135

H

Hierarchical messaging, 39

I

Inheritance, 21, 110
Iterators, 83, 132

J

Join session, 30

L

Load balancing, 55, 132. See also RoleChangeOperation, MasterIDChangeOperation
    Objects, 56
    Sub classing, 57
Local clock, 92
Local corrections, 80
Local station, 31

M

MasterIDChange operation, 88
Match operation, 90
Member list, 37
Member queue, 37
Member vector, 37
Message routing, 38
MigrationContext. See Call contexts
Multicasted updates, 39, 132

N

Network configuration options, 125

O

Object duplication, 32
    Create object, 32
    Duplication spaces. See Duplication spaces
    Fetch object, 32
    Global object. See Global object
Observer. See Dead reckoning
Operation denial, 91
Operation locks, 90
Operation sequences, 85
OperationBegin, 84
OperationEnd, 84

P

Pacer, 94
Peer station, 31
Performance fine-tuning, 96
    Emulation device, 97
    Performance counters, 96
        Dataset, 97
        Transport, 96
Platforms, 2
Plug-ins, 95
Postdate updates, 78
PS match function, 41, 43, 133. See also Match operation
Publish object, 31
Publisher, 10, 40, 133

Q

Quality factor, 71

R

Referencing objects, 33

Refresh, 14, 38. See also BufferRefresh operation, Refresh operation

Refresh operation, 89

Remote method calls, 10, 16, 51, 133

- Argument mapping, 52
- Arguments, 16
- Context. See Call contexts
- Modifiers, 16

RemoveFromStore operation, 87

RMC. See Remote method calls

RMCContext. See Call contexts

RoleChange operation, 88

S

Samples, 101

- ActionDungeonZ, 111
- Basic3DSample, 115
- BasicSample, 103
- Delegation, 114
- DungeonZ, 111
- EnumerateLANSessions, 109
- Inheritance, 110
- Joining a session, 102
- RoomZ, 116
- Running a sample, 102
- SphereZ, 119

Scalability, 123

Session clock, 92

- Fine-tuning synchronization, 92

Session description, 29, 30

Session management, 29

Session master, 133

Session name, 18, 29

Software developer kit, 1

Station, 31

- State, 31

Subscriber, 10, 40, 133

System callbacks, 133. See also ApproveFaultRecovery, ApproveEmigration, OperationBegin, OperationEnd

System errors, 98

System operations, 84

- Operation locks, 90
- Operation sequences, 85
  - AddToStore, 86
  - BufferRefresh, 90
  - FaultRecovery, 89
  - MasterIDChange, 88
  - Match, 90
  - Refresh, 89
  - RemoveFromStore, 87
  - RoleChange, 88

System requirements, 2

System state, 31

System traces, 85, 99

T

Traces. See System traces

Tracking angle, 75

Transport protocol. See Plug-ins

U

Update, 37
Update policy, 12, 133
Update qualifiers, 12, 14, 133
User-defined contexts, 83, 84

W

Well-known objects, 18, 35, 133